

Numerical Methods in Optimization

Complete Course in English

Zakari HACHEMI

Based on Pierre Gruet's course - Numerical Methods - MMMEF
2025-2026

Contents

1	Introduction to Numerical Methods for Optimization	1
1.1	Motivations and introductory example	1
1.2	General formulation of an optimization problem	2
1.3	Classification of optimization problems	3
1.3.1	Unconstrained or constrained	3
1.3.2	Continuous, discrete and stochastic optimization	3
1.3.3	Linear, quadratic, nonlinear problems	3
1.3.4	Convexity, concavity and convex programs	4
1.3.5	Penalty methods	4
1.4	General principle of optimization algorithms	4
1.5	Desirable properties of numerical methods	5
1.6	Stopping criteria	5
2	Theory of Unconstrained Optimization	7
2.1	Introduction	7
2.1.1	General Problem	7
2.1.2	Reformulation	7
2.2	Identifying Minimizers	8
2.2.1	Taylor's Theorem	8
2.2.2	First-Order Necessary Condition	9
2.2.3	Second-Order Necessary Condition	10
2.2.4	Second-Order Sufficient Condition	10
2.2.5	Convex Case	11
2.3	Iteration Strategies	11
2.3.1	Trust Region Methods	12
2.3.2	Line Search Methods	12
2.4	Search Directions for Line Search	16
2.4.1	Gradient Descent (steepest descent)	16
2.4.2	Newton Direction	18
2.4.3	Characterization of Convergence	18
2.4.4	Convergence Rate of Steepest Descent	20
2.4.5	Newton's Method: quadratic convergence	22
2.4.6	Hessian Modification for Newton	23
2.5	Practice Exercises (Nocedal & Wright)	25
3	Conjugate Gradient Methods	28
3.1	Linear Conjugate Gradient	28
3.1.1	Associated Quadratic Problem	28
3.1.2	A-Conjugate Directions	29
3.1.3	Conjugate Direction Method	29
3.1.4	Linear Conjugate Gradient Algorithm	29

3.1.5	Error Bound and Conditioning	30
3.2	Geometric Interpretation	30
3.2.1	Level sets of a quadratic function	30
3.3	Comparison with Gradient Descent	31
3.4	Preconditioning	31
3.5	Nonlinear Conjugate Gradient	31
3.6	Summary	32
3.6.1	Properties of Iterates and Krylov Subspaces	32
3.6.2	Derivation of the efficient version (Standard CG)	32
3.7	Rate of Convergence	33
3.7.1	Illustration: Effect of eigenvalue distribution	33
3.7.2	Comparison of convergence rates	34
3.8	Preconditioning	34
3.8.1	General Idea	34
3.8.2	Choosing the preconditioning matrix	35
3.8.3	Practical Implementation	35
3.8.4	Example: Incomplete Cholesky Preconditioning	35
4	Nonlinear Conjugate Gradient Methods	37
4.1	Introduction	37
4.2	Fletcher-Reeves Method	37
4.2.1	1. Line Search	37
4.2.2	2. Substituting the residual with the gradient	37
4.2.3	Formula for β_{FR}	37
4.2.4	Illustration on the Rosenbrock Function	38
4.2.5	Global Descent Property	39
4.3	Polak-Ribière Method	41
4.3.1	Definition and Properties	41
4.3.2	β^{PR+} Variant	41
4.3.3	Practical Remarks and Convergence Rate	41
4.4	Global Convergence Analysis	41
4.4.1	Assumptions (Assumptions A)	41
4.4.2	Al-Baali's Theorem	42
5	Quasi-Newton Methods	43
5.1	Introduction	43
5.2	The BFGS Method	43
5.3	The Davidon-Fletcher-Powell (DFP) Update Formula	43
5.3.1	The Secant Equation	44
5.3.2	Curvature Condition	45
5.3.3	Construction of the DFP Update	45
5.3.4	Expression for the Inverse (Sherman-Morrison-Woodbury Formula)	46
5.4	The BFGS Approach	46
5.5	Limited-Memory Quasi-Newton Methods (L-BFGS)	47
5.6	Remarks and Properties	48
5.7	The Broyden Class	48
5.8	Properties on Quadratic Functions	49
5.9	Convergence Analysis	49
5.9.1	Global Convergence	49
5.9.2	Superlinear Convergence of BFGS	50

6	Large-Scale Unconstrained Optimization Problems	51
6.1	Introduction	51
6.2	Inexact Newton Methods	51
6.2.1	Local Convergence	51
6.2.2	Rate of Convergence	52
6.2.3	Newton-CG Methods with Line Search	52
6.3	Limited-Memory Quasi-Newton Methods (L-BFGS)	52
6.3.1	Review of BFGS	53
6.3.2	Principles of L-BFGS	53
7	Derivative-Free Optimization	55
7.1	Introduction	55
7.2	Some Possible Solutions	55
7.3	The Problem of Noise	55
7.3.1	Noise Level and Approximation Error	56
7.3.2	Example: Stochastic Noise	56
7.4	Model-based methods	57
7.4.1	General Idea	57
7.4.2	Building the model	57
7.4.3	Update	58
7.4.4	Initialization and Geometry	58
7.5	Coordinate or pattern-search methods	59
7.5.1	Coordinate search	59
7.5.2	Pattern-search methods	60
7.6	Nelder-Mead Simplex Method	62
7.6.1	General Principle	62
7.6.2	Notation and Ordering	62
7.6.3	Centroid	63
7.6.4	Candidate Points	63
7.6.5	Remarks	64
7.6.6	Algorithm 15: Nelder-Mead Method	65
7.6.7	Illustration on the Rosenbrock Function	66
8	Theory of Constrained Optimization	67
8.1	General Problem	67
8.2	Feasible Set	67
8.3	Local Solutions	67
8.4	Review: Smooth/Nonsmooth Problem Equivalence	68
8.5	Linear Programming: The Simplex Method	69
8.5.1	Standard Form	69
8.5.2	Basic and Nonbasic Variables	69
8.5.3	Algorithm 16: One Step of the Simplex Method	70
8.6	Practice Exercises (Nocedal & Wright)	71
9	Optimality Conditions	73
9.1	Introduction and Definitions	73
9.2	Examples	74
9.2.1	One Equality Constraint	74
9.2.2	One Inequality Constraint	77
9.2.3	Two Inequality Constraints	80
9.3	Tangent Cone and Constraint Qualifications	82
9.3.1	Definitions	82

9.3.2	Linear Independence Constraint Qualification (LICQ)	84
9.4	Optimality Conditions	84
9.4.1	The Lagrangian Function	85
9.4.2	KKT Conditions	85
9.4.3	Second-Order Conditions	85
9.5	Duality	88
9.5.1	Primal Problem	88
9.5.2	Dual Function and Dual Problem	88
9.5.3	Example	89
9.5.4	Duality Properties and Theorems	89
9.5.5	Example: Duality in Linear Programming	90
10	Linear Programming: The Simplex Method	92
10.1	Optimality and Duality	92
10.1.1	Optimality of the Primal Problem	92
10.1.2	The Dual Problem	92
10.2	Geometry of the Feasible Set	93
10.3	Overview of the Simplex Method	93
10.4	Implementation and Practical Aspects	94
10.4.1	Linear Algebra and Revised Simplex	94
10.4.2	Degeneracy and Cycling	94
10.4.3	Initialization (Phase I and Phase II)	94
11	Quadratic Programming	95
11.1	Introduction	95
11.2	QP with Equality Constraints	95
11.2.1	Properties	95
11.2.2	Direct Solution of the KKT System	97
11.2.3	Iterative Solution of the KKT System	100
11.3	Problems with Inequality Constraints	101
11.3.1	Properties	101
11.4	Practice Exercises (Nocedal & Wright)	107
12	Penalty and Augmented Lagrangian Methods	111
12.1	The Quadratic Penalty Method	111
12.1.1	Motivation	111
12.1.2	Convergence of the Method (with Equality Constraints)	113
12.2	Nonsmooth Penalty Functions (l^1)	115
12.2.1	Definition and Properties	115
12.2.2	Characterization of Behavior	115
12.3	Practical Approach: Modified l^1 Model	116
12.4	Augmented Lagrangian Methods	117
12.4.1	Definition and Motivation	117
12.4.2	Algorithm (Method of Multipliers)	117
12.5	Augmented Lagrangian Method	118
12.5.1	Properties of the equality-constrained case	118
12.5.2	Practical Augmented Lagrangian Methods	118
12.6	Practice Exercises (Nocedal & Wright)	119

Chapter 1

Introduction to Numerical Methods for Optimization

1.1 Motivations and introductory example

Optimization consists of choosing, among a set of possible decisions, the one that minimizes (or maximizes) a quantity called the *objective function*. In this course, we mainly focus on *minimization* problems, although maximization problems can be treated equivalently by considering the opposite of the function.

Optimization problems encountered in practice can be:

- **continuous**, when variables take values in $\mathbb{R}^n$;
- **discrete**, when some variables must be integers (integer programming);
- **mixed** (MIP: Mixed Integer Programming), when some variables are continuous and others are discrete.

Discrete or mixed problems are generally harder to solve than continuous problems and will not be studied in detail in this course.

We also encounter problems where parameters are random. This leads to:

- **stochastic optimization**, where we minimize an expectation;
- **chance constraints** (chance-constrained), where certain constraints must be satisfied with high probability;
- **robust optimization**, where constraints must be satisfied for all admissible realizations of an uncertainty set.

These frameworks are essential in fields such as finance, energy management or systems engineering, but they fall outside the immediate scope of this chapter.

Example 1.1 (Quadratic problem with linear constraints). We consider the problem

$$\min_{x \in \mathbb{R}^2} f(x) := (x_1 - 1)^2 + 2(x_2 + 3)^2 \quad (1.1)$$

under the constraints

$$x_1 \geq 2, \quad x_1 - x_2 = 4. \quad (1.2)$$

The objective function f measures the “badness” of a vector $x = (x_1, x_2)$. The smaller $f(x)$ is, the more satisfactory the solution is. The constraints impose that the admissible solutions live on a line and in a half-plane. The level curves of f are ellipses centered at $(1, -3)$.

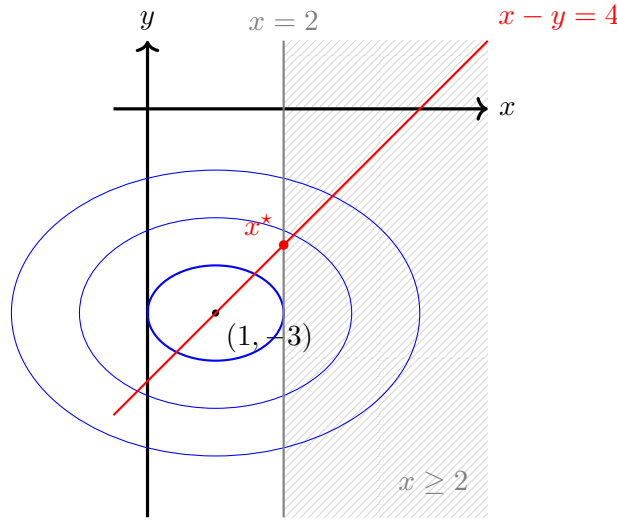


Figure 1.1: Illustration of the introductory quadratic problem.

In Figure 1.1, the **blue ellipses** represent the level curves of the objective function f : the smaller the ellipse, the lower the value of f . The **center** of the ellipses, located at $(1, -3)$, corresponds to the unconstrained minimum of f . The **hatched area** represents the feasible region $x \geq 2$. The **vertical gray line** $x = 2$ materializes the boundary of this constraint, and the **red line** represents the equality constraint $x - y = 4$. The point x^* , intersection of the red line with the boundary of the feasible region, is the **optimal solution** of the constrained problem.

This problem illustrates three essential components:

- the **objective function** $f : \mathbb{R}^n \rightarrow \mathbb{R}$;
- the **set of constraints**, defining the feasible points;
- the **optimal solution** x^* , minimizing f among the feasible points.

1.2 General formulation of an optimization problem

Definition 1.2 (Optimization problem). An *optimization problem* is the search for a vector $x \in \mathbb{R}^n$ minimizing a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ under equality and/or inequality constraints, that is:

$$\begin{aligned} & \min_{x \in \mathbb{R}^n} f(x) \\ & \text{under the constraints} \quad c_i(x) \geq 0, \quad i \in I, \\ & \quad \quad \quad c_j(x) = 0, \quad j \in E. \end{aligned} \tag{1.3}$$

Definition 1.3 (Feasible set). The feasible set is

$$\mathcal{X} = \{x \in \mathbb{R}^n : c_i(x) \geq 0 \ \forall i \in I, \ c_j(x) = 0 \ \forall j \in E\}.$$

Any point in $\mathcal{X}$ is called *feasible* (or *admissible*).

Definition 1.4 (Optimal solution). A point $x^* \in \mathcal{X}$ is a *global optimal solution* if

$$f(x^*) \leq f(x), \quad \forall x \in \mathcal{X}.$$

We speak of a *local minimum* if this property only holds in a neighborhood of x^* .

Remark 1.5. In practice, it is rare to determine x^* exactly. Numerical methods produce a sequence (x_k) whose gradient values and/or function values decrease towards zero.

1.3 Classification of optimization problems

1.3.1 Unconstrained or constrained

- **Unconstrained optimization:**

$$\min_{x \in \mathbb{R}^n} f(x).$$

- **Constrained optimization:**

$$x \in \mathcal{X} \subsetneq \mathbb{R}^n.$$

A common special case is that of box constraints:

$$\ell_i \leq x_i \leq u_i, \quad i = 1, \dots, n.$$

1.3.2 Continuous, discrete and stochastic optimization

Integer linear programming. When certain variables x_i must be integers, the problem becomes *discrete* (NP-hard in general).

Mixed problems (MIP). Combination of continuous and discrete variables.

Stochastic optimization. Presence of random parameters:

- minimization of an expectation;
- constraints satisfied with probability $\geq p$ (chance-constrained);
- satisfaction for all possible scenarios (robust optimization).

1.3.3 Linear, quadratic, nonlinear problems

- **Linear Programming (LP):**

$$f(x) = c^\top x, \quad c_i(x) = a_i^\top x - b_i.$$

- **Quadratic Programming (QP):**

$$f(x) = \frac{1}{2}x^\top Qx + c^\top x,$$

where Q is symmetric.

- **Nonlinear Programming (NLP):** f and/or the constraints are nonlinear.

1.3.4 Convexity, concavity and convex programs

Definition 1.6 (Convex set). A set $\mathcal{C}$ is convex if

$$\theta x + (1 - \theta)y \in \mathcal{C}, \quad \forall x, y \in \mathcal{C}, \theta \in [0, 1].$$

Definition 1.7 (Convex function). A function f is convex if

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y).$$

It is *strictly convex* if the inequality is strict for $x \neq y$. It is *concave* if $-f$ is convex.

Definition 1.8 (Convex program). An optimization problem is called *convex* when:

- the objective function f is convex;
- the equality constraints are linear;
- the inequality constraints $c_i(x) \geq 0$ are concave.

In this framework, any local minimum is automatically a global minimum.

1.3.5 Penalty methods

Certain constraints can be integrated into the objective function. For example:

$$\min_{x \in \mathbb{R}^2} (x_1 - 1)^2 + (x_2 + 3)^2 \quad \text{subject to} \quad x_1 + x_2 = 5$$

can be reformulated as an unconstrained problem:

$$\min_{x \in \mathbb{R}^2} (x_1 - 1)^2 + (x_2 + 3)^2 + \frac{\eta}{2} (x_1 + x_2 - 5)^2, \quad \eta > 0.$$

We will discuss this again in the last chapter on penalization, but this second problem is only equivalent to the first for $\eta \rightarrow \infty$. Precisely, its solution is

$$x_1 = \frac{9\eta + 2}{2\eta + 2}, \quad x_2 = \frac{\eta - 6}{2\eta + 2}.$$

Alternatively, we can consider the problem

$$\min (x_1 - 1)^2 + (x_2 + 3)^2 + \eta |x_1 + x_2 - 5|,$$

for which we will indicate in the last chapter that the solution coincides with that of the initial constrained problem as soon as $\eta > 7$.

1.4 General principle of optimization algorithms

Numerical optimization methods are *iterative*: they build a sequence (x_k) converging towards an optimal (or stationary) solution. The general scheme is:

$$x_{k+1} = x_k + \alpha_k p_k,$$

where p_k is a search direction and $\alpha_k > 0$ is a step length.

Input: an initial point x_0 .
For $k = 0, 1, 2, \dots$ **while** the stopping criterion is not satisfied:

1. Choose a direction p_k (for example $p_k = -\nabla f(x_k)$).
2. Choose a step length $\alpha_k > 0$ (line search, trust region, etc.).
3. Update $x_{k+1} = x_k + \alpha_k p_k$.

End For

For unconstrained methods, the algorithm relies on the following information:

- only $f(x_k)$ (“black box” methods);
- the gradient $\nabla f(x_k)$ (gradient methods);
- the Hessian $\nabla^2 f(x_k)$ (Newton-type methods);
- approximations of the Hessian (quasi-Newton methods).

1.5 Desirable properties of numerical methods

An efficient optimization method must satisfy several criteria:

- **Robustness:** good behavior for different problems and starting points;
- **Efficiency:** reasonable cost in time and memory;
- **Accuracy:** low sensitivity to rounding errors and calculation inaccuracies;
- **Numerical stability:** avoiding the amplification of errors;
- **Reliability:** avoiding stagnations or chaotic behaviors.

1.6 Stopping criteria

An optimization algorithm generally stops when one of the following criteria is reached:

- **small gradient norm:**

$$\|\nabla f(x_k)\| \leq \varepsilon_{\text{grad}};$$

- **small variation in the solution:**

$$\|x_{k+1} - x_k\| \leq \varepsilon_{\text{step}};$$

- **small variation in the function:**

$$|f(x_{k+1}) - f(x_k)| \leq \varepsilon_f;$$

- **maximum number of iterations reached.**

This first chapter establishes the general framework of numerical optimization: formulation, classification, convexity, broad typology of problems, algorithm structure, and essential properties. The following chapters will be devoted to the detailed study of solving methods,

Checkpoint : Did you understand the essentials ?

- **Vocabulary:** Can you define: objective function, feasible set, local vs global solution?
- **Structure:** What are the ingredients of a descent algorithm (direction + step)?
- **Convexity:** Why is it important? (A local min is global).
- **Stopping criteria:** Cite two conditions to stop an algorithm.
- **Positive definite Hessian:** $\nabla^2 f(x^*)$ is positive definite if $\eta^\top \nabla^2 f(x^*) \eta > 0$ for all vectors $\eta \neq 0$.

beginning with unconstrained optimization.

Chapter 2

Theory of Unconstrained Optimization

2.1 Introduction

2.1.1 General Problem

We consider the unconstrained optimization problem

$$\min_{x \in \mathbb{R}^n} f(x), \quad (2.1)$$

where $n \geq 1$ and $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is assumed to be sufficiently smooth (typically $\mathcal{C}^1$ or $\mathcal{C}^2$).

Remark 2.1. We leave aside nonsmooth optimization: it can be difficult to analyze the local behavior of f in the neighborhood of x_k . For a general presentation, see for example Hiriart-Urruty and Lemaréchal.

2.1.2 Reformulation

It is sometimes possible to reformulate a nonsmooth problem into a smooth problem by increasing the dimension.

Let $f_1, \dots, f_m : \mathbb{R}^n \rightarrow \mathbb{R}$ be smooth functions, and define

$$f(x) = \max_{j=1, \dots, m} f_j(x). \quad (2.2)$$

Then the problem

$$(P) : \min_{x \in \mathbb{R}^n} f(x) = \min_{x \in \mathbb{R}^n} \max_{1 \leq j \leq m} f_j(x) \quad (2.3)$$

is a nonsmooth problem constructed from smooth functions.

This problem can be rewritten as

$$\min_{(x,t) \in \mathbb{R}^{n+1}} t \quad \text{under the constraints} \quad t - f_j(x) \geq 0, \quad j = 1, \dots, m, \quad (2.4)$$

the latter condition meaning

$$f_j(x) \leq t, \quad j = 1, \dots, m. \quad (2.5)$$

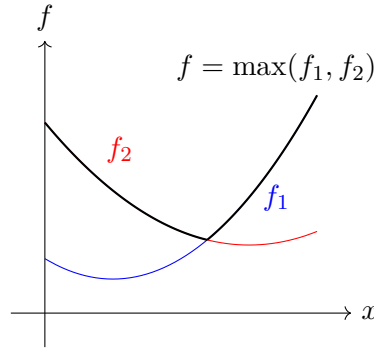


Figure 2.1: Example of a nonsmooth function $f(x) = \max(f_1(x), f_2(x))$.

In Figure 2.1, the thick black curve represents the function $f(x) = \max(f_1(x), f_2(x))$. It follows the upper portion of each curve: f_2 (red) when x is small, and f_1 (blue) when x is large. The intersection point marks a break in the slope, making f non-differentiable at this point (nonsmooth function).

2.2 Identifying Minimizers

Definitions

Definition 2.2. Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$. A point $x^* \in \mathbb{R}^n$ is

- a **global minimizer** of f if

$$f(x^*) \leq f(x), \quad \forall x \in \mathbb{R}^n;$$

- a **local minimizer (weak)** if there exists a neighborhood V of x^* such that

$$f(x^*) \leq f(x), \quad \forall x \in V;$$

- a **strict local minimizer** if there exists a neighborhood V of x^* such that

$$f(x^*) < f(x), \quad \forall x \in V \setminus \{x^*\};$$

- an **isolated local minimizer** if there exists a neighborhood V of x^* such that x^* is the unique local minimizer of f in V .

2.2.1 Taylor's Theorem

Theorem 2.3 (Mean Value Theorem). Suppose that $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is continuously differentiable ($\mathcal{C}^1$). Then for all $x \in \mathbb{R}^n$ and all $\eta \in \mathbb{R}^n$, there exists $t \in (0, 1)$ such that

$$f(x + \eta) = f(x) + \nabla f(x + t\eta)^\top \eta. \quad (2.6)$$

Theorem 2.4 (Taylor Expansions). *If $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is twice continuously differentiable ($\mathcal{C}^2$), then for all $x, \eta \in \mathbb{R}^n$:*

1. *There exists $t_1 \in (0, 1)$ such that*

$$\nabla f(x + \eta) = \nabla f(x) + \nabla^2 f(x + t_1 \eta) \eta.$$

2. *There exists $t_2 \in (0, 1)$ such that*

$$f(x + \eta) = f(x) + \nabla f(x)^\top \eta + \frac{1}{2} \eta^\top \nabla^2 f(x + t_2 \eta) \eta.$$

Gradient and Hessian. For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the gradient at x is

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{pmatrix} \in \mathbb{R}^n.$$

The Hessian matrix of f at $x \in \mathbb{R}^n$ is

$$\nabla^2 f(x) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2}(x) & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n}(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1}(x) & \cdots & \frac{\partial^2 f}{\partial x_n^2}(x) \end{pmatrix} \in \mathbb{R}^{n \times n}.$$

For $x \in \mathbb{R}^n$, we also denote $x^\top = (x_1, \dots, x_n)$.

2.2.2 First-Order Necessary Condition

Theorem 2.5 (First-Order Necessary Condition). *Suppose that f is continuously differentiable in a neighborhood of x^* , and that x^* is a local minimizer of f . Then*

$$\nabla f(x^*) = 0. \tag{2.7}$$

Proof. We reason by contradiction. Suppose that $\nabla f(x^*) \neq 0$ and let $\eta = -\nabla f(x^*) \neq 0$. Since f is $\mathcal{C}^1$ in the neighborhood of x^* , we have

$$\eta^\top \nabla f(x^*) = -\|\nabla f(x^*)\|^2 < 0.$$

By the mean value theorem, there exists $t \in (0, T)$ such that

$$f(x^* + t\eta) = f(x^*) + t \nabla f(x^* + t'\eta)^\top \eta$$

for some $t' \in (0, t)$. For $t > 0$ small enough, the product $\nabla f(x^* + t'\eta)^\top \eta$ remains strictly negative by continuity. We then obtain

$$f(x^* + t\eta) < f(x^*),$$

and this holds for all $t \in [0, T]$: we have indeed captured a descent direction, and not just an isolated point. This contradicts the fact that x^* is a local minimizer. Thus, we must have $\nabla f(x^*) = 0$. $\square$

2.2.3 Second-Order Necessary Condition

Definition 2.6. A point x^* is a *stationary point* of f if

$$\nabla f(x^*) = 0.$$

Theorem 2.5 states that every local minimizer is a stationary point.

Theorem 2.7 (Second-Order Necessary Condition). *Suppose that f is twice continuously differentiable in a neighborhood of x^* , and that x^* is a local minimizer of f . Then*

$$\nabla f(x^*) = 0 \quad \text{and} \quad \nabla^2 f(x^*) \succeq 0,$$

that is

$$\forall \eta \in \mathbb{R}^n, \quad \eta^\top \nabla^2 f(x^*) \eta \geq 0.$$

Proof. We already know by Theorem 2.5 that $\nabla f(x^*) = 0$. Let us reason by contradiction for the Hessian. Suppose that $\nabla^2 f(x^*)$ is not positive semi-definite. Then there exists $\eta \in \mathbb{R}^n$ such that

$$\eta^\top \nabla^2 f(x^*) \eta < 0.$$

By continuity of $\nabla^2 f$ in the neighborhood of x^* , there exists $T > 0$ such that

$$\forall t \in [0, T], \quad \eta^\top \nabla^2 f(x^* + t\eta) \eta < 0.$$

In particular, for $t \in (0, T)$, the Taylor expansion gives

$$f(x^* + t\eta) = f(x^*) + t \nabla f(x^*)^\top \eta + \frac{t^2}{2} \eta^\top \nabla^2 f(x^* + \theta t\eta) \eta$$

for some $\theta \in (0, 1)$. Since $\nabla f(x^*) = 0$ and $\eta^\top \nabla^2 f(x^* + \theta t\eta) \eta < 0$, we obtain

$$f(x^* + t\eta) < f(x^*),$$

and this for all $t \in [0, T]$: we have a descent direction and not an isolated point. This contradicts the fact that x^* is a local minimizer. Therefore, $\nabla^2 f(x^*)$ must be positive semi-definite. $\square$

2.2.4 Second-Order Sufficient Condition

Theorem 2.8 (Second-Order Sufficient Condition). *Suppose that f is twice continuously differentiable on an open neighborhood of x^* , and that*

$$\nabla f(x^*) = 0, \quad \nabla^2 f(x^*) \succ 0,$$

that is

$$\forall \eta \in \mathbb{R}^n \setminus \{0\}, \quad \eta^\top \nabla^2 f(x^*) \eta > 0.$$

Then x^ is a strict local minimizer of f .*

Proof. The Hessian is continuous and positive definite at x^* . There thus exists $r > 0$ such that $\nabla^2 f(x) \succ 0$ for all x in the ball

$$D = \{x \in \mathbb{R}^n \mid \|x - x^*\| < r\}.$$

Let $\eta \in \mathbb{R}^n$ such that $\|\eta\| \leq r$. By Taylor's theorem 2.4, there exists $t \in (0, 1)$ such that

$$f(x^* + \eta) = f(x^*) + \eta^\top \nabla f(x^*) + \frac{1}{2} \eta^\top \nabla^2 f(x^* + t\eta) \eta.$$

Since $\nabla f(x^*) = 0$ and $\nabla^2 f(x^* + t\eta) \succ 0$, we deduce

$$f(x^* + \eta) > f(x^*)$$

as soon as $\eta \neq 0$ and $\|\eta\|$ is small enough. Thus x^* is a strict local minimizer. $\square$

2.2.5 Convex Case

Theorem 2.9 (Convex Case). *Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a convex function.*

1. *Every local minimizer of f is also a global minimizer.*
2. *If f is additionally differentiable, then every stationary point ($\nabla f(x^*) = 0$) is a global minimizer.*

Proof of point (1). Suppose that x^* is a local minimizer but not a global one. Then there exists $y \in \mathbb{R}^n$ such that

$$f(y) < f(x^*).$$

By convexity of f , for all $\lambda \in [0, 1]$ we have

$$f(\lambda x^* + (1 - \lambda)y) \leq \lambda f(x^*) + (1 - \lambda)f(y) < f(x^*) \quad (\lambda \neq 1).$$

Let V be a neighborhood of x^* . For $\lambda \in (0, 1)$ sufficiently small, the point $z = \lambda x^* + (1 - \lambda)y$ belongs to V and satisfies $f(z) < f(x^*)$, which contradicts the local minimality of x^* . $\square$

2.3 Iteration Strategies

Optimization algorithms build a sequence $(x_k)_{k \geq 0}$ such that

$$x_{k+1} = x_k + \alpha_k p_k,$$

where p_k is a search direction (generally a descent direction) and $\alpha_k > 0$ is a step length (displacement length).

In practice:

- we choose a starting point x_0 ,
- at iteration k , we use local information (values of f , of ∇f , or even $\nabla^2 f$) to define p_k and α_k ,
- we typically seek to ensure

$$f(x_{k+1}) < f(x_k),$$

or at least a decrease after several iterations:

$$f(x_{k+m}) < f(x_k).$$

Two large families of strategies exist:

- **trust region** methods,
- **line search** methods.

2.3.1 Trust Region Methods

At iteration k , we fix a radius $\Delta_k > 0$ defining the trust region around x_k :

$$\{x \in \mathbb{R}^n \mid \|x - x_k\| \leq \Delta_k\}.$$

Using Taylor's formula, we approximate f by a quadratic model

$$m_k(x) = f(x_k) + (x - x_k)^\top \nabla f(x_k) + \frac{1}{2}(x - x_k)^\top \nabla^2 f(x_k) (x - x_k). \quad (2.8)$$

We then choose x_{k+1} as a solution (or an approximation of the solution) of the trust region problem

$$x_{k+1} = \arg \min_x m_k(x) \quad \text{s.t.} \quad \|x - x_k\| \leq \Delta_k. \quad (2.9)$$

Letting $p_k = x_{k+1} - x_k$, this amounts to minimizing $m_k(p)$ under the constraint $\|p\| \leq \Delta_k$:

$$p_k = \arg \min_{\|p\| \leq \Delta_k} \left(f(x_k) + \nabla f(x_k)^\top p + \frac{1}{2} p^\top \nabla^2 f(x_k) p \right). \quad (2.10)$$

We introduce the ratio

$$\rho_k = \frac{f(x_k) - f(x_k + p_k)}{m_k(0) - m_k(p_k)}. \quad (2.11)$$

The typical acceptance rules are:

- if $\rho_k < 0$: the new iteration is rejected, and we reduce Δ_k ;
- if $0 < \rho_k$ is small: we accept x_{k+1} but keep Δ_k roughly unchanged;
- if $\rho_k \approx 1$: we accept x_{k+1} and increase Δ_k .

2.3.2 Line Search Methods

Line search methods take the form

$$x_{k+1} = x_k + \alpha_k p_k,$$

where p_k is a descent direction and α_k is adaptively chosen step length.

A frequent choice for the direction p_k is

$$p_k = -B_k \nabla f_k, \quad (2.12)$$

where B_k is a symmetric positive definite (or at least invertible) matrix and

$$f_k = f(x_k), \quad \nabla f_k = \nabla f(x_k), \quad \nabla^2 f_k = \nabla^2 f(x_k).$$

In this case, we have

$$p_k^\top \nabla f_k = -\nabla f_k^\top B_k \nabla f_k < 0, \quad (2.13)$$

since B_k is positive definite and $\nabla f_k \neq 0$. Thus p_k is indeed a descent direction. Furthermore

$$p_k^\top \nabla f_k = \|p_k\| \|\nabla f_k\| \cos(\angle(p_k, \nabla f_k)) < 0,$$

which implies that the angle between p_k and ∇f_k is strictly greater than $\frac{\pi}{2}$.

The basic idea is to choose α_k as

$$\alpha_k = \arg \min_{\alpha > 0} \varphi(\alpha), \quad \varphi(\alpha) = f(x_k + \alpha p_k),$$

but this one-dimensional problem can be expensive to solve exactly. We therefore search for an *acceptable* value α_k , satisfying sufficient decrease conditions.

Sufficient decrease condition (Armijo). We impose

$$f(x_k + \alpha p_k) \leq f(x_k) + c_1 \alpha \nabla f_k^\top p_k, \quad (2.14)$$

where $c_1 \in (0, 1)$ is a parameter (typically $c_1 = 10^{-4}$).

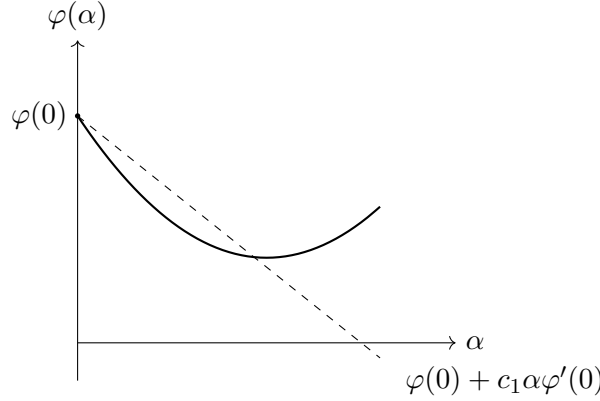


Figure 2.2: Sufficient decrease condition (Armijo).

Figure 2.2 illustrates the sufficient decrease condition: the curve of the function $\varphi(\alpha)$ must lie below the dotted line with equation $\varphi(0) + c_1 \alpha \varphi'(0)$. This guarantees that the function decreases sufficiently relative to its initial slope.

Remark 2.10. The Armijo condition is always satisfied for sufficiently small α . It ensures that the decrease of f is proportional to α and to the directional derivative $\nabla f_k^\top p_k$.

Definition 2.11 (Wolfe Conditions). We say that α satisfies the *Wolfe conditions* if

$$f(x_k + \alpha p_k) \leq f(x_k) + c_1 \alpha \nabla f_k^\top p_k, \quad (2.15)$$

$$\nabla f(x_k + \alpha p_k)^\top p_k \geq c_2 \nabla f_k^\top p_k, \quad (2.16)$$

for constants $0 < c_1 < c_2 < 1$. By denoting $\varphi(\alpha) = f(x_k + \alpha p_k)$, the second condition can also be written as $\varphi'(\alpha) \geq c_2 \varphi'(0)$.

Definition 2.12 (Strong Wolfe Conditions). The *strong Wolfe conditions* are obtained by replacing the previous curvature condition by

$$|\nabla f(x_k + \alpha p_k)^\top p_k| \leq -c_2 \nabla f_k^\top p_k, \quad (2.17)$$

that is

$$|\varphi'(\alpha)| \leq -c_2 \varphi'(0).$$

This excludes values of α for which the slope is too positive.

Remark 2.13 (Comparison standard vs strong Wolfe). The strong Wolfe condition reinforces the standard condition with an upper and lower bound.

- **Standard Wolfe:** $\varphi'(\alpha) \geq c_2 \varphi'(0)$. It forbids slopes that are too negative, but allows very positive slopes (overshooting the minimum).
- **Strong Wolfe:** $|\varphi'(\alpha)| \leq -c_2 \varphi'(0) \iff c_2 \varphi'(0) \leq \varphi'(\alpha) \leq -c_2 \varphi'(0)$. It also forbids slopes that are too positive, forcing α to stay in a neighborhood where the derivative is close to zero.

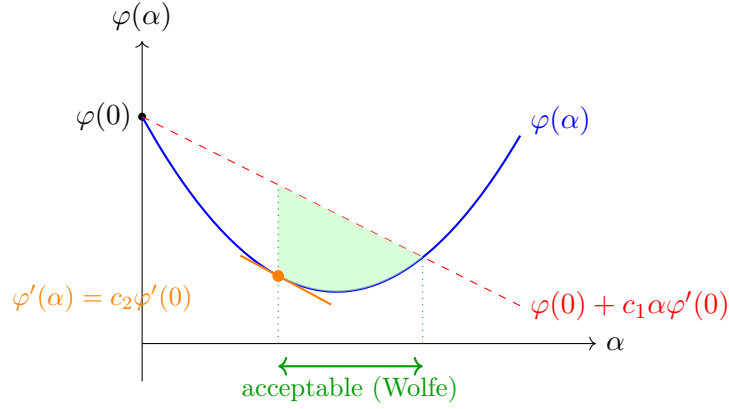


Figure 2.3: Wolfe conditions: the acceptable zone (green) satisfies both the sufficient decrease (Armijo) and the curvature condition.

In Figure 2.3, the **acceptable zone** (in green) is the interval of steps α which simultaneously satisfy the two Wolfe conditions: 1. The Armijo condition (sufficient decrease): the blue curve is below the red line. 2. The curvature condition: the slope of the curve (orange tangent) is greater than or equal to c_2 times the initial slope.

Remark 2.14. For methods of the Newton or quasi-Newton type, one often chooses $c_2 \approx 0.9$, while for nonlinear conjugate gradient methods, one tends to take $c_2 \approx 0.1$.

Lemma 2.15. Suppose that f is continuously differentiable ($\mathcal{C}^1$) and that p_k is a descent direction. Suppose that f is bounded from below along the ray

$$\{x_k + \alpha p_k \mid \alpha > 0\}.$$

Then, for any $0 < c_1 < c_2 < 1$, there exist intervals of values of α satisfying the Wolfe conditions (standard or strong).

Proof. Let $\varphi(\alpha) = f(x_k + \alpha p_k)$, and $\varphi'(0) = \nabla f_k^\top p_k < 0$. Since f is bounded from below on the ray, φ is as well.

For $c_1 \in (0, 1)$, consider the affine function

$$\ell(\alpha) = f(x_k) + c_1 \alpha \nabla f_k^\top p_k.$$

Since $\ell(\alpha) \rightarrow -\infty$ as $\alpha \rightarrow +\infty$, the graphs of φ and ℓ intersect. Let $\hat{\alpha} > 0$ be the smallest intersection point:

$$\varphi(\hat{\alpha}) = f(x_k + \hat{\alpha} p_k) = f(x_k) + c_1 \hat{\alpha} \nabla f_k^\top p_k.$$

The Armijo condition is therefore strictly verified for all $\alpha \in (0, \hat{\alpha})$.

By the mean value theorem, there exists $\bar{\alpha} \in (0, \hat{\alpha})$ such that

$$\varphi(\hat{\alpha}) - \varphi(0) = \hat{\alpha} \varphi'(\bar{\alpha}).$$

Combining with the definition of $\hat{\alpha}$, we obtain

$$c_1 \nabla f_k^\top p_k = \varphi'(\bar{\alpha}).$$

Since $c_1 < c_2$ and $\nabla f_k^\top p_k < 0$, it comes

$$c_2 \nabla f_k^\top p_k < \varphi'(\bar{\alpha}),$$

which is the Wolfe condition

$$\nabla f(x_k + \bar{\alpha}p_k)^\top p_k > c_2 \nabla f_k^\top p_k.$$

By continuity of φ' and φ , this condition and the Armijo condition remain valid in a small interval around $\bar{\alpha}$, which gives an interval of α satisfying the Wolfe conditions. Moreover, since $\varphi'(\bar{\alpha}) < 0$, we can also satisfy the strong Wolfe condition on a neighboring interval. $\square$

Line search by *backtracking*. A simple way to ensure the Armijo condition is the *backtracking line search*.

Algorithm 1: Line search with backtracking

- 1 Choose $\bar{\alpha} > 0$, $\rho \in (0, 1)$, $c \in (0, 1)$; Set $\alpha \leftarrow \bar{\alpha}$;
 - 2 **repeat**
 - 3 | $\alpha \leftarrow \rho\alpha$;
 - 4 **until** $f(x_k + \alpha p_k) \leq f(x_k) + c\alpha \nabla f_k^\top p_k$;
 - 5 Terminate with $\alpha_k = \alpha$.
-

Remark 2.16. In practice, we almost always start the line search with $\bar{\alpha} = 1$.

Line search algorithm with zoom. To satisfy the Wolfe conditions, a more sophisticated algorithm that uses a *zoom* procedure is used.

Algorithm 2: Line search algorithm

- 1 Set $\alpha_0 \leftarrow 0$, choose $\alpha_{\max} > 0$ and $\alpha_1 \in (0, \alpha_{\max})$;
 - 2 $i \leftarrow 1$;
 - 3 **repeat**
 - 4 | Evaluate $\phi(\alpha_i)$;
 - 5 | **if** $\phi(\alpha_i) > \phi(0) + c_1\alpha_i\phi'(0)$ **or** $[\phi(\alpha_i) \geq \phi(\alpha_{i-1})$ **and** $i > 1]$ **then**
 - 6 | | $\alpha_* \leftarrow \text{zoom}(\alpha_{i-1}, \alpha_i)$ **and stop**;
 - 7 | Evaluate $\phi'(\alpha_i)$;
 - 8 | **if** $|\phi'(\alpha_i)| \leq -c_2\phi'(0)$ **then**
 - 9 | | set $\alpha_* \leftarrow \alpha_i$ **and stop**;
 - 10 | **if** $\phi'(\alpha_i) \geq 0$ **then**
 - 11 | | set $\alpha_* \leftarrow \text{zoom}(\alpha_i, \alpha_{i-1})$ **and stop**;
 - 12 | Choose $\alpha_{i+1} \in (\alpha_i, \alpha_{\max})$;
 - 13 | $i \leftarrow i + 1$;
 - 14 **until**;
-

Algorithm 3: Zoom procedure

```

1 repeat
2   Interpolate to find a trial step length  $\alpha_j$  between  $\alpha_{lo}$  and  $\alpha_{hi}$ ;
3   Evaluate  $\phi(\alpha_j)$ ;
4   if  $\phi(\alpha_j) > \phi(0) + c_1\alpha_j\phi'(0)$  or  $\phi(\alpha_j) \geq \phi(\alpha_{lo})$  then
5      $\alpha_{hi} \leftarrow \alpha_j$ ;
6   else
7     Evaluate  $\phi'(\alpha_j)$ ;
8     if  $|\phi'(\alpha_j)| \leq -c_2\phi'(0)$  then
9       Set  $\alpha_* \leftarrow \alpha_j$  and stop;
10    if  $\phi'(\alpha_j)(\alpha_{hi} - \alpha_{lo}) \geq 0$  then
11       $\alpha_{hi} \leftarrow \alpha_{lo}$ ;
12     $\alpha_{lo} \leftarrow \alpha_j$ ;
13 until;

```

Remark 2.17. The function $\phi(\alpha) = f(x_k + \alpha p_k)$ and $\phi'(\alpha) = \nabla f(x_k + \alpha p_k)^\top p_k$. The constants c_1 and c_2 are those of the Wolfe conditions, with typically $c_1 = 10^{-4}$ and $c_2 = 0.9$.

2.4 Search Directions for Line Search

2.4.1 Gradient Descent (steepest descent)

Let p be a direction and $\alpha > 0$ a step. The Taylor expansion gives

$$f(x_k + \alpha p) = f(x_k) + \alpha p^\top \nabla f(x_k) + \frac{\alpha^2}{2} p^\top \nabla^2 f(x_k + t p) p$$

for some $t \in (0, \alpha)$. The linear rate of change of f along p is the coefficient of α :

$$p^\top \nabla f(x_k).$$

The direction of steepest decrease (of norm 1) is obtained by solving

$$\min_{\|p\|=1} p^\top \nabla f(x_k).$$

By using

$$p^\top \nabla f(x_k) = \|p\| \|\nabla f(x_k)\| \cos(\angle(p, \nabla f(x_k))) = \|\nabla f(x_k)\| \cos(\angle(p, \nabla f(x_k))),$$

we see that this minimum is reached when $\cos(\angle(p, \nabla f(x_k))) = -1$, that is to say when

$$p = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|}.$$

The direction of steepest descent is therefore

$$p_k = -\frac{\nabla f_k}{\|\nabla f_k\|}. \quad (2.18)$$

It is orthogonal to the level curves of f and points in the direction of maximum instantaneous decrease.

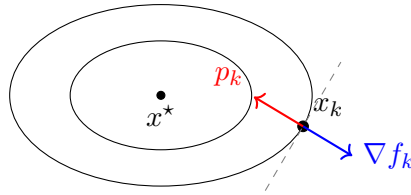


Figure 2.4: Steepest descent direction.

Figure 2.4 shows that the gradient ∇f_k is orthogonal (perpendicular) to the level curve of f passing through x_k and points outwards (growth of the function). The steepest descent direction $p_k = -\nabla f_k$ is thus also orthogonal to the level curve, but points inwards (maximum local decrease).

Remark 2.18. For gradient descent:

- we only require the gradient ∇f ;
- the direction is always a descent direction:

$$p_k^\top \nabla f_k = -\frac{1}{\|\nabla f_k\|} \|\nabla f_k\|^2 < 0;$$

- the method can nevertheless be very slow, particularly on poorly conditioned problems.

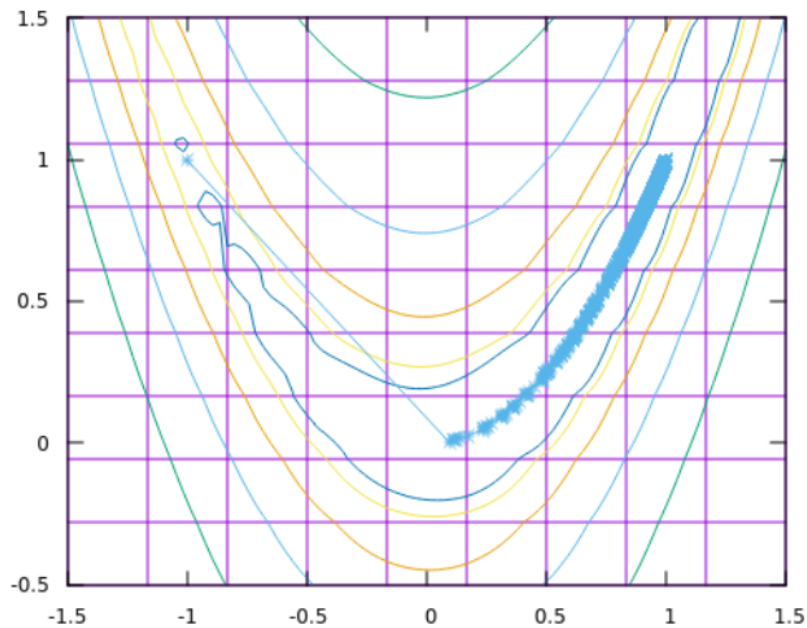


Figure 2.5: Iterations of the steepest descent method on the level curves of the Rosenbrock function.

Key points to observe:

- **Zigzag trajectory:** The method oscillates perpendicularly to the valley, because the gradient always points orthogonally to the level curves.
- **Slow convergence:** Despite multiple iterations, the progress towards the minimum $(1, 1)$ is very slow along the valley.
- **Fundamental limitation:** Without curvature information (Hessian), the method cannot "straighten" its direction towards the bottom of the valley.

This slow convergence motivates the use of more sophisticated methods (Newton, BFGS, conjugate gradient).

2.4.2 Newton Direction

Consider the second-order Taylor approximation

$$f(x_k + \eta) = f(x_k) + \eta^\top \nabla f(x_k) + \frac{1}{2} \eta^\top \nabla^2 f(x_k + t\eta) \eta,$$

for some $t \in (0, 1)$. We approximate f by the quadratic model

$$m_k(\eta) = f(x_k) + \eta^\top \nabla f(x_k) + \frac{1}{2} \eta^\top \nabla^2 f(x_k) \eta.$$

If the Hessian $\nabla^2 f_k$ is positive definite, minimizing m_k with respect to η leads to the optimality condition

$$\nabla f_k + \nabla^2 f_k \eta_k^N = 0,$$

i.e.,

$$\eta_k^N = -(\nabla^2 f_k)^{-1} \nabla f_k. \quad (2.19)$$

The Newton direction is therefore $p_k^N = \eta_k^N$.

If $\nabla^2 f_k \succ 0$, we have

$$\nabla f_k^\top p_k^N = -(p_k^N)^\top \nabla^2 f_k p_k^N < 0,$$

hence p_k^N is a descent direction.

Moreover, if $\nabla^2 f$ is sufficiently smooth, the error of the Taylor approximation is of order

$$f(x_k + \eta) - m_k(\eta) = \mathcal{O}(\|\eta\|^3).$$

Remark 2.19. The natural step in Newton's method is $\alpha = 1$:

$$x_{k+1} = x_k + p_k^N = x_k - (\nabla^2 f_k)^{-1} \nabla f_k.$$

In practice, a line search is often performed (e.g., with Wolfe conditions), starting with $\alpha = 1$.

2.4.3 Characterization of Convergence

Consider a line search type algorithm

$$x_{k+1} = x_k + \alpha_k p_k,$$

where p_k is a descent direction and α_k satisfies the Wolfe conditions.

Theorem 2.20 (Zoutendijk). *Suppose that:*

- $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is bounded below,
- f is continuously differentiable ($\mathcal{C}^1$) on an open set $N \subset \mathbb{R}^n$ containing the level set

$$\{x \in \mathbb{R}^n \mid f(x) \leq f(x_0)\},$$

- the gradient ∇f is Lipschitz continuous on N , i.e., there exists $L > 0$ such that

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|, \quad \forall x, y \in N.$$

Then

$$\sum_{k=0}^{\infty} \cos^2(\theta_k) \|\nabla f_k\|^2 < +\infty, \quad (2.20)$$

where θ_k is the angle between $-\nabla f_k$ and the direction p_k , defined by

$$\cos(\theta_k) = \frac{-\nabla f_k^\top p_k}{\|\nabla f_k\| \|p_k\|}.$$

Remark 2.21. A proof can be found in [9].

Illustration 1. As a consequence, we obtain

$$\cos^2(\theta_k) \|\nabla f_k\|^2 \xrightarrow[k \rightarrow \infty]{} 0.$$

If we assume that there exists $\delta > 0$ such that

$$\cos(\theta_k) \geq \delta > 0, \quad \forall k,$$

then necessarily

$$\|\nabla f_k\| \xrightarrow[k \rightarrow \infty]{} 0.$$

Definition 2.22. An algorithm is said to be *globally convergent* if it satisfies

$$\|\nabla f_k\| \rightarrow 0$$

for any sequence of iterates (x_k) generated from a given starting point x_0 . In the absence of information about the Hessian, this is the best global result one can hope for: it guarantees convergence to a *stationary point*, not necessarily a minimizer.

Illustration 2: Newton's Case. Recall the Newton direction

$$d_k = \eta_k^N = -(\nabla^2 f_k)^{-1} \nabla f_k,$$

assuming $\nabla^2 f_k \succ 0$. Suppose that the condition number of the Hessian is uniformly bounded, i.e., there exists $M > 0$ such that

$$\|\nabla^2 f_k\| \cdot \|(\nabla^2 f_k)^{-1}\| \leq M, \quad \forall k.$$

Then it can be shown that $\cos(\theta_k)$ is uniformly bounded below by a strictly positive constant.

Indeed,

$$\cos(\theta_k) = \frac{-\nabla f_k^\top d_k}{\|\nabla f_k\| \|d_k\|} = \frac{\nabla f_k^\top (\nabla^2 f_k)^{-1} \nabla f_k}{\|\nabla f_k\| \|(\nabla^2 f_k)^{-1} \nabla f_k\|}.$$

We have

$$\|(\nabla^2 f_k)^{-1} \nabla f_k\| \leq \|(\nabla^2 f_k)^{-1}\| \|\nabla f_k\|.$$

Hence

$$\cos(\theta_k) \geq \frac{\nabla f_k^\top (\nabla^2 f_k)^{-1} \nabla f_k}{\|\nabla f_k\|^2 \|(\nabla^2 f_k)^{-1}\|}.$$

Using the Cholesky decomposition $\nabla^2 f_k = LL^\top$, we get

$$\nabla f_k^\top (\nabla^2 f_k)^{-1} \nabla f_k = \|L^{-1} \nabla f_k\|^2 \geq \frac{\|\nabla f_k\|^2}{\|L\|^2}.$$

Furthermore, for the spectral norm, $\|L\|^2 = \|\nabla^2 f_k\|$. We then deduce

$$\cos(\theta_k) \geq \frac{1}{\|\nabla^2 f_k\| \|(\nabla^2 f_k)^{-1}\|} \geq \frac{1}{M} > 0.$$

By Zoutendijk's theorem, we conclude that $\|\nabla f_k\| \rightarrow 0$.

Remark 2.23. For certain algorithms, one only proves that

$$\liminf_{k \rightarrow \infty} \|\nabla f_k\| = 0,$$

which means that at least one subsequence of iterates converges to a stationary point.

2.4.4 Convergence Rate of Steepest Descent

Quadratic case. Consider

$$f(x) = \frac{1}{2} x^\top Q x - b^\top x,$$

where Q is symmetric and positive definite.

Theorem 2.24 (Luenberger). *Let (x_k) be the sequence generated by the steepest descent method (gradient) with exact line search applied to f . Then*

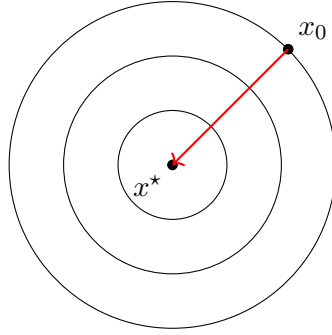
$$\|x_{k+1} - x^*\|_Q \leq \left(\frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} \right) \|x_k - x^*\|_Q,$$

where x^* is the minimizer of f , $\|x\|_Q = \sqrt{x^\top Q x}$, and $\lambda_{\min} \leq \dots \leq \lambda_{\max}$ are the eigenvalues of Q .

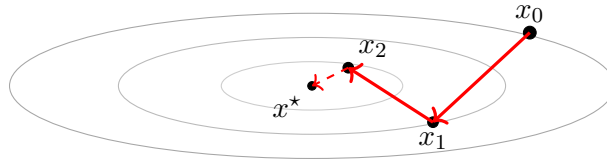
Example 2.25. If $Q = \lambda I$ for some $\lambda > 0$, then

$$\frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = 0,$$

and convergence occurs in a single iteration.



(a) Equal eigenvalues ($Q = \lambda I$): direct convergence in one iteration.



(b) Very different eigenvalues ($\kappa \gg 1$): slow zigzag convergence.

Figure 2.6: Effect of the conditioning of Q on steepest descent.

Nonlinear case:

Theorem 2.26. Suppose that f is twice differentiable and that the iterates generated by the steepest descent method converge to a point x^* such that the Hessian $\nabla^2 f(x^*)$ is positive definite. Let $\lambda_1 \leq \dots \leq \lambda_m$ be the eigenvalues of $\nabla^2 f(x^*)$ and let

$$\omega \in \left(\frac{\lambda_m - \lambda_1}{\lambda_m + \lambda_1}, 1 \right).$$

Then, for k large enough,

$$f(x_{k+1}) - f(x^*) \leq \omega^2 (f(x_k) - f(x^*)).$$

Example 2.27. Suppose that

$$f(x_0) = 1, \quad f(x^*) = 0,$$

and that

$$\kappa(\nabla^2 f(x^*)) = \|\nabla^2 f(x^*)\| \|(\nabla^2 f(x^*))^{-1}\| = \frac{\lambda_m}{\lambda_1} = 800.$$

Then the linear convergence constant is

$$\omega = \frac{800 - 1}{800 + 1}.$$

After 1000 iterations,

$$f(x_{1001}) \approx \omega^{2000} f(x_0) \approx 0.082.$$

Definition 2.28 (Rate of convergence). Let (x_k) be a sequence converging to x^* .

- We say that (x_k) converges *Q-linearly* to x^* if there exists $\omega \in (0, 1)$ and k_0 such that

$$\frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|} \leq \omega, \quad \forall k \geq k_0.$$

- The convergence is *Q-superlinear* if

$$\frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|} \xrightarrow{k \rightarrow \infty} 0.$$

- The convergence is *Q-quadratic* if there exists $M > 0$ and k_0 such that

$$\frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|^2} \leq M, \quad \forall k \geq k_0.$$

2.4.5 Newton's Method: quadratic convergence

Theorem 2.29 (Quadratic convergence of Newton). Suppose that f is of class $\mathcal{C}^2$ and that $\nabla^2 f$ is Lipschitz continuous in a neighborhood of a solution x^* satisfying the second-order sufficient conditions (see [Theorem 2.8](#)). Consider Newton's method

$$x_{k+1} = x_k - (\nabla^2 f(x_k))^{-1} \nabla f(x_k).$$

Then:

1. if x_0 is close enough to x^* , the sequence (x_k) converges to x^* ;
2. the convergence is quadratic:

$$\|x_{k+1} - x^*\| \leq C \|x_k - x^*\|^2$$

for a positive constant $C > 0$ and large enough k ;

3. the gradient norms also satisfy a quadratic convergence:

$$\|\nabla f(x_{k+1})\| \leq C' \|\nabla f(x_k)\|^2.$$

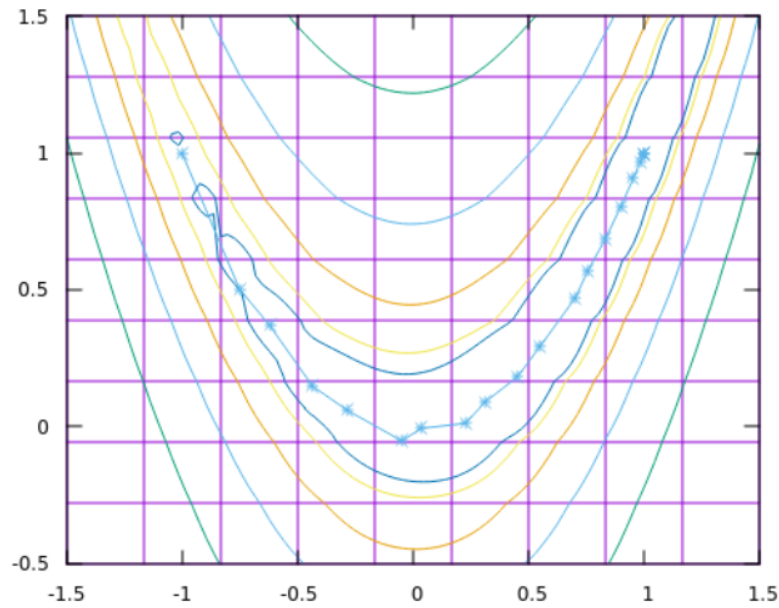


Figure 2.7: Iterations of Newton's method on the level curves of the Rosenbrock function.

Key points to observe:

- **Direct trajectory:** Unlike gradient descent (Figure 2.5), Newton reaches the minimum in very few iterations.
- **Use of the Hessian:** The curvature information allows "correcting" the direction of the gradient to follow the bottom of the valley.
- **Quadratic convergence:** Near the minimum, each iteration doubles the number of correct significant digits (Theorem 2.29).
- **Cost:** Each iteration requires computing and inverting the Hessian, which can be prohibitive in high dimensions.

2.4.6 Hessian Modification for Newton

The efficiency of Newton's method relies on the positiveness of the Hessian. Far from the solution, it may happen that $\nabla^2 f_k$ is not positive definite, so that the direction

$$\eta_k^N = -(\nabla^2 f_k)^{-1} \nabla f_k$$

might not be a descent direction.

One then introduces a matrix B_k which plays the role of a modified Hessian, and solves

$$B_k \eta_k = -\nabla f_k.$$

Algorithm 4: Newton with line search and modified Hessian**Input:** an initial point x_0

- 1 **for** $k = 0, 1, \dots$ **do**
- 2 Factorize the matrix $B_k = \nabla^2 f(x_k) + E_k$, where $E_k = 0$ if $\nabla^2 f(x_k)$ is sufficiently positive definite. Otherwise, E_k is chosen to ensure that B_k is sufficiently positive definite;
- 3 Solve $B_k p_k = -\nabla f(x_k)$;
- 4 Set $x_{k+1} \leftarrow x_k + \alpha_k p_k$, where α_k satisfies the Wolfe, Goldstein, or Armijo backtracking conditions;

Definition 2.30 (Bounded Modified Factorization Property (BMFP)). We say that the sequence (B_k) satisfies the *bounded modified factorization property* (BMFP) if there exists $C > 0$ such that

$$\kappa(B_k) = \|B_k\| \cdot \|B_k^{-1}\| \leq C, \quad \forall k.$$

Theorem 2.31 (Moré–Sorensen). Suppose that f is of class $\mathcal{C}^2$ on an open set $D \subset \mathbb{R}^n$. Suppose that the level set

$$\{x \in \mathbb{R}^n \mid f(x) \leq f(x_0)\}$$

is compact. If the sequence (B_k) used in the modified Newton algorithm satisfies the BMFP property, then

$$\|\nabla f_k\| \xrightarrow[k \rightarrow \infty]{} 0.$$

Examples of constructions for B_k .

- **Spectral decomposition.** We can diagonalize $\nabla^2 f_k$, replace the non-positive eigenvalues by a small constant $\delta > 0$ (or possibly flip the sign of the negative eigenvalues), and then reconstruct the matrix.
- **Adding a multiple of the identity.** We seek $\tau > 0$ such that $\nabla^2 f_k + \tau I$ is positive definite.

Algorithm 5: Cholesky with addition of a multiple of the identity

- 1 Choose $\beta > 0$;
- 2 **if** $\min_i(a_{i,i}) > 0$ **then**
- 3 Set $\tau_0 \leftarrow 0$;
- 4 **else**
- 5 $\tau_0 \leftarrow -\min_i(a_{i,i}) + \beta$;
- 6 **for** $k = 0, 1, \dots$ **do**
- 7 Attempt to apply the Cholesky algorithm to obtain $LL' = A + \tau_k I$;
- 8 **if** the factorization is completed successfully **then**
- 9 **stop and return** L ;
- 10 **else**
- 11 $\tau_{k+1} \leftarrow \max(2\tau_k, \beta)$;

Modified Cholesky factorization. Any symmetric positive definite matrix A can be written $A = LDL^\top$ where D is diagonal and L is lower triangular with unit diagonal. We can modify the Cholesky algorithm to ensure that the elements of D remain strictly positive, which provides a stable method for constructing positive definite matrices B_k . See for example [2] or [7] for more details.

2.5 Practice Exercises (Nocedal & Wright)

Part 1: Course Questions (Exam Type)

- Exercise 2.1** (Comprehension Questions). 1. **Line Search:** Describe the principle of a Line Search. Graphically explain the Armijo condition and the curvature condition (Wolfe).
2. **Newton vs Gradient:** Why would one prefer Newton's method over the steepest descent method? What is the associated cost?
3. **Ill-conditioned Hessian:** What to do if one wishes to implement Newton's method but the Hessian is very large or singular? (Discussion on Modified Newton, Quasi-Newton, or Trust Region).
4. **Rate of convergence:** Define linear, superlinear, and quadratic convergence rates. Give an example of a method for each rate.
5. **Descent direction:** Show mathematically that if B_k is positive definite, the direction $-B_k^{-1}\nabla f_k$ is always a descent direction.
6. **Scaling:** Explain how poor variable scaling affects the steepest descent method (Zig-zag).

Suggested Correction. 1. **Line Search:** We are looking for a step $\alpha_k > 0$ along a descent direction p_k . Armijo ensures sufficient decrease (line $f(x_k) + c_1\alpha\nabla f_k^T p_k$). Curvature ensures that we do not take steps that are too small (slope increases). (*Draw a convex curve and secant lines*).

2. **Newton:** Quadratic convergence (very fast) near the solution vs linear convergence for Gradient. Cost: Computation and inversion (or factorization) of the Hessian $O(n^3)$.
3. **Large/Singular Hessian:**
- *Modified Newton:* Add τI (Modified Cholesky) to make it positive definite.
 - *Quasi-Newton (BFGS):* Approximate B_k iteratively (only gradients needed, $O(n^2)$).
 - *Trust Region:* Limit the step to a trust region if the quadratic model is not good.
 - *Newton-CG:* Solve the linear system via truncated Conjugate Gradient (no need to store the explicit Hessian, just matrix-vector products).
4. **Definitions:** See Appendix A.
5. **Descent:** $p_k^T \nabla f_k = -(\nabla f_k)^T B_k^{-1} \nabla f_k$. Let $v = \nabla f_k$. We have $-v^T B_k^{-1} v$. If $B_k \succ 0$, then $B_k^{-1} \succ 0$, therefore $v^T B_k^{-1} v > 0$ for $v \neq 0$. So inner product < 0 , it is a descent.

Part 2: Application Exercises

Exercise 2.2 (Rosenbrock Function). Let $f(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$.

1. Calculate the gradient $\nabla f(x)$ and the Hessian $\nabla^2 f(x)$.
2. Find the critical points.
3. Verify the Second-Order Sufficient Conditions (SOSC) to confirm the nature of these points.

Suggested Correction. **1. Calculations:**

$$\nabla f(x) = \begin{pmatrix} -400x_1(x_2 - x_1^2) - 2(1 - x_1) \\ 200(x_2 - x_1^2) \end{pmatrix}$$

$$\nabla^2 f(x) = \begin{pmatrix} -400x_2 + 1200x_1^2 + 2 & -400x_1 \\ -400x_1 & 200 \end{pmatrix}$$

2. Critical Points (FOC): $\nabla f(x) = 0$. (ii) $200(x_2 - x_1^2) = 0 \implies x_2 = x_1^2$. (i) $-400x_1(0) - 2(1 - x_1) = 0 \implies x_1 = 1$. Unique stationary point: $x^* = (1, 1)^T$.

3. Second-Order Conditions (SOSC): Let's evaluate the Hessian at $x^* = (1, 1)$.

$$H = \nabla^2 f(1, 1) = \begin{pmatrix} -400(1) + 1200(1) + 2 & -400 \\ -400 & 200 \end{pmatrix} = \begin{pmatrix} 802 & -400 \\ -400 & 200 \end{pmatrix}$$

To check that it is positive definite: - Principal minors: $M_1 = 802 > 0$. - Determinant: $\det(H) = 802(200) - (-400)^2 = 160400 - 160000 = 400 > 0$. The leading principal minors are strictly positive, therefore $H \succ 0$.

Checkpoint : Did you understand the essentials ?

- **Optimality:** What is the first-order necessary condition ($\nabla f = 0$)? And second-order sufficient ($\nabla^2 f \succ 0$)?
- **Taylor:** Do you know how to write the 2nd order Taylor expansion for a multivariate function?
- **Methods:** What is the fundamental difference between the steepest descent method (Gradient Descent) and Newton's method (use of curvature/Hessian)?
- **Convergence:** What is a quadratic convergence? Which of the two above methods has it?

Exercise 2.3 (Counter-example Wolfe Conditions). Show that if $0 < c_2 < c_1 < 1$, it is possible that no step α satisfies the Wolfe conditions.

Suggested Correction. The curvature condition (Wolfe 2) requires $\nabla f(x_k + \alpha p_k)^T p_k \geq c_2 \nabla f_k^T p_k$. Since $\nabla f_k^T p_k < 0$ (descent direction) and $c_2 < c_1$, the requested slope is "less negative" (flatter) than that allowed by the strict Armijo condition if the function is convex/quadratic. If we have

a convex quadratic function, the slope increases monotonically. The Armijo condition is written $\phi(\alpha) \leq \phi(0) + c_1 \alpha \phi'(0)$. If $c_2 < c_1$, the admissible interval for curvature can be disjoint from the admissible interval for Armijo, specifically if the function goes back up "too quickly" or if the parameters are poorly chosen relative to the true curvature. (See Nocedal & Wright Ex 3.2 for geometric details).

Exercise 2.4 (Steepest descent method on a quadratic). Consider the steepest descent method with exact line search applied to the convex quadratic function $f(x) = \frac{1}{2}x^T Qx - b^T x$. Show that if the starting point is such that $x_0 - x^*$ is parallel to an eigenvector of Q , then the method finds the solution in a single step.

Suggested Correction. Let $x_0 - x^* = v$ where $Qv = \lambda v$. The gradient is $\nabla f(x) = Qx - b = Q(x - x^*) + (Qx^* - b) = Q(x - x^*)$. Thus $\nabla f(x_0) = Q(x_0 - x^*) = \lambda(x_0 - x^*)$. The steepest descent direction is $p_0 = -\nabla f(x_0) = -\lambda(x_0 - x^*)$. So it is exactly the direction towards the solution x^* . An exact line search will find the minimum along this line, which is precisely x^* .

Exercise 2.5 (Rate of convergence - for discussion). (See Appendix A.2 of the book for exact definitions).

1. Show that the sequence $x_k = 1/k$ is not Q-linearly convergent, although it converges to zero. (Sublinear convergence).
2. Show that the sequence $x_k = 1 + (0.5)^{2^k}$ converges Q-quadratically to 1.
3. Does the sequence $x_k = 1/k!$ converge Q-superlinearly? Q-quadratically?

Suggested Correction. 1. For $x_k = 1/k$, the ratio $\frac{|x_{k+1}-0|}{|x_k-0|} = \frac{1/(k+1)}{1/k} = \frac{k}{k+1} \rightarrow 1$. For a Q-linear convergence, this ratio must be strictly bounded by a constant $r < 1$. Here, this is not the case.

2. Error $e_k = (0.5)^{2^k}$. $e_{k+1} = (0.5)^{2^{k+1}} = (0.5)^{2 \cdot 2^k} = ((0.5)^{2^k})^2 = e_k^2$. Thus $\frac{e_{k+1}}{e_k^2} = 1$. This is a Q-quadratic convergence.

3. $x_k = 1/k!$. $e_k = 1/k!$. Ratio: $\frac{e_{k+1}}{e_k} = \frac{1}{(k+1)!} \cdot k! = \frac{1}{k+1} \rightarrow 0$. It is therefore Q-superlinear. For quadratic: $\frac{e_{k+1}}{e_k^2} = \frac{1}{(k+1)!} \cdot (k!)^2 = \frac{(k!)^2}{k!(k+1)!} = \frac{k!}{k+1} \rightarrow \infty$. Thus it converges less quickly than quadratically.

Chapter 3

Conjugate Gradient Methods

In this chapter, we present the *conjugate gradient methods*, which constitute a family of very efficient algorithms for solving high-dimensional quadratic problems, as well as their nonlinear extensions.

Conjugate gradient methods were introduced in the 1950s by Hestenes and Stiefel [3] to solve linear systems

$$Ax = b,$$

where $A \in \mathbb{R}^{n \times n}$ is symmetric positive definite. Later, Fletcher and Reeves [1] proposed a *nonlinear* extension for the minimization of a general function $f : \mathbb{R}^n \rightarrow \mathbb{R}$.

Compared to gradient descent:

- they do not require storing the matrix A ;
- they converge much faster on well-conditioned quadratic problems;
- they are particularly suitable for high dimensions.

3.1 Linear Conjugate Gradient

3.1.1 Associated Quadratic Problem

Consider the linear system

$$Ax = b,$$

where A is symmetric positive definite. This problem is equivalent to minimizing

$$q(x) = \frac{1}{2}x^\top Ax - b^\top x. \quad (3.1)$$

Its gradient is

$$\nabla q(x) = Ax - b,$$

and thus x^* minimizes q if and only if $Ax^* = b$.

For a direction p_k , we minimize $q(x_k + \alpha p_k)$:

$$\psi(\alpha) = q(x_k + \alpha p_k).$$

The condition $\psi'(\alpha) = 0$ gives

$$p_k^\top A(x_k + \alpha p_k) - p_k^\top b = 0.$$

Since $r_k = Ax_k - b$:

$$\alpha_k = -\frac{p_k^\top r_k}{p_k^\top A p_k}.$$

3.1.2 A -Conjugate Directions

Definition 3.1. Two non-zero vectors p_i, p_j are A -conjugate if

$$p_i^\top A p_j = 0.$$

A family $(p_0, \dots, p_m)$ is A -conjugate if all its distinct vectors are pairwise A -conjugate.

An A -conjugate family forms a basis in which the minimization becomes independent between components: a correction in one direction does not degrade the ones already made.

3.1.3 Conjugate Direction Method

Assume we are given A -conjugate directions $(\eta_0, \dots, \eta_{m-1})$. We set:

$$x_{k+1} = x_k + \alpha_k \eta_k, \quad \alpha_k = -\frac{\eta_k^\top r_k}{\eta_k^\top A \eta_k}.$$

Theorem 3.2 (Minimization over expanding subspaces). For $k \geq 1$:

$$r_k^\top \eta_i = 0 \quad (i < k),$$

and

$$x_k = \arg \min_{x \in x_0 + \text{span}(\eta_0, \dots, \eta_{k-1})} q(x).$$

3.1.4 Linear Conjugate Gradient Algorithm

We denote $r_k = Ax_k - b$.

We define the directions by

$$p_0 = -r_0, \quad p_k = -r_k + \beta_k p_{k-1}.$$

The A -conjugacy condition $p_k^\top A p_{k-1} = 0$ requires

$$\beta_k = \frac{r_k^\top A p_{k-1}}{p_{k-1}^\top A p_{k-1}} \quad (\text{Linear Hestenes–Stiefel [3]}).$$

In practice, one often uses the equivalent form

$$\beta_k = \frac{r_k^\top r_k}{r_{k-1}^\top r_{k-1}}.$$

Algorithm 6: Preliminary Version of Conjugate Gradient

```

1 Given  $x_0$ ;
2 Set  $r_0 \leftarrow Ax_0 - b$ ,  $p_0 \leftarrow -r_0$ ,  $k \leftarrow 0$ ;
3 while  $r_k \neq 0$  do
4    $\alpha_k \leftarrow -\frac{r_k^\top p_k}{p_k^\top A p_k}$ ;
5    $x_{k+1} \leftarrow x_k + \alpha_k p_k$ ;
6    $r_{k+1} \leftarrow Ax_{k+1} - b$ ;
7    $\beta_{k+1} \leftarrow \frac{r_{k+1}^\top A p_k}{p_k^\top A p_k}$ ;
8    $p_{k+1} \leftarrow -r_{k+1} + \beta_{k+1} p_k$ ;
9    $k \leftarrow k + 1$ ;

```

Algorithm 7: Conjugate Gradient (practical version)

```

1 Given  $x_0$ ;
2 Set  $r_0 \leftarrow Ax_0 - b$ ,  $p_0 \leftarrow -r_0$ ,  $k \leftarrow 0$ ;
3 while  $r_k \neq 0$  do
4    $\alpha_k \leftarrow \frac{r'_k r_k}{p'_k A p_k}$ ;
5    $x_{k+1} \leftarrow x_k + \alpha_k p_k$ ;
6    $r_{k+1} \leftarrow r_k + \alpha_k A p_k$ ;
7    $\beta_{k+1} \leftarrow \frac{r'_{k+1} r_{k+1}}{r'_k r_k}$ ;
8    $p_{k+1} \leftarrow -r_{k+1} + \beta_{k+1} p_k$ ;
9    $k \leftarrow k + 1$ ;

```

Remark 3.3. The practical version avoids recalculating Ax_{k+1} by using the recurrence relation $r_{k+1} = r_k + \alpha_k A p_k$. Furthermore, the formulas for α_k and β_{k+1} only require inner products.

The following properties hold:

$$r_i^\top r_j = 0, \quad p_i^\top A p_j = 0 \quad (i \neq j).$$

3.1.5 Error Bound and Conditioning

If $\lambda_{\min}$ and $\lambda_{\max}$ are the extreme eigenvalues of A , then

$$\frac{\|x_k - x^*\|_A}{\|x_0 - x^*\|_A} \leq 2 \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^k, \quad \kappa(A) = \frac{\lambda_{\max}}{\lambda_{\min}}.$$

3.2 Geometric Interpretation

3.2.1 Level sets of a quadratic function

The level sets of (3.1) are ellipses centered at x^* . The conjugate gradient follows successively A -conjugate directions, which allows "correcting" the error in each direction independently.

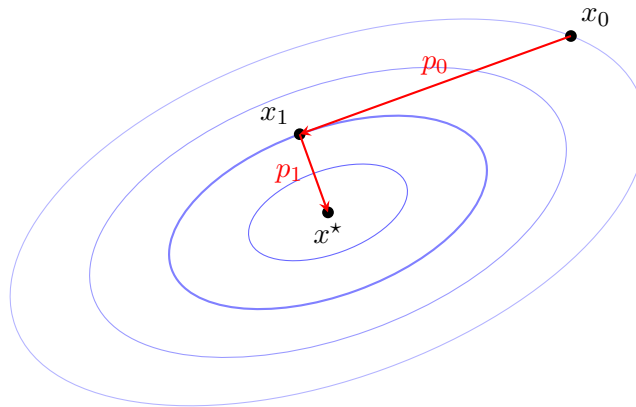


Figure 3.1: Geometric interpretation: level sets of q and conjugate directions.

In two dimensions, the conjugate gradient reaches exactly x^* in two iterations.

3.3 Comparison with Gradient Descent

Gradient descent zigzags on elongated ellipses, while the conjugate gradient chooses directions that will progressively "diagonalize" the matrix A .

3.4 Preconditioning

When A is ill-conditioned, we introduce an SPD matrix M approximating A or A^{-1} . We consider the equivalent system:

$$M^{-1}Ax = M^{-1}b.$$

We introduce $z_k = M^{-1}r_k$.

Algorithm 8: Preconditioned Conjugate Gradient

```

1 Given  $x_0$ , preconditioner  $M$ ;
2 Set  $r_0 \leftarrow Ax_0 - b$ ;
3 Solve  $My_0 = r_0$  for  $y_0$ ;
4 Set  $p_0 \leftarrow -y_0$ ,  $k \leftarrow 0$ ;
5 while  $r_k \neq 0$  do
6    $\alpha_k \leftarrow \frac{r'_k y_k}{p'_k A p_k}$ ;
7    $x_{k+1} \leftarrow x_k + \alpha_k p_k$ ;
8    $r_{k+1} \leftarrow r_k + \alpha_k A p_k$ ;
9   Solve  $My_{k+1} = r_{k+1}$ ;
10   $\beta_{k+1} \leftarrow \frac{r'_{k+1} y_{k+1}}{r'_k y_k}$ ;
11   $p_{k+1} \leftarrow -y_{k+1} + \beta_{k+1} p_k$ ;
12   $k \leftarrow k + 1$ ;

```

3.5 Nonlinear Conjugate Gradient

We seek to minimize $f(x)$, assumed to be $\mathcal{C}^1$. Let $g_k = \nabla f(x_k)$.

Algorithm 9: Fletcher-Reeves Method

```

1 Given  $x_0$ ;
2 Evaluate  $f_0 = f(x_0)$ ,  $\nabla f_0 = \nabla f(x_0)$ ;
3 Set  $p_0 = -\nabla f_0$ ,  $k \leftarrow 0$ ;
4 while  $\nabla f_k \neq 0$  do
5   Compute  $\alpha_k$  and set  $x_{k+1} = x_k + \alpha_k p_k$ ;
6   Evaluate  $\nabla f_{k+1}$ ;
7    $\beta_{k+1}^{FR} \leftarrow \frac{\nabla f'_{k+1} \nabla f_{k+1}}{\nabla f'_k \nabla f_k}$ ;
8    $p_{k+1} \leftarrow -\nabla f_{k+1} + \beta_{k+1}^{FR} p_k$ ;
9    $k \leftarrow k + 1$ ;

```

Theorem 3.4. *If the line search satisfies the Wolfe conditions and f is smooth, then*

$$\liminf_{k \rightarrow \infty} \|g_k\| = 0.$$

3.6 Summary

- The linear conjugate gradient is optimal for SPD quadratic problems.
- Directions are A -conjugate and residuals orthogonal.
- Preconditioning drastically improves convergence.
- The nonlinear conjugate gradient is efficient and inexpensive.
- The choice of β_k strongly influences performance.

3.6.1 Properties of Iterates and Krylov Subspaces

The following properties justify the theoretical efficiency of the method.

Theorem 3.5. *Suppose that the k -th iterate is not equal to the solution x^* . Then:*

(i) *The residuals are orthogonal:*

$$\forall i = 0, \dots, k-1, \quad r_k^\top r_i = 0.$$

(ii) *The space spanned by the residuals is a Krylov subspace of degree k for r_0 :*

$$\text{span}(r_0, \dots, r_k) = \text{span}(r_0, Ar_0, \dots, A^k r_0).$$

(iii) *The space spanned by the descent directions coincides with this space:*

$$\text{span}(p_0, \dots, p_k) = \text{span}(r_0, Ar_0, \dots, A^k r_0).$$

(iv) *The directions are A -conjugate to the previous ones:*

$$\forall i = 0, \dots, k-1, \quad p_k^\top A p_i = 0.$$

Proof. Reference: [9]. □

Consequences. From point (iv), the conjugate gradient converges to x^* in at most n iterations (in exact arithmetic), since there can be no more than n mutually A -conjugate non-zero directions in $\mathbb{R}^n$.

Remark 3.6. It is essential for these properties that the initialization is $p_0 = -r_0 = -(Ax_0 - b)$.

3.6.2 Derivation of the efficient version (Standard CG)

Up to now, we had the general formula for α_k . We can simplify it to reduce the computational cost.

We know that:

$$\alpha_k = \frac{-r_k^\top p_k}{p_k^\top A p_k}.$$

According to the expanding subspace minimization (ESM) theorem, we know that $r_k^\top p_i = 0$ for all $i = 0, \dots, k-1$. Since $p_k = -r_k + \beta_k p_{k-1}$, we have:

$$-r_k^\top p_k = -r_k^\top (-r_k + \beta_k p_{k-1}) = r_k^\top r_k - \beta_k \underbrace{r_k^\top p_{k-1}}_{=0}.$$

Thus, the simplified formula for the step is:

$$\alpha_k = \frac{r_k^\top r_k}{p_k^\top A p_k}.$$

Next, for the coefficient β_{k+1} , we start from the definition ensuring A -conjugacy:

$$\beta_{k+1} = \frac{r_{k+1}^\top A p_k}{p_k^\top A p_k}.$$

We use the residual recurrence relation: $r_{k+1} = r_k + \alpha_k A p_k$, which implies $A p_k = \frac{1}{\alpha_k}(r_{k+1} - r_k)$. Substituting into the numerator:

$$r_{k+1}^\top A p_k = \frac{1}{\alpha_k} r_{k+1}^\top (r_{k+1} - r_k) = \frac{1}{\alpha_k} (r_{k+1}^\top r_{k+1} - \underbrace{r_{k+1}^\top r_k}_{=0}) = \frac{r_{k+1}^\top r_{k+1}}{\alpha_k}.$$

Similarly for the denominator, we know that $p_k^\top A p_k = \frac{r_k^\top r_k}{\alpha_k}$.

Thus, the ratio simplifies remarkably:

$$\beta_{k+1} = \frac{\frac{r_{k+1}^\top r_{k+1}}{\alpha_k}}{\frac{r_k^\top r_k}{\alpha_k}} = \frac{r_{k+1}^\top r_{k+1}}{r_k^\top r_k}.$$

Remarks on Algorithm 7 (Standard CG):

- We eliminated the matrix-vector products in calculating the scalar coefficients (they only remain in the update $A p_k$).
- We only need to store the values of x , p , and r over two iterations.

3.7 Rate of Convergence

Theorem 3.7 ([9]). *If the matrix A has only r distinct eigenvalues, then convergence occurs in at most r iterations.*

A more precise error bound is given by the following theorem, which depends on the eigenvalue distribution.

Theorem 3.8 ([6]). *If A has eigenvalues $\lambda_1 \leq \dots \leq \lambda_n$, then:*

$$\|x_{k+1} - x^*\|_A^2 \leq \left(\frac{\lambda_{n-k} - \lambda_1}{\lambda_{n-k} + \lambda_1} \right)^2 \|x_0 - x^*\|_A^2.$$

This result suggests that the conjugate gradient eliminates the influence of the extreme eigenvalues as the iterations proceed.

3.7.1 Illustration: Effect of eigenvalue distribution

Consider a matrix A having 2 "clusters" of eigenvalues:

- A large group of $n - m$ eigenvalues close to λ_1 .
- A small group of m eigenvalues (outliers) located further away.

According to Luenberger's theorem, after m iterations, we eliminate the influence of the m extreme eigenvalues. The error at step $m + 1$ then satisfies:

$$\|x_{m+1} - x^*\|_A^2 \leq \left(\frac{\lambda_{n-m} - \lambda_1}{\lambda_{n-m} + \lambda_1} \right)^2 \|x_0 - x^*\|_A^2.$$

Roughly speaking, this means effective convergence starts after dealing with the few isolated eigenvalues.

3.7.2 Comparison of convergence rates

It is instructive to compare the error bound of Conjugate Gradient (CG) with that of the Steepest Descent (SD) method.

We denote the condition number $\kappa(A) = \|A\| \cdot \|A^{-1}\| = \frac{\lambda_n}{\lambda_1}$.

- **For Conjugate Gradient (CG):**

$$\|x_k - x^*\|_A \leq 2 \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^k \|x_0 - x^*\|_A.$$

- **For Steepest Descent (SD):**

$$\|x_k - x^*\|_A \leq \left(\frac{\kappa(A) - 1}{\kappa(A) + 1} \right)^k \|x_0 - x^*\|_A.$$

Remark 3.9. When $\kappa(A)$ is large (ill-conditioned problem), the term $\frac{\kappa(A)-1}{\kappa(A)+1}$ is very close to 1, which implies a very slow convergence (numerical "zigzagging"). In contrast, the term with the square root $\sqrt{\kappa(A)}$ in CG is much more favorable, ensuring faster convergence.

3.8 Preconditioning

3.8.1 General Idea

The idea of preconditioning is to "transform" the eigenvalue distribution of A to accelerate convergence.

We perform a change of variable from x to $\hat{x}$:

$$\hat{x} = Cx \iff x = C^{-1}\hat{x},$$

where C is an invertible (non-singular) matrix.

The quadratic objective function $\phi(x)$ then becomes a function of $\hat{x}$, denoted $\hat{\phi}(\hat{x})$:

$$\hat{\phi}(\hat{x}) = \phi(C^{-1}\hat{x}) = \frac{1}{2}(C^{-1}\hat{x})^\top A(C^{-1}\hat{x}) - b^\top(C^{-1}\hat{x}).$$

Expanding, we obtain:

$$\hat{\phi}(\hat{x}) = \frac{1}{2}\hat{x}^\top \left[(C^{-1})^\top A C^{-1} \right] \hat{x} - \left((C^{-1})^\top b \right)^\top \hat{x}.$$

3.8.2 Choosing the preconditioning matrix

The function $\hat{\phi}$ is still quadratic. If we use the Conjugate Gradient to minimize $\hat{\phi}$, performance will now depend on the eigenvalues of the new coefficient matrix:

$$\hat{A} = (C^{-1})^\top A C^{-1}.$$

We must therefore choose the matrix C such that:

1. The condition number of $(C^{-1})^\top A C^{-1}$ is lower than that of A (i.e. $\kappa(\hat{A}) < \kappa(A)$).
2. Or the eigenvalues of $(C^{-1})^\top A C^{-1}$ are well clustered.

In practice, we do not explicitly build $\hat{A}$. We often set $M = C^\top C$ (the preconditioner) and adapt the CG algorithm to implicitly solve the system $M^{-1}Ax = M^{-1}b$.

3.8.3 Practical Implementation

In practice, we only need to use the matrix C through the preconditioner M defined by:

$$M = C^\top C.$$

This leads to Algorithm 8 (Preconditioned Conjugate Gradient).

Balance. The use of preconditioning implies a trade-off:

- (+) Preconditioning improves the eigenvalue distribution (reducing the number of iterations).
- (-) It introduces linear system solves $My = r$ at each step.

The goal is thus to choose M so that the system $My = r$ is inexpensive to solve.

3.8.4 Example: Incomplete Cholesky Preconditioning

A classical method relies on the Cholesky decomposition. For a symmetric positive definite matrix A , there exists a lower triangular matrix L such that $A = LL^\top$.

Idea: We seek an approximation of L by a *sparse* matrix $\tilde{L}$ such that:

$$A \approx \tilde{L}\tilde{L}^\top.$$

We then choose $C = \tilde{L}^\top$. The preconditioner becomes $M = (\tilde{L}^\top)^\top \tilde{L}^\top = \tilde{L}\tilde{L}^\top$.

The matrix of the transformed system is then:

$$(C^{-1})^\top A C^{-1} = \tilde{L}^{-1} A (\tilde{L}^{-1})^\top \approx I.$$

The eigenvalue distribution of this matrix is generally favorable.

System Resolution. With this choice, solving $My = r$ amounts to solving $\tilde{L}\tilde{L}^\top y = r$. This is done by two consecutive triangular system solves (very fast):

1. Solve $\tilde{L}z = r$ to find z (forward substitution).
2. Solve $\tilde{L}^\top y = z$ to find y (backward substitution).

Checkpoint : Did you understand the essentials ?

- **Conjugate directions:** Define two A -conjugate directions. What is the advantage over simple orthogonal directions?
- **Exact convergence:** In how many iterations at most does the conjugate gradient converge for an n -dimensional convex quadratic function?
- **Krylov:** What is the link between the generated directions and Krylov subspaces?
- **Preconditioning:** What is the goal of preconditioning the system $Ax = b$? (Reduce the condition number $\kappa(A)$ and cluster eigenvalues).

Chapter 4

Nonlinear Conjugate Gradient Methods

4.1 Introduction

Idea: Adapt the linear conjugate gradient (efficient for quadratic problems) to general nonlinear objective functions.

4.2 Fletcher-Reeves Method

Fletcher and Reeves [1] proposed two major changes compared to the linear algorithm to minimize any function f .

4.2.1 1. Line Search

Unlike the quadratic case where the step α_k is given by an exact formula, here we substitute the formula:

$$\alpha_k = \frac{r_k^\top r_k}{p_k^\top A p_k}$$

with a **line search** to approximate the minimum of f along the direction p_k .

4.2.2 2. Substituting the residual with the gradient

In the linear case, the residual r_k corresponded to the gradient of the quadratic function ϕ . For the general case, we substitute the gradient of ϕ with the gradient of f :

$$r_k = \nabla \phi(x_k) \longrightarrow \nabla f_k.$$

4.2.3 Formula for β_{FR}

The formula for β used in the linear case ($\beta = \frac{r_{k+1}^\top r_{k+1}}{r_k^\top r_k}$) becomes the Fletcher-Reeves formula:

$$\beta_{FR} = \frac{\nabla f_{k+1}^\top \nabla f_{k+1}}{\nabla f_k^\top \nabla f_k}.$$

This defines Algorithm 9.

Remark 4.1. There is no minus sign in this formula.

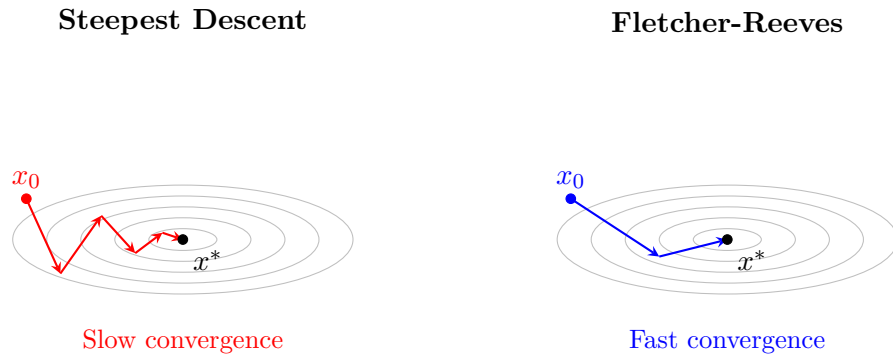


Figure 4.1: Comparison of steepest descent (left) and Fletcher-Reeves (right) on elliptical level curves. The Fletcher-Reeves method uses conjugate directions to converge faster.

Figure 4.1 illustrates the advantage of the nonlinear conjugate gradient (Fletcher-Reeves) over steepest descent. On an ill-conditioned quadratic function (elongated ellipses), steepest descent zigzags by alternating between orthogonal directions, while Fletcher-Reeves exploits conjugate directions to reach the minimum in fewer iterations.

4.2.4 Illustration on the Rosenbrock Function

The **Rosenbrock function** $f(x, y) = (1-x)^2 + 100(y-x^2)^2$ is a classic test case for optimization algorithms. It has a unique minimum at $(1, 1)$ but presents a very narrow and strongly curved valley that makes convergence difficult.

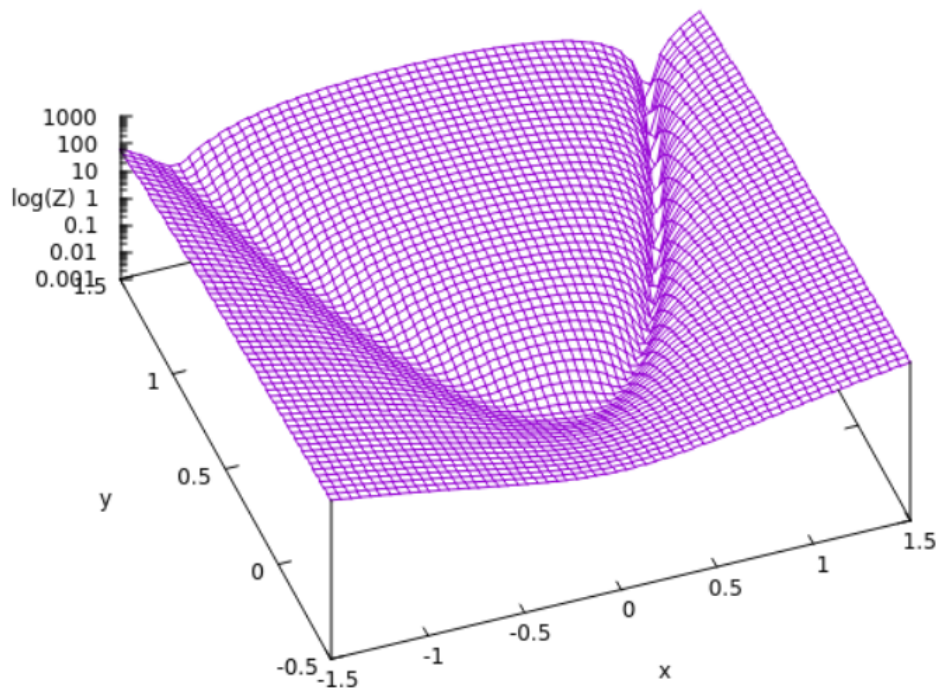


Figure 4.2: The Rosenbrock function $f(x, y) = (1-x)^2 + 100(y-x^2)^2$, having a unique minimum at $(1, 1)$.

Key points to observe:

- **Shape of the surface:** The parabolic valley ("banana valley") is clearly visible. The global minimum $(1, 1)$ is at the bottom of this narrow valley.
- **Logarithmic scale:** The vertical axis uses $\log(Z)$ to visualize the large variations of the function.
- **Optimization difficulty:** Reaching the bottom of the valley is easy, but making progress along the valley towards the minimum is difficult for many methods.

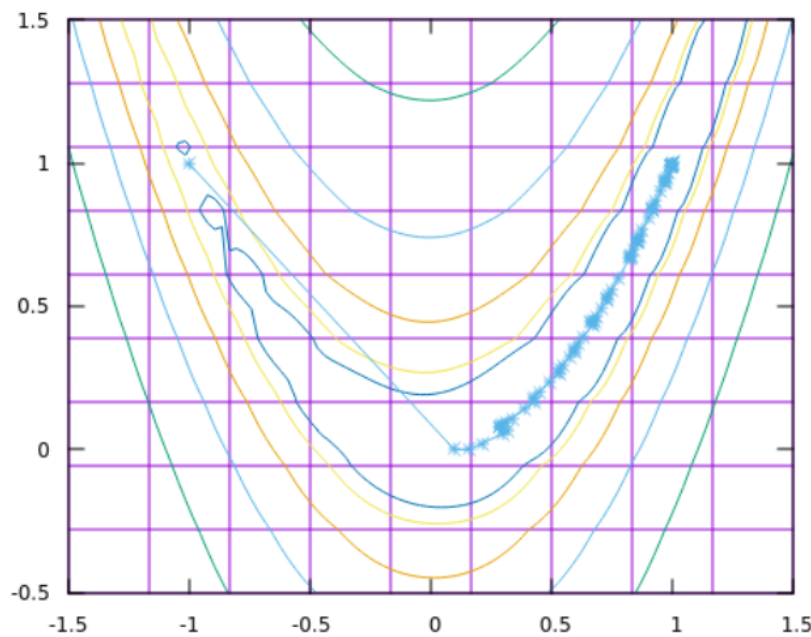


Figure 4.3: Iterations of the Fletcher-Reeves method (nonlinear conjugate gradient) on the level curves of the Rosenbrock function.

Key points to observe:

- **Efficient trajectory:** The method converges to the minimum $(1, 1)$ by exploiting conjugate directions, without explicitly using the Hessian.
- **Trade-off:** Fletcher-Reeves offers faster convergence than gradient descent, while avoiding the cost of computing the Hessian (necessary for Newton).
- **Adaptation to the nonlinear case:** Although conjugate gradient theory is optimal for quadratic functions, the method remains effective on this strongly non-quadratic function.

4.2.5 Global Descent Property

Unlike the quadratic case, the direction p_k generated by Fletcher-Reeves is not automatically a descent direction.

We have the relation:

$$\nabla f_k^\top p_k = -\|\nabla f_k\|^2 + \beta_{FR} \nabla f_k^\top p_{k-1}.$$

If the second term is large, the direction can go upward. To guarantee descent, strict conditions must be imposed on the line search.

Lemma 4.2 (Descent Condition). *Suppose that the step α_k satisfies the **strong Wolfe conditions** with a parameter $0 < c_2 < \frac{1}{2}$. Then, the direction p_k is always a descent direction and satisfies the following bounds for all k :*

$$-\frac{1}{1-c_2} \leq \frac{\nabla f_k^\top p_k}{\|\nabla f_k\|^2} \leq \frac{2c_2-1}{1-c_2}.$$

Proof. Preliminary. Consider the function $t(\xi) = \frac{2\xi-1}{1-\xi}$. This function is increasing on $[0, \frac{1}{2}]$. We have $t(0) = -1$ and $t(1/2) = 0$. Since $0 < c_2 < 1/2$, we have:

$$-1 < \frac{2c_2-1}{1-c_2} < 0.$$

Initialization ($k = 0$). We have $p_0 = -\nabla f_0$, so $\nabla f_0^\top p_0 = -\|\nabla f_0\|^2$. The ratio is:

$$\frac{\nabla f_0^\top p_0}{\|\nabla f_0\|^2} = -1.$$

The inequality is satisfied because:

$$-\frac{1}{1-c_2} \leq -1 < \frac{2c_2-1}{1-c_2} \quad (\text{verified because } c_2 < 1/2).$$

Induction step. Suppose the result is true for k . Let's compute the ratio for $k+1$:

$$\frac{\nabla f_{k+1}^\top p_{k+1}}{\|\nabla f_{k+1}\|^2} = \frac{\nabla f_{k+1}^\top (-\nabla f_{k+1} + \beta_{k+1}^{FR} p_k)}{\|\nabla f_{k+1}\|^2} = -1 + \beta_{k+1}^{FR} \frac{\nabla f_{k+1}^\top p_k}{\|\nabla f_{k+1}\|^2}.$$

Using the definition of $\beta_{FR} = \frac{\|\nabla f_{k+1}\|^2}{\|\nabla f_k\|^2}$, we simplify to:

$$\frac{\nabla f_{k+1}^\top p_{k+1}}{\|\nabla f_{k+1}\|^2} = -1 + \frac{\nabla f_{k+1}^\top p_k}{\|\nabla f_k\|^2}.$$

The curvature condition (second strong Wolfe condition) requires:

$$|\nabla f_{k+1}^\top p_k| \leq c_2 |\nabla f_k^\top p_k|.$$

Since p_k is a descent direction (induction hypothesis), $\nabla f_k^\top p_k < 0$. Thus $|\nabla f_k^\top p_k| = -\nabla f_k^\top p_k$. The inequality becomes:

$$|\nabla f_{k+1}^\top p_k| \leq -c_2 \nabla f_k^\top p_k.$$

This implies:

$$c_2 \nabla f_k^\top p_k \leq \nabla f_{k+1}^\top p_k \leq -c_2 \nabla f_k^\top p_k.$$

Dividing by $\|\nabla f_k\|^2$ and adding -1 , we obtain the bounds:

$$-1 + c_2 \frac{\nabla f_k^\top p_k}{\|\nabla f_k\|^2} \leq \frac{\nabla f_{k+1}^\top p_{k+1}}{\|\nabla f_{k+1}\|^2} \leq -1 - c_2 \frac{\nabla f_k^\top p_k}{\|\nabla f_k\|^2}.$$

Using the induction hypothesis on the right-hand term shows that the bounds are preserved for $k+1$. $\square$

4.3 Polak-Ribière Method

Another very popular variant, often considered more efficient in practice, is the Polak-Ribière (PR) method.

4.3.1 Definition and Properties

Polak and Ribière proposed the following formula for β :

$$\beta_{k+1}^{PR} = \frac{\nabla f_{k+1}^\top (\nabla f_{k+1} - \nabla f_k)}{\|\nabla f_k\|^2}.$$

Coincidence with FR. If the function f is strictly quadratic, then successive gradients are orthogonal ($\nabla f_{k+1}^\top \nabla f_k = 0$). In this case, the subtraction term vanishes and:

$$\beta_{k+1}^{PR} = \beta_{k+1}^{FR}.$$

PR therefore coincides with FR and the linear conjugate gradient on quadratic problems.

Descent problem. However, unlike Fletcher-Reeves, even strong Wolfe conditions do not guarantee that the direction p_k generated by Polak-Ribière is a descent direction.

4.3.2 β^{PR+} Variant

To guarantee generating descent directions, one frequently uses the following modification:

$$\beta_{k+1}^{PR+} = \max(\beta_{k+1}^{PR}, 0).$$

This amounts to resetting the direction to the steepest descent direction ($p_{k+1} = -\nabla f_{k+1}$) when β_{k+1}^{PR} becomes negative.

4.3.3 Practical Remarks and Convergence Rate

- **Performance:** In practice, the PR method is often better than FR for general nonlinear functions and when the line search is inexact.
- **Restart:** A common strategy is to "restart" the algorithm every n iterations by taking a Steepest Descent step. Note that this descent step corresponds to the initialization of the conjugate gradient method.
- **Convergence Rate:** With appropriate restarts, an n -step quadratic convergence can be proven:

$$\|x_{k+n} - x^*\| = O(\|x_k - x^*\|^2).$$

4.4 Global Convergence Analysis

Here we study the conditions under which the gradient descends to zero.

4.4.1 Assumptions (Assumptions A)

To establish convergence, we make the following assumptions on the function f and the level set $\mathcal{L} = \{x \in \mathbb{R}^n \mid f(x) \leq f(x_0)\}$:

1. The level set $\mathcal{L}$ is bounded.
2. f is continuously differentiable and its gradient is Lipschitz continuous in a neighborhood of $\mathcal{L}$.

4.4.2 Al-Baali's Theorem

This important result establishes the global convergence of the Fletcher-Reeves method under strong Wolfe conditions with a sufficiently small c_2 parameter.

Theorem 4.3 (Al-Baali). *Under assumptions A, if the Fletcher-Reeves algorithm is implemented with strong Wolfe conditions and parameters satisfying:*

$$0 < c_1 < c_2 < \frac{1}{2},$$

then the sequence of gradients converges to zero in the following sense:

$$\liminf_{k \rightarrow \infty} \|\nabla f_k\| = 0.$$

Checkpoint : Did you understand the essentials ?

- **Adaptation:** How do we go from linear CG to nonlinear CG (replacing the residual with the gradient, α_k with Line Search)?
- **β Formulas:** What is the difference between Fletcher-Reeves (FR) and Polak-Ribière (PR)? Which one is generally better in practice?
- **Restart:** Why do we "restart" the algorithm (steepest descent step) periodically?
- **Descent:** Does nonlinear CG always guarantee a descent direction? (No for PR without modification, yes for FR with Strong Wolfe).

Chapter 5

Quasi-Newton Methods

5.1 Introduction

Principle. The goal is to avoid computing the Hessian by using a symmetric positive definite matrix instead.

Scope of application. In this chapter, we focus on small to medium-sized problems. Indeed, these methods require solving linear systems; the issue of large problems is deferred to the next chapter, where such resolutions will be avoided.

5.2 The BFGS Method

The **Broyden, Fletcher, Goldfarb, Shanno** (BFGS) method is the most popular Quasi-Newton method.

Consider the following quadratic model of the objective function f at iteration k :

$$m_k(p) = f_k + \nabla f_k^\top p + \frac{1}{2} p^\top B_k p, \quad (5.1)$$

where B_k is a symmetric positive definite matrix that is updated at each iteration.

Note that the model satisfies the following consistency properties at $p = 0$:

- $m_k(0) = f_k = f(x_k)$
- $\nabla m_k(0) = \nabla f_k = \nabla f(x_k)$

The minimizer p_k of the quadratic model m_k is obtained by setting its gradient to zero:

$$\nabla m_k(p_k) = \nabla f_k + B_k p_k = 0. \quad (5.2)$$

This implies that the search direction is given by:

$$p_k = -B_k^{-1} \nabla f_k. \quad (5.3)$$

Once the direction p_k is obtained, we perform a line search to determine the step size α_k .

5.3 The Davidon-Fletcher-Powell (DFP) Update Formula

Davidon (1959) proposed updating B_k to account for the change in curvature that occurred since the last iteration.

Consider the new affine model at x_{k+1} :

$$m_{k+1}(p) = f_{k+1} + \nabla f_{k+1}^\top p + \frac{1}{2} p^\top B_{k+1} p. \quad (5.4)$$

What are reasonable requirements for updating B_k ?

5.3.1 The Secant Equation

For example, the gradient of m_{k+1} must coincide with the true gradient of f at points x_{k+1} and x_k .

1. At x_{k+1} (i.e., for $p = 0$):

$$\nabla m_{k+1}(0) = \nabla f_{k+1}.$$

This condition is verified by definition.

2. At x_k (i.e., for the inverse displacement $p = -\alpha_k p_k$): We want:

$$\nabla m_{k+1}(-\alpha_k p_k) = \nabla f_{k+1} + B_{k+1}(-\alpha_k p_k) = \nabla f_k. \quad (5.5)$$

Let's rearrange this last equation:

$$B_{k+1}(\alpha_k p_k) = \nabla f_{k+1} - \nabla f_k.$$

Let us introduce the vectors s_k (displacement) and y_k (change in gradient):

$$s_k = x_{k+1} - x_k = \alpha_k p_k, \quad (5.6)$$

$$y_k = \nabla f_{k+1} - \nabla f_k. \quad (5.7)$$

We then obtain the **secant equation**:

$$B_{k+1}s_k = y_k. \quad (5.8)$$

The matrix B_{k+1} must transform s_k into y_k .

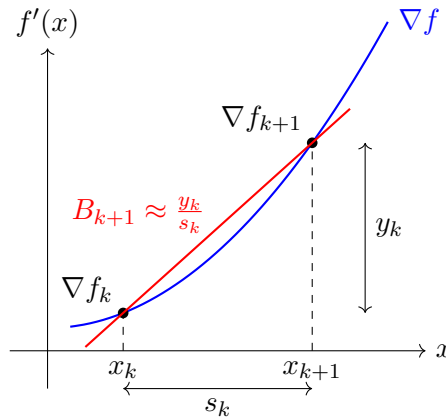


Figure 5.1: Geometric interpretation of the secant equation (1D case): B_{k+1} is the slope of the secant line to the ∇f curve.

Figure 5.1 illustrates the principle of the secant equation in 1 dimension. The matrix (here a scalar) B_{k+1} corresponds to the slope of the secant line (in red) connecting the points $(\nabla f_k, x_k)$ and $(\nabla f_{k+1}, x_{k+1})$ on the gradient curve (in blue). This slope approximates the average curvature of the function between these two points.

5.3.2 Curvature Condition

A necessary condition for B_{k+1} to be positive definite (as we want to ensure a descent direction) is obtained by multiplying the secant equation by $s_k^\top$ on the left:

$$s_k^\top B_{k+1} s_k = s_k^\top y_k.$$

If $B_{k+1} \succ 0$, then we must have:

$$s_k^\top y_k > 0. \quad (5.9)$$

This is called the **curvature condition**.

- This condition is always satisfied if f is strongly convex.
- Otherwise, we must impose it by enforcing the **Wolfe conditions** during the line search.

Proof using the Wolfe conditions

The second Wolfe condition is:

$$\nabla f_{k+1}^\top p_k \geq c_2 \nabla f_k^\top p_k. \quad (5.10)$$

Since $s_k = \alpha_k p_k$ with $\alpha_k > 0$, this is equivalent to:

$$\nabla f_{k+1}^\top s_k \geq c_2 \nabla f_k^\top s_k.$$

Thus:

$$\begin{aligned} y_k^\top s_k &= (\nabla f_{k+1} - \nabla f_k)^\top s_k \\ &\geq c_2 \nabla f_k^\top s_k - \nabla f_k^\top s_k \\ &= (c_2 - 1) \nabla f_k^\top s_k \\ &= (c_2 - 1) \alpha_k \nabla f_k^\top p_k. \end{aligned}$$

Since $c_2 < 1$ (hence $c_2 - 1 < 0$) and p_k is a descent direction ($\nabla f_k^\top p_k < 0$), the product is strictly positive:

$$y_k^\top s_k > 0.$$

5.3.3 Construction of the DFP Update

The secant equation $B_{k+1} s_k = y_k$ admits infinitely many solutions (because B_{k+1} has $n(n+1)/2$ degrees of freedom for only n constraints).

We need another criterion: choose B_{k+1} to be as "close" as possible to B_k . We solve:

$$B_{k+1} = \arg \min_B \|B - B_k\| \quad \text{s.t. } B = B^\top \text{ and } B s_k = y_k. \quad (5.11)$$

DFP chose the weighted Frobenius norm:

$$\|A\|_W = \|W^{1/2} A W^{1/2}\|_F, \quad \text{where } \|C\|_F^2 = \sum_{i,j} C_{ij}^2.$$

The weighting matrix W is chosen such that $W y_k = s_k$.

For instance, one can set $W = (\bar{G}_k)^{-1}$, where $\bar{G}_k$ is the average Hessian:

$$\bar{G}_k = \int_0^1 \nabla^2 f(x_k + \tau \alpha_k p_k) d\tau.$$

By Taylor's theorem:

$$\nabla f(x_{k+1}) = \nabla f(x_k) + \bar{G}_k (x_{k+1} - x_k),$$

which means $y_k = \bar{G}_k s_k$, verifying the choice of W .

With this choice of W , the solution is given by the DFP update formula:

$$B_{k+1} = (I - \rho_k y_k s_k^\top) B_k (I - \rho_k s_k y_k^\top) + \rho_k y_k y_k^\top, \quad (5.12)$$

where

$$\rho_k = \frac{1}{y_k^\top s_k}. \quad (5.13)$$

5.3.4 Expression for the Inverse (Sherman-Morrison-Woodbury Formula)

If we express the update in terms of the inverse Hessian $H_k = B_k^{-1}$, as in Newton's method where $p_k = -H_k \nabla f_k$, we can use the Sherman-Morrison-Woodbury formula.

The DFP update formula for the inverse is given by:

$$H_{k+1}^{DFP} = H_k - \frac{H_k y_k y_k^\top H_k}{y_k^\top H_k y_k} + \frac{s_k s_k^\top}{y_k^\top s_k}. \quad (5.14)$$

We note that this update corresponds to adding two rank-1 matrices (a rank-2 update). Furthermore, $\text{rank}(H_{k+1} - H_k) \leq 2$.

5.4 The BFGS Approach

The BFGS (Broyden-Fletcher-Goldfarb-Shanno) method uses the same reasoning as DFP, but works directly on the inverse Hessian H_k instead of B_k .

The secant equation for the inverse becomes:

$$H_{k+1} y_k = s_k. \quad (5.15)$$

We then look for H_{k+1} solving:

$$H_{k+1} = \arg \min_H \|H - H_k\|_W \quad \text{s.t. } H = H^\top \text{ and } H y_k = s_k. \quad (5.16)$$

The solution yields the **BFGS update formula** for the inverse:

$$H_{k+1}^{BFGS} = (I - \rho_k s_k y_k^\top) H_k (I - \rho_k y_k s_k^\top) + \rho_k s_k s_k^\top, \quad (5.17)$$

with $\rho_k = \frac{1}{y_k^\top s_k}$.

Algorithm 10: BFGS Method

- 1 Given starting point x_0 , convergence tolerance $\varepsilon > 0$, inverse Hessian approximation H_0 ;
 - 2 $k \leftarrow 0$;
 - 3 **while** $\|\nabla f_k\| > \varepsilon$ **do**
 - 4 Compute search direction $p_k = -H_k \nabla f_k$;
 - 5 Set $x_{k+1} = x_k + \alpha_k p_k$, where α_k is computed from a line search procedure to satisfy the Wolfe conditions;
 - 6 $s_k \leftarrow x_{k+1} - x_k$;
 - 7 $y_k \leftarrow \nabla f_{k+1} - \nabla f_k$;
 - 8 Compute $H_{k+1} = (I - \rho_k s_k y_k^\top) H_k (I - \rho_k y_k s_k^\top) + \rho_k s_k s_k^\top$;
 - 9 $k \leftarrow k + 1$;
-

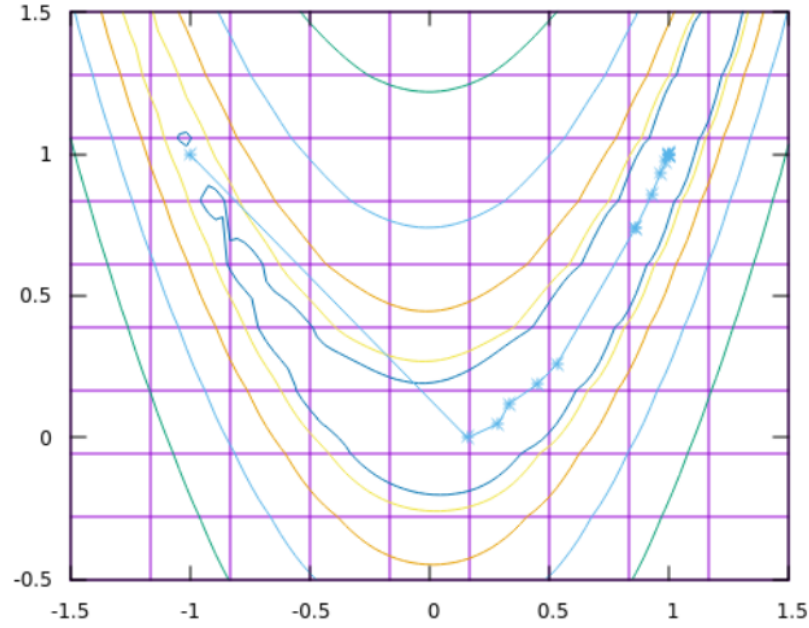


Figure 5.2: Iterations of the BFGS method on the level curves of the Rosenbrock function.

Key points to observe:

- **Direct trajectory:** Unlike gradient descent which zigzags, BFGS converges rapidly towards the minimum $(1, 1)$ by using the curvature approximation.
- **Local adaptation:** The method progressively learns the shape of the Hessian over the iterations, allowing it to "straighten" its search directions.
- **Comparison:** The number of iterations is much lower than that of gradient descent (corresponding Figure in Chapter 2), but slightly higher than Newton (which uses the true Hessian).

5.5 Limited-Memory Quasi-Newton Methods (L-BFGS)

For high-dimensional problems, storing the matrix H_k (of size $n \times n$) becomes prohibitive. The L-BFGS (Limited-memory BFGS) method solves this problem by storing only the last m pairs (s_k, y_k) .

Algorithm 11: L-BFGS two-loop recursion

```

1  $q \leftarrow \nabla f_k$ ;
2 for  $i = k - 1, k - 2, \dots, k - m$  do
3    $\alpha_i \leftarrow \rho_i s_i' q$ ;
4    $q \leftarrow q - \alpha_i y_i$ ;
5  $r \leftarrow H_k^0 q$ ;
6 for  $i = k - m, k - m + 1, \dots, k - 1$  do
7    $\beta \leftarrow \rho_i y_i' r$ ;
8    $r \leftarrow r + s_i(\alpha_i - \beta)$ ;
9 stop with result  $H_k \nabla f_k = r$ .
```

Algorithm 12: L-BFGS

```

1 Choose starting point  $x_0$ , integer  $m > 0$ ;
2  $k \leftarrow 0$ ;
3 repeat
4   Choose  $H_k^0$ , for instance  $H_k^0 = \frac{s'_{k-1}y_{k-1}}{y'_{k-1}y_{k-1}}I$ ;
5   Compute  $p_k \leftarrow -H_k \nabla f_k$  from Algorithm 11;
6   Compute  $x_{k+1} \leftarrow x_k + \alpha_k p_k$ , where  $\alpha_k$  is chosen to satisfy the Wolfe conditions;
7   if  $k > m$  then
8      $\lfloor$  Discard the vector pair  $\{s_{k-m}, y_{k-m}\}$  from storage;
9   Compute and save  $s_k \leftarrow x_{k+1} - x_k$ ,  $y_k = \nabla f_{k+1} - \nabla f_k$ ;
10   $k \leftarrow k + 1$ ;
11 until convergence;

```

5.6 Remarks and Properties

Initialization. There is no universal optimal initialization for H_0 . The possibilities are:

- Approximate the initial inverse Hessian.
- Take a multiple of the identity $H_0 = \gamma I$.

Cost. The computational cost of each iteration is $O(n^2)$ (matrix-vector multiplications and rank 1/2 updates), as opposed to $O(n^3)$ for Newton (which requires a system inversion).

Positive Definiteness. If H_k is positive definite, then H_{k+1} is too, provided that $y_k^\top s_k > 0$. Indeed, for any $z \neq 0$:

$$z^\top H_{k+1} z = z^\top (I - \rho s y^\top) H (I - \rho y s^\top) z + \rho (z^\top s)^2 \geq 0.$$

Strictly speaking, at this stage we only have ≥ 0 because $(I - \rho y s^\top)z$ and $z^\top s$ could be zero. However, if $z^\top s = 0$, then only the term $z^\top (I - \rho s y^\top) H (I - \rho y s^\top) z$ remains, which is > 0 because H is positive definite and $(I - \rho y s^\top)z \neq 0$ (otherwise $z = \rho(y^\top z)s$, which would imply $z^\top s = \rho(y^\top z)s^\top s \neq 0$, a contradiction). Thus $z^\top H_{k+1} z > 0$.

5.7 The Broyden Class

There is a family of updates, called the Broyden class, defined by:

$$B_{k+1} = (1 - \phi_k) B_{k+1}^{BFGS} + \phi_k B_{k+1}^{DFP}. \quad (5.18)$$

Or in explicit form:

$$B_{k+1} = B_k - \frac{B_k s_k s_k^\top B_k}{s_k^\top B_k s_k} + \frac{y_k y_k^\top}{y_k^\top s_k} + \phi_k (s_k^\top B_k s_k) v_k v_k^\top, \quad (5.19)$$

where ϕ_k is a scalar parameter and

$$v_k = \frac{y_k}{y_k^\top s_k} - \frac{B_k s_k}{s_k^\top B_k s_k}.$$

Special cases:

- If $\phi_k = 0$, we recover **BFGS**.
- If $\phi_k = 1$, we recover **DFP**.

Restricted Broyden Class. The restricted Broyden class corresponds to the choice $\phi_k \in [0, 1)$. The methods in this class are convex combinations of BFGS and DFP.

5.8 Properties on Quadratic Functions

Consider a strongly quadratic function $f(x) = \frac{1}{2}x^\top Qx - b^\top x$. Assume that B_0 is symmetric positive definite and that α_k is the exact step (exact line search).

Then we have the following properties (Admitted Theorem):

1. The generated iterates x_k are independent of the choice of ϕ_k in the Broyden class.
2. The secant equation is satisfied for all previous search directions:

$$B_k s_j = y_j, \quad \forall j = 0, \dots, k-1. \quad (5.20)$$

3. If we choose $B_0 = I$, then the iterates are identical to those of the conjugate gradient method, and the search directions are conjugate:

$$s_j^\top Q s_k = 0, \quad \forall j \neq k. \quad (5.21)$$

4. If we perform n iterations, then the Hessian matrix is identified exactly:

$$B_n = Q. \quad (5.22)$$

5.9 Convergence Analysis

5.9.1 Global Convergence

Remark. BFGS and restricted Broyden class methods ($\phi_k \in [0, 1)$) are very robust in practice. However, there is no global convergence result for general nonlinear (non-convex) functions.

We will state a result for convex functions.

Assumptions B.

- The objective function f is twice continuously differentiable (C^2).
- The level set $\mathcal{L} = \{x \in \mathbb{R}^n \mid f(x) \leq f(x_0)\}$ is convex.
- There exist constants $0 < m \leq M$ such that for any $z \in \mathbb{R}^n$ and any $x \in \mathcal{L}$:

$$m\|z\|^2 \leq z^\top \nabla^2 f(x) z \leq M\|z\|^2. \quad (5.23)$$

This implies that f is strongly convex on $\mathcal{L}$.

Theorem 5.1 (Global Convergence of BFGS). *Let B_0 be any symmetric positive definite matrix. Let x_0 be a point such that Assumptions B are verified. Then the sequence of iterates (x_k) generated by the BFGS algorithm (with $\epsilon = 0$) converges to the minimizer x^* of f (which is unique according to Assumptions B).*

Proof. See [9].

Remark. This theorem can be generalized to the entire restricted Broyden class.

5.9.2 Superlinear Convergence of BFGS

It is also possible to prove that the convergence is fast enough so that:

$$\sum_{k=0}^{\infty} \|x_k - x^*\| < \infty.$$

Assumption C. The Hessian matrix $G = \nabla^2 f$ is Lipschitz continuous at x^* : there exists $L > 0$ such that for x sufficiently close to x^* :

$$\|\nabla^2 f(x) - \nabla^2 f(x^*)\| \leq L\|x - x^*\|.$$

Theorem 5.2 (Admitted). Suppose that f is twice continuously differentiable (C^2). Consider the iterates $x_{k+1} = x_k + \alpha_k p_k$ where α_k satisfies the Wolfe conditions with $c_1 \leq \frac{1}{2}$. If the sequence (x_k) converges to a point x^* where $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ is positive definite. And if the search directions satisfy the Dennis-Moré condition:

$$\lim_{k \rightarrow \infty} \frac{\|\nabla f_k + \nabla^2 f_k p_k\|}{\|p_k\|} = 0, \quad (5.24)$$

then the value $\alpha_k = 1$ is admissible for all $k \geq k_0$ (where k_0 is a certain integer), and if one chooses $\alpha_k = 1$ for $k \geq k_0$, the convergence of (x_k) to x^* is **superlinear**.

Remark. If $p_k = -B_k^{-1} \nabla f_k$ is a quasi-Newton direction, then the above condition rewrites as:

$$\lim_{k \rightarrow \infty} \frac{\|(B_k - \nabla^2 f(x_k))p_k\|}{\|p_k\|} = 0.$$

Checkpoint : Did you understand the essentials ?

- **Secant Equation:** What relation must B_{k+1} (or H_{k+1}) satisfy with respect to s_k and y_k ?
- **BFGS vs DFP:** Which method is the most robust in practice?
- **Limited Memory:** How does L-BFGS reduce the space complexity from $O(n^2)$ to $O(mn)$?
- **Convergence:** How fast does BFGS converge (Superlinear)?

Chapter 6

Large-Scale Unconstrained Optimization Problems

6.1 Introduction

Practical applications can involve hundreds or thousands of variables. With the techniques from the previous chapters, we may be faced with high costs:

- Computational costs (CPU time).
- Storage costs (memory).

In this chapter, we will present techniques to overcome these difficulties.

6.2 Inexact Newton Methods

In Newton methods, we find the direction p_k^N by solving the linear system:

$$\nabla^2 f_k p_k^N = -\nabla f_k.$$

(Formally, $p_k^N = -(\nabla^2 f_k)^{-1} \nabla f_k$).

Our goal is to find an approximation p_k of p_k^N that is not too costly to compute: these are the inexact Newton methods. For example, we can use iterative methods such as the conjugate gradient to solve the system approximately.

The quality of the approximation will determine the convergence properties.

6.2.1 Local Convergence

It is reasonable to stop solving the system $\nabla^2 f_k p_k^N = -\nabla f_k$ when the residual

$$r_k = \nabla^2 f_k p_k + \nabla f_k$$

is small.

For instance, by using a **forcing sequence** $(\eta_k) \in (0, 1)^{\mathbb{N}}$ and stopping when:

$$\|r_k\| \leq \eta_k \|\nabla f_k\|. \quad (6.1)$$

The two theorems in this subsection apply to all inexact Newton methods satisfying the above condition. (Proofs: *Dembo et al.*)

Theorem 6.1 (Local convergence). *Suppose that $\nabla^2 f(x)$ exists and is continuous in a neighborhood of a minimizer x^* , with $\nabla^2 f(x^*)$ positive definite. Let $x_{k+1} = x_k + p_k$ (where we assume a unit step $\alpha_k = 1$ near the solution) where p_k satisfies:*

$$\|r_k\| \leq \eta_k \|\nabla f_k\|.$$

Suppose that $\eta_k \leq \eta$ for all k , where $\eta \in [0, 1)$.

Then, if x_0 is sufficiently close to x^ , the sequence (x_k) converges to x^* . Moreover, there exists a certain $\hat{\eta} \in (\eta, 1)$ such that:*

$$\|\nabla^2 f(x^*)(x_{k+1} - x^*)\| \leq \hat{\eta} \|\nabla^2 f(x^*)(x_k - x^*)\|.$$

6.2.2 Rate of Convergence

Theorem 6.2 (Rate of convergence). *Suppose the conditions of the previous theorem hold, and that (x_k) converges to x^* .*

- *The convergence rate is **superlinear** if $\lim_{k \rightarrow \infty} \eta_k = 0$.*
- *If $\nabla^2 f$ is Lipschitz-continuous near x^* and if $\eta_k = O(\|\nabla f_k\|)$, then the convergence is **quadratic**.*

Proof elements. We can show that:

$$\frac{\|\nabla f_{k+1}\|}{\|\nabla f_k\|} \leq \eta_k + o(1).$$

- If $\eta_k \rightarrow 0$, then the gradient (and thus the iterates) converges superlinearly.
- If $\nabla^2 f$ is Lipschitz continuous, we have $\nabla f_{k+1} = O(\|\nabla f_k\|^2)$, which implies quadratic convergence.

Remarks.

- The choice $\eta_k = \min(0.5, \sqrt{\|\nabla f_k\|})$ provides a **superlinear** convergence.
- The choice $\eta_k = \min(0.5, \|\nabla f_k\|)$ provides a **quadratic** convergence.
- These results are local: we assumed that x_0 was close to x^* and that $\alpha_k = 1$.

6.2.3 Newton-CG Methods with Line Search

The system $\nabla^2 f_k p_k = -\nabla f_k$ is solved using the conjugate gradient (CG) method. We must stop if we encounter a direction of negative curvature (i.e., if $\nabla^2 f_k$ is not positive definite).

6.3 Limited-Memory Quasi-Newton Methods (L-BFGS)

When the Hessian is not available at a reasonable cost (in time or storage), limited-memory Quasi-Newton methods are useful. They replace the storage of $n \times n$ matrices with the storage of a limited number of vectors of size n . Here we focus on the **L-BFGS** (Low-storage BFGS) method, which stores the curvature information from the m previous iterations. (Derivation: [8]).

6.3.1 Review of BFGS

The BFGS steps are $x_{k+1} = x_k - \alpha_k H_k \nabla f_k$, where H_k is updated according to:

$$H_{k+1} = V_k^\top H_k V_k + \rho_k s_k s_k^\top, \quad (6.2)$$

with $V_k = I - \rho_k y_k s_k^\top$ and $\rho_k = \frac{1}{y_k^\top s_k}$. We denote $s_k = x_{k+1} - x_k$ and $y_k = \nabla f_{k+1} - \nabla f_k$.

6.3.2 Principles of L-BFGS

- Instead of storing the dense matrix H_k , we store m vector pairs (s_i, y_i) .
- At each iteration, we replace the oldest pair (s_{k-m}, y_{k-m}) with the new one (s_k, y_k) .
- In practice, m is chosen between 3 and 20.
- At iteration k , we have the current iterate x_k and the pairs (s_i, y_i) for $i = k - m, \dots, k - 1$.
- We choose an initial approximation H_k^0 .

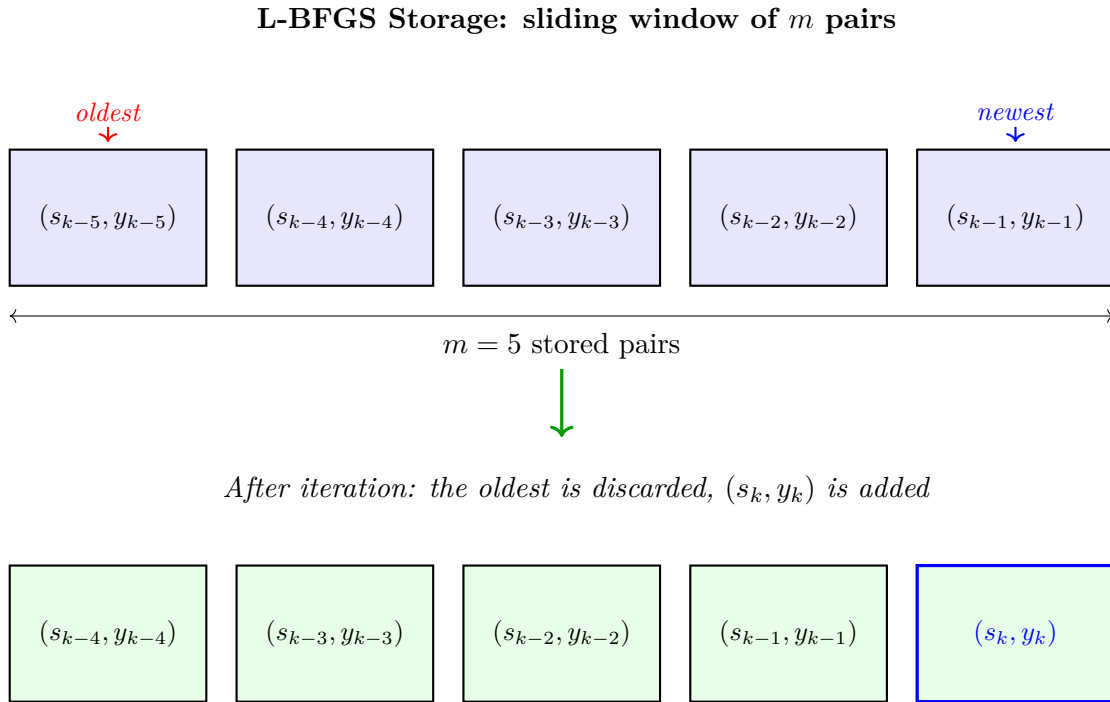


Figure 6.1: Principle of L-BFGS storage: sliding window of m pairs (s_i, y_i) . At each iteration, the oldest pair is eliminated and the new pair is added.

Figure 6.1 illustrates the storage principle of L-BFGS. Instead of maintaining the matrix H_k (of size $n \times n$), we only keep the last m pairs of vectors (s_i, y_i) . This approach reduces the storage cost from $O(n^2)$ to $O(mn)$, which is essential for large-scale problems.

We can unroll the update formula recursively:

$$\begin{aligned}
 H_k &= V_{k-1}^\top H_{k-1} V_{k-1} + \rho_{k-1} s_{k-1} s_{k-1}^\top \\
 &= V_{k-1}^\top (V_{k-2}^\top H_{k-2} V_{k-2} + \rho_{k-2} s_{k-2} s_{k-2}^\top) V_{k-1} + \rho_{k-1} s_{k-1} s_{k-1}^\top \\
 &= \dots \\
 &= (V_{k-1}^\top \dots V_{k-m}^\top) H_k^0 (V_{k-m} \dots V_{k-1}) \\
 &\quad + \rho_{k-m} (V_{k-1}^\top \dots V_{k-m+1}^\top) s_{k-m} s_{k-m}^\top (V_{k-m+1} \dots V_{k-1}) \\
 &\quad + \dots + \rho_{k-1} s_{k-1} s_{k-1}^\top.
 \end{aligned}$$

This formula allows computing the product $H_k \nabla f_k$ without ever forming the matrix H_k (see Algorithm 11, *two-loop recursion*).

Complexity. Algorithm 11 requires $4mn$ operations, plus the cost of computing $H_k^0 q$ (n multiplications if H_k^0 is diagonal).

Choice of H_k^0 . In practice, choosing $H_k^0 = \gamma_k I$ with:

$$\gamma_k = \frac{s_{k-1}^\top y_{k-1}}{y_{k-1}^\top y_{k-1}}, \quad (6.3)$$

gives satisfactory results. One can show that γ_k roughly approximates one of the eigenvalues of the Hessian.

Line Search. As for BFGS, we use the (strong) Wolfe conditions and initialize the line search with $\alpha_k = 1$.

Checkpoint : Did you understand the essentials ?

- **Inexact Newton:** What is the stopping criterion for solving the linear system?
- **Newton-CG:** Why is this method suitable for high dimensions (Matrix-vector products only)?
- **Negative Curvature Condition:** What does Newton-CG do if $p^T \nabla^2 f p < 0$?

Chapter 7

Derivative-Free Optimization

(Brent, 1973)

7.1 Introduction

Often, derivatives are unavailable or difficult to compute. Examples:

- The result of an experimental measurement.
- The result of a stochastic simulation.
- f is known but computing the derivatives is too expensive.

7.2 Some Possible Solutions

- **Automatic differentiation:** if a binary code computes f , we can apply automatic differentiation.

Principle. If $f(x) = g(h(m(x)))$, we denote $w = m(x)$, $y = h(w)$, $z = g(y)$. We use the *chain rule*:

$$f'(x) = \left. \frac{\partial g}{\partial y} \right|_y \times \left. \frac{\partial h}{\partial w} \right|_w \times \left. \frac{\partial m}{\partial x} \right|_x.$$

- **Finite differences:** use finite differences to approximate ∇f (and perhaps $\nabla^2 f$), then apply the algorithms from the previous chapters. **But** this may be impossible due to approximations or *noise*.

7.3 The Problem of Noise

In many applications, the objective function looks like:

$$f(x) = h(x) + \phi(x), \tag{7.1}$$

where h is a smooth (differentiable) function, and ϕ represents the noise (possibly independent of x).

The finite difference approximation of the gradient $\nabla f(x)$ for $\epsilon > 0$ is given by:

$$\nabla_\epsilon f(x) = \left(\frac{f(x + \epsilon e_i) - f(x - \epsilon e_i)}{2\epsilon} \right)_{i=1, \dots, n},$$

where $e_i = (0, \dots, 1, \dots, 0)^\top$ is the i -th vector of the canonical basis.

7.3.1 Noise Level and Approximation Error

Let us define the local noise level $\eta(x, \epsilon)$ by:

$$\eta(x, \epsilon) = \sup_{\|z-x\|_\infty < \epsilon} |\phi(z)|. \quad (7.2)$$

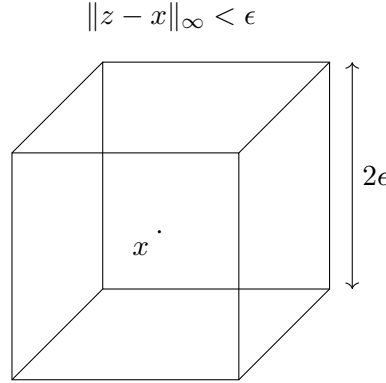


Figure 7.1: Neighborhood considered for the noise level.

Figure 7.1 represents the cubic neighborhood $\|z - x\|_\infty < \epsilon$ around the point x . The local noise level $\eta(x, \epsilon)$ corresponds to the largest amplitude of the noise $\phi(z)$ encountered inside this cube of side 2ϵ .

It can be shown that if $\nabla^2 h$ is L -Lipschitz continuous in the neighborhood $\{z \mid \|z - x\|_\infty < \epsilon\}$, then:

$$\|\nabla_\epsilon f(x) - \nabla h(x)\|_\infty \leq \underbrace{L\epsilon^2}_{\text{finite differences}} + \underbrace{\frac{\eta(x, \epsilon)}{\epsilon}}_{\text{noise}}. \quad (7.3)$$

Thus, one cannot expect a good approximation of $\nabla h(x)$ if the noise is significant, because reducing ϵ increases the amplification of the noise (η/ϵ).

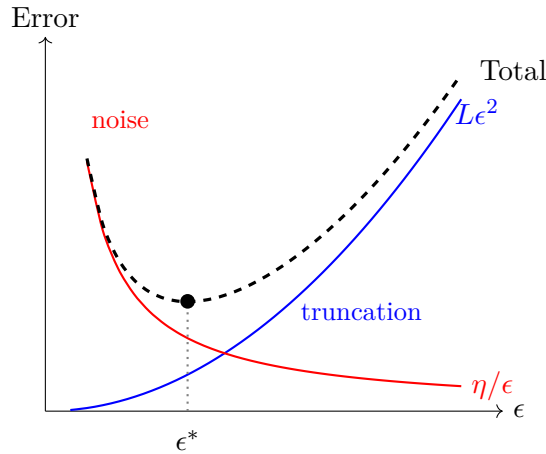


Figure 7.2: Trade-off truncation error vs noise amplification. The total error (dashed) has a minimum at ϵ^* .

7.3.2 Example: Stochastic Noise

Let $X_1, \dots, X_n$ be i.i.d. (independent and identically distributed) random variables with a mean $h(x)$ and a finite variance σ^2 .

Then the Central Limit Theorem gives:

$$\sqrt{n} \left(\frac{1}{n} \sum_{i=1}^n X_i - h(x) \right) \xrightarrow{d} \mathcal{N}(0, \sigma^2).$$

Roughly:

$$\frac{1}{n} \sum_{i=1}^n X_i - h(x) \approx \frac{\sigma}{\sqrt{n}} G,$$

where $G \sim \mathcal{N}(0, 1)$. This shows that the error decreases only as $1/\sqrt{n}$, which may require a very large number of samples to reduce the noise to an acceptable level for finite differences.

So that:

$$\underbrace{\frac{1}{n} \sum_{i=1}^n X_i}_{\text{observed}} \simeq \underbrace{h(x)}_{\text{useful part}} + \underbrace{\frac{\sigma}{\sqrt{n}} G}_{\text{noise } \phi}.$$

In this situation, ϕ is independent of x , and ϕ is even unbounded (since the normal distribution is unbounded).

Caution: From now on, we assume that f has continuous derivatives, even though these will not be used in the algorithms.

We propose a non-exhaustive selection of methods.

7.4 Model-based methods

7.4.1 General Idea

The idea is to build a model m_k (often quadratic) that interpolates the function f on a set of well-chosen points.

Suppose that at step k , we have an interpolation set of points $\mathcal{Y} = \{y_1, \dots, y_q\}$ with $y_i \in \mathbb{R}^n$ for $i = 1, \dots, q$, and that the current iterate x_k is one of them. Moreover, no point in $\mathcal{Y}$ has a function value lower than $f(x_k)$.

7.4.2 Building the model

Our model will be of the form:

$$m_k(x_k + p) = c + g^\top p + \frac{1}{2} p^\top G p, \quad (7.4)$$

where $c \in \mathbb{R}$, $g \in \mathbb{R}^n$, and $G \in \mathbb{R}^{n \times n}$ is symmetric.

We determine c , g and G by imposing the interpolation conditions:

$$m_k(y_i) = f(y_i), \quad i = 1, \dots, q. \quad (7.5)$$

There are $1 + n + \frac{n(n+1)}{2} = \frac{(n+1)(n+2)}{2}$ coefficients to adjust. If we choose $q = \frac{(n+1)(n+2)}{2}$, the system admits a unique solution (provided the points are well positioned).

Then, we compute the step p by solving the trust-region subproblem:

$$\min_p m_k(x_k + p) \quad \text{subject to } \|p\|_2 \leq \Delta, \quad (7.6)$$

for some trust-region radius Δ .

7.4.3 Update

- If the decrease is "significant", we will increase Δ for the next iteration.
- Otherwise, we update the set $\mathcal{Y}$ or reduce the trust-region radius Δ , and reject the step.

7.4.4 Initialization and Geometry

For the complete algorithm (Algorithm 13), initialization is often done by taking the $n + 1$ vertices and the $\frac{n(n+1)}{2}$ edge midpoints of a simplex in $\mathbb{R}^n$.

Definition 7.1 (Simplex). A simplex in $\mathbb{R}^n$ is a set of $n + 1$ vectors $y_1, \dots, y_{n+1}$ in $\mathbb{R}^n$. The simplex $\{y_1, \dots, y_{n+1}\}$ is called **non-degenerate** if the matrix

$$\begin{bmatrix} y_2 - y_1 & y_3 - y_1 & \cdots & y_{n+1} - y_1 \end{bmatrix}$$

is non-singular (i.e., the directions are independent).

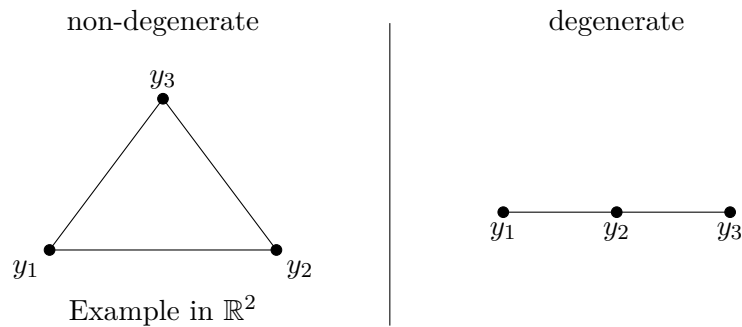


Figure 7.3: Illustration of a non-degenerate vs degenerate simplex in $\mathbb{R}^2$.

Figure 7.3 illustrates the concept of degeneracy. On the left, the vectors $y_2 - y_1$ and $y_3 - y_1$ are linearly independent, forming a true triangle (non-zero volume). On the right, the points are collinear: the vectors are collinear, the associated matrix is singular, and the simplex is degenerate (zero volume).

Algorithm 13: Model-Based Derivative-Free Method

```

1 Choose an interpolation set  $Y = \{y^1, \dots, y^q\}$  such that the linear system defined by
 $m_k(y^\ell) = f(y^\ell), \ell = 1, \dots, q$  is nonsingular, and select  $x_0$  as a point in this set such
that  $f(x_0) \leq f(y^i)$  for all  $y^i \in Y$ . Choose an initial trust region radius  $\Delta_0$ , a constant
 $\eta \in (0, 1)$ , and set  $k \leftarrow 0$ ;
2 repeat
3   Form the quadratic model  $m_k(x_k + p)$  that satisfies the interpolation conditions
 $m_k(y^\ell) = f(y^\ell), \ell = 1, \dots, q$ ;
4   Compute a step  $p$  by approximately solving  $\min_p m_k(x_k + p)$ , subject to  $\|p\|_2 \leq \Delta_k$ ;
5   Define the trial point  $x_k^+ = x_k + p$ ;
6   Compute the ratio  $\rho = \frac{f(x_k) - f(x_k^+)}{m_k(x_k) - m_k(x_k^+)}$ ;
7   if  $\rho \geq \eta$  then
8     Replace an element of  $Y$  by  $x_k^+$ ;
9     Choose  $\Delta_{k+1} \geq \Delta_k$ ;
10    Set  $x_{k+1} \leftarrow x_k^+$ ;
11    Set  $k \leftarrow k + 1$  and go to the next iteration;
12  else if the set  $Y$  need not be improved then
13    Choose  $\Delta_{k+1} < \Delta_k$ ;
14    Set  $x_{k+1} \leftarrow x_k$ ;
15    Set  $k \leftarrow k + 1$  and go to the next iteration;
16  Invoke a geometry-improving procedure to update  $Y$ : at least one of the points in  $Y$ 
is replaced by some other point, with the goal of improving the conditioning of
 $m_k(y^\ell) = f(y^\ell), \ell = 1, \dots, q$ ;
17  Set  $\Delta_{k+1} \leftarrow \Delta_k$ ;
18  Set  $x_k^+ \leftarrow \hat{x}$  and recompute  $\rho$  by  $\rho = \frac{f(x_k) - f(x_k^+)}{m_k(x_k) - m_k(x_k^+)}$ ;
19  if  $\rho \geq \eta$  then
20    Set  $x_{k+1} \leftarrow x_k^+$ ;
21  else
22    Set  $x_{k+1} \leftarrow x_k$ ;
23   $k \leftarrow k + 1$ ;
24 until a convergence test is satisfied;

```

7.5 Coordinate or pattern-search methods

Starting from the current iterate, we look at certain points along specified directions, and we decide the step.

7.5.1 Coordinate search

Also called *coordinate descent*.

Idea. We cycle through the n coordinate directions $e_1, \dots, e_n$. We minimize f along e_i , then we start again...

Warning. This method may never approach a point where the gradient vanishes (*gradient-vanishing point*)! It can stagnate at the corners of sharp level contours if the directions are not adapted to the local geometry of the function.

When convergence does occur, it may be slower than *steepest descent*.

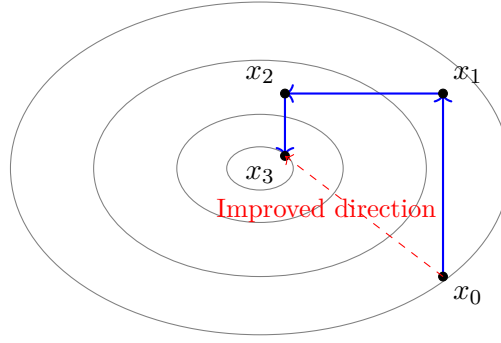


Figure 7.4: Illustration of coordinate search (zigzag) and its potential slowness.

Figure 7.4 shows that the method progresses in steps (zigzag). The coordinate directions do not necessarily point perfectly toward the minimum, forcing many small steps.

Possible Improvement. Perform n iterations and then run a line search along the direction $(x_k - x_{k-n})$.

7.5.2 Pattern-search methods

This class of methods generalizes the previous approach.

Idea. Choose a set of search directions D_k , and at each iteration, evaluate f at a given step length γ_k along these directions. If a "more interesting" point is found, it becomes the new iterate.

Algorithm 14 (General Scheme)

It can be shown that there exist stationary accumulation points. At step k , let x_k be the current iterate, D_k the current set of directions, and γ_k the step length.

- **Success:** If one of the directions in D_k produces a significant decrease, we move to this new point and increase the step γ_k .
- **Failure:** Otherwise, we keep x_k and decrease γ_k .
- Under certain conditions, the set D_k can be modified.

Conditions on D_k

To ensure convergence, the set D_k must sufficiently cover the space. There must be at least one descent direction in D_k , meaning a direction $d \in D_k$ such that:

$$\cos(\theta) = \frac{-\nabla f_k^\top d}{\|\nabla f_k\| \|d\|} > 0$$

Zoutendijk's Condition (adapted). Let us ensure that there exists $\delta > 0$ such that $\cos(\theta) > \delta$ for at least one direction. We define the quality measure of the set of directions $\kappa(D_k)$ by:

$$\kappa(D_k) = \min_{v \in \mathbb{R}^n} \max_{p \in D_k} \frac{v^\top p}{\|v\| \|p\|} \quad (7.7)$$

The condition is then: $\kappa(D_k) \geq \delta > 0$.

Interpretation: This condition means that for any vector v , there exists a direction $p \in D_k$ that forms an angle "sufficiently far" from $\pi/2$ with v . More precisely, the cosine of the angle between v and p is bounded from below by a strictly positive constant δ .

Bounds on the step

There must exist positive constants $\underline{\beta}, \bar{\beta}$ such that for all $p \in D_k$:

$$\underline{\beta} \leq \|p\|_2 \leq \bar{\beta}$$

The step γ_k must capture the effective diameter of the search.

Under these two conditions, for any k , there exists a vector $p \in D_k$ such that:

$$-\nabla f_k^\top p \geq \kappa(D_k) \|\nabla f_k\| \|p\| \geq \delta \|\nabla f_k\| \underline{\beta} \quad (7.8)$$

This guarantees a sufficient decrease as long as the gradient is not zero.

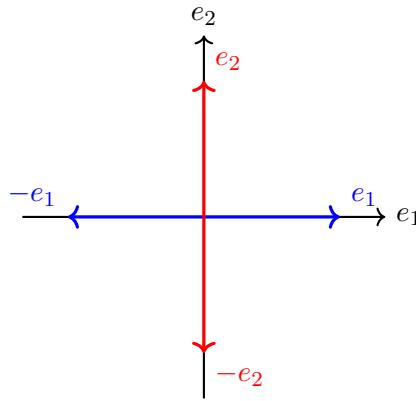
Possible Sets D_k satisfying these conditions

Here are a few examples of sets D_k that satisfy the stated conditions:

Example 1: Canonical directions and their opposites.

$$D_k = \{e_1, -e_1, e_2, -e_2, \dots, e_n, -e_n\} \quad (7.9)$$

where e_i is the i -th vector of the canonical basis.



Case $n = 2$

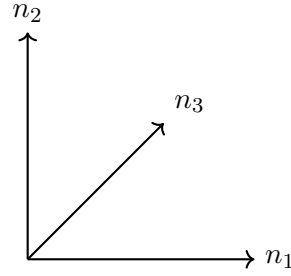
Figure 7.5: Direction set $D_k = \{e_1, -e_1, e_2, -e_2\}$ in $\mathbb{R}^2$.

Example 2: Simplex directions. For $n \geq 1$, we define:

$$n_i = \frac{1}{2n} e - e_i, \quad i = 1, \dots, n, \quad (7.10)$$

$$n_{n+1} = \frac{1}{2n} e, \quad (7.11)$$

where $e = (1, \dots, 1)^\top \in \mathbb{R}^n$.

Case $n = 2$ Figure 7.6: Simplex directions in $\mathbb{R}^2$.

Remark 7.2. • One can always enrich the set D_k with interesting heuristics.

- It is not necessary to examine all directions at every step.

Algorithm 14: Pattern search method

```

1 Given convergence tolerance  $\gamma_{tol}$ , contraction parameter  $\theta_{max}$ , sufficient decrease
  function  $\rho : [0, \infty) \rightarrow \mathbf{R}$  with  $\rho$  an increasing function and  $\frac{\rho(t)}{t} \rightarrow 0$  as  $t \downarrow 0$ ;
2 Choose initial point  $x_0$ , initial step length  $\gamma_0 > \gamma_{tol}$ , initial direction set  $\mathcal{D}_0$ ;
3 for  $k = 1, 2, \dots$  do
4   if  $\gamma_k \leq \gamma_{tol}$  then
5     stop;
6   if  $f(x_k + \gamma_k p_k) < f(x_k) - \rho(\gamma_k)$  for some  $p_k \in \mathcal{D}_k$  then
7     Set  $x_{k+1} \leftarrow x_k + \gamma_k p_k$  for some such  $p_k$ ;
8     Set  $\gamma_{k+1} \leftarrow \phi_k \gamma_k$  for some  $\phi_k \geq 1$ ; ▷ increase step length
9   else
10    Set  $x_{k+1} \leftarrow x_k$ ;
11    Set  $\gamma_{k+1} \leftarrow \theta_k \gamma_k$ , where  $0 < \theta_k \leq \theta_{max} < 1$ ;

```

7.6 Nelder-Mead Simplex Method

7.6.1 General Principle

At each step, we keep track of $n + 1$ points in $\mathbb{R}^n$, which form a simplex.

At each iteration:

- We remove a point from the simplex.
- We replace it with a point located on the line connecting the old point to the centroid of the n best points.
- The simplex will be **expanded**, **reflected**, or **contracted**.

If there is no such point that improves the function, we **shrink** the simplex towards the best point.

7.6.2 Notation and Ordering

At a given step, suppose the vertices of the simplex are ordered by increasing function value:

$$f(x_1) \leq f(x_2) \leq \dots \leq f(x_{n+1}) \quad (7.12)$$

Thus, x_1 is the best point (smallest value of f), and x_{n+1} is the worst point.

7.6.3 Centroid

The centroid of the n best points (i.e., all except x_{n+1}) is:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (7.13)$$

7.6.4 Candidate Points

The points on the line connecting x_{n+1} and $\bar{x}$ are given by:

$$\bar{x}(t) = \bar{x} + t(x_{n+1} - \bar{x}), \quad t \in \mathbb{R} \quad (7.14)$$

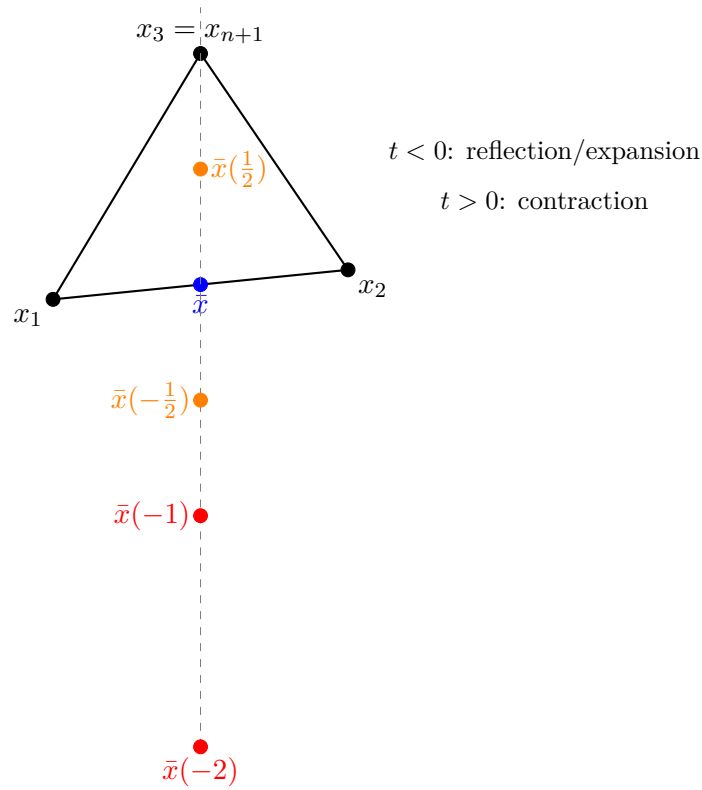


Figure 7.7: Candidate points on the line connecting x_{n+1} and the centroid $\bar{x}$.

In practice, we evaluate:

- $\bar{x}(-2)$: expansion
- $\bar{x}(-1)$: reflection
- $\bar{x}(-\frac{1}{2})$: outside contraction
- $\bar{x}(\frac{1}{2})$: inside contraction

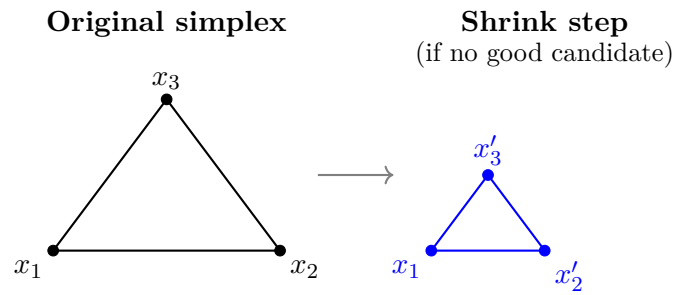
Figure 7.8: Shrink operation towards the best point x_1 .

Figure 7.8 illustrates the shrink operation. When none of the candidate points $\bar{x}(-2), \bar{x}(-1), \bar{x}(-\frac{1}{2}), \bar{x}(\frac{1}{2})$ improves the function, the simplex is shrunk towards the best vertex x_1 .

7.6.5 Remarks

Remark 7.3. The evaluation order $\bar{x}(\frac{1}{2}), \bar{x}(-\frac{1}{2}), \bar{x}(-1), \bar{x}(-2)$ is fairly standard in implementations.

Remark 7.4. Unless the shrinkage step is performed, the quantity

$$\frac{1}{n+1} \sum_{i=1}^{n+1} f(x_i) \quad (7.15)$$

decreases at each step.

Remark 7.5. For convergence theory, see [4] or [5].

7.6.6 Algorithm 15: Nelder-Mead Method

Algorithm 15: One step of the Nelder-Mead Method

```

1 Compute the reflection point  $\bar{x}(-1)$  and evaluate  $f_{-1} = f(\bar{x}(-1))$ ;
2 if  $f(x_1) \leq f_{-1} < f(x_n)$  then
    | ;▷ Reflected point is neither best nor worst in the new simplex
3 | Replace  $x_{n+1}$  by  $\bar{x}(-1)$  and go to the next iteration;
4 else if  $f_{-1} < f(x_1)$  then
    | ;▷ Reflected point is better than the current best, try to go further
    |   along this direction
5 | Compute the expansion point  $\bar{x}(-2)$  and evaluate  $f_{-2} = f(\bar{x}(-2))$ ;
6 | if  $f_{-2} < f_{-1}$  then
7 | | Replace  $x_{n+1}$  by  $\bar{x}(-2)$  and go to the next iteration;
8 | else
9 | | Replace  $x_{n+1}$  by  $\bar{x}(-1)$  and go to the next iteration;
10 else if  $f_{-1} \geq f(x_n)$  then
    | ;▷ Reflected point is still worse than  $x_n$ , contract
11 | if  $f(x_n) \leq f_{-1} < f(x_{n+1})$  then
    | | ;▷ try to perform "outside" contraction
12 | | Evaluate  $f_{-1/2} = f(\bar{x}(-1/2))$ ;
13 | | if  $f_{-1/2} \leq f_{-1}$  then
14 | | | Replace  $x_{n+1}$  by  $\bar{x}(-1/2)$  and go to the next iteration;
15 | else
    | | ;▷ Try to perform "inside" contraction
16 | | Evaluate  $f_{1/2} = f(\bar{x}(1/2))$ ;
17 | | if  $f_{1/2} < f(x_{n+1})$  then
18 | | | Replace  $x_{n+1}$  by  $\bar{x}(1/2)$  and go to the next iteration;
    | ;▷ Neither outside nor inside contraction was acceptable, shrink the
    |   simplex toward  $x_1$ 
19 | Replace  $x_i \leftarrow \frac{1}{2}(x_1 + x_i)$  for  $i = 2, 3, \dots, n+1$ ;

```

7.6.7 Illustration on the Rosenbrock Function

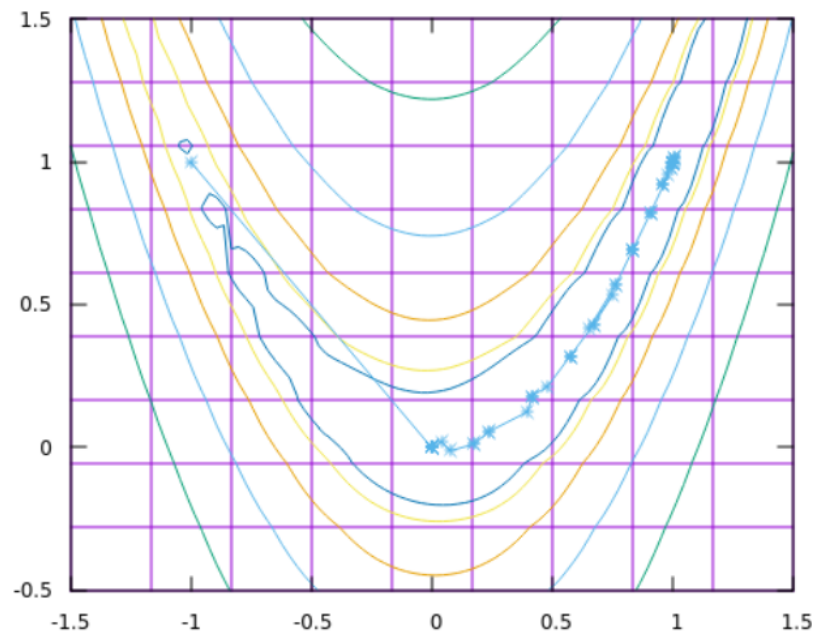


Figure 7.9: Iterations of the Nelder-Mead simplex method on the level curves of the Rosenbrock function.

Key points to observe:

- **Derivative-free method:** Nelder-Mead does not use any information about the gradient, only evaluations of f . The trajectory is therefore less direct than Newton or BFGS.
- **Progression in the valley:** We can observe how the simplex deforms and adapts to the geometry of the Rosenbrock valley through successive reflections and contractions.
- **Robustness vs efficiency:** The method converges despite the absence of derivatives, but the number of evaluations of f is much higher than steepest descent (because all other methods also use the gradient).
- **Use case:** This method is particularly useful when the derivatives are unavailable or too expensive to compute (simulations, experimental measurements).

Checkpoint : Did you understand the essentials ?

- **Finite differences:** What is the trade-off to find for the choice of the step size ϵ (Truncation Error vs Noise)?
- **Nelder-Mead:** Name the three main geometric operations on the simplex (Reflection, Expansion, Contraction).
- **Pattern Search:** What condition on the directions ensures convergence?

Chapter 8

Theory of Constrained Optimization

8.1 General Problem

We consider the following constrained optimization problem:

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad \begin{cases} c_i(x) = 0, & i \in \mathcal{E}, \\ c_i(x) \geq 0, & i \in \mathcal{I}, \end{cases} \quad (8.1)$$

where f and c_i are smooth functions, and $\mathcal{E}$ and $\mathcal{I}$ are finite index sets.

- f : objective function.
- $c_i, i \in \mathcal{E}$: equality constraints.
- $c_i, i \in \mathcal{I}$: inequality constraints.

8.2 Feasible Set

Definition 8.1 (Feasible Set). The feasible set is the set of points that satisfy all constraints:

$$\Omega = \{x \in \mathbb{R}^n \mid c_i(x) = 0, i \in \mathcal{E}; c_i(x) \geq 0, i \in \mathcal{I}\}. \quad (8.2)$$

The problem can then be equivalently written as:

$$\min_{x \in \Omega} f(x). \quad (8.3)$$

In this chapter, we will study the necessary and sufficient optimality conditions for this problem.

8.3 Local Solutions

Definition 8.2 (Local Solution). A vector $x^* \in \mathbb{R}^n$ is a **local solution** if:

1. $x^* \in \Omega$, and
2. there exists a neighborhood $\mathcal{N}$ of x^* such that:

$$f(x) \geq f(x^*) \quad \text{for all } x \in \mathcal{N} \cap \Omega. \quad (8.4)$$

Definition 8.3 (Strict Local Solution). A vector x^* is a **strict local solution** if:

1. $x^* \in \Omega$, and
2. there exists a neighborhood $\mathcal{N}$ of x^* such that:

$$f(x) > f(x^*) \quad \text{for all } x \in \mathcal{N} \cap \Omega, \quad x \neq x^*. \quad (8.5)$$

Definition 8.4 (Isolated Local Solution). A vector x^* is an **isolated local solution** if:

1. $x^* \in \Omega$, and
2. there exists a neighborhood $\mathcal{N}$ of x^* such that x^* is the only local solution in $\mathcal{N} \cap \Omega$.

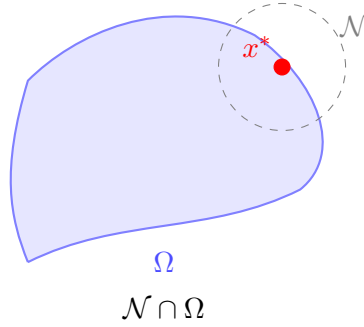


Figure 8.1: Local solution x^* on the boundary of Ω : the neighborhood $\mathcal{N}$ is not necessarily included in Ω .

8.4 Review: Smooth/Nonsmooth Problem Equivalence

Remark 8.5. If $g_1, \dots, g_m$ are smooth scalar functions, then the nonsmooth unconstrained problem:

$$\min_{x \in \mathbb{R}^n} \left[\max_{\mu=1, \dots, m} g_\mu(x) \right] \quad (8.6)$$

is equivalent to the smooth constrained problem:

$$\min_{(x,t) \in \mathbb{R}^{n+1}} t \quad \text{subject to} \quad t - g_\mu(x) \geq 0, \quad \mu = 1, \dots, m. \quad (8.7)$$

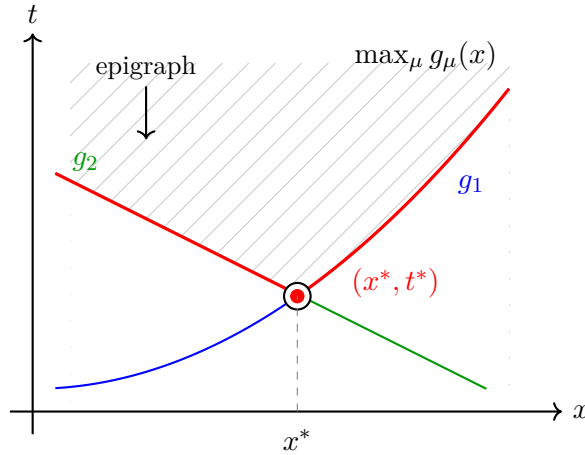


Figure 8.2: Equivalence between minimizing the max (nonsmooth) and the constrained problem (smooth). The variable t represents an upper bound on all functions g_μ . The hatched region represents the epigraph $\{(x, t) : t \geq \max_\mu g_\mu(x)\}$. For $x < x^*$, we have $\max(g_1, g_2) = g_2$; for $x > x^*$, we have $\max(g_1, g_2) = g_1$.

Figure 8.2 illustrates how the problem of minimizing the maximum of several functions (red envelope, which is non-differentiable at the intersection point) can be reformulated as a smooth problem by introducing an auxiliary variable t that upper bounds all functions $g_\mu(x)$. The optimal solution (x^*, t^*) is located at the lowest point of the envelope, i.e., where the two functions cross.

8.5 Linear Programming: The Simplex Method

For the special case of linear programming, we have an efficient algorithm: the simplex method.

8.5.1 Standard Form

A linear programming problem in standard form is written as:

$$\min_{x \in \mathbb{R}^n} c^\top x \quad \text{s.t.} \quad Ax = b, \quad x \geq 0, \quad (8.8)$$

where $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$.

8.5.2 Basic and Nonbasic Variables

We partition the columns of A into:

- $\mathcal{B}$: set of indices of the basic variables ($|\mathcal{B}| = m$).
- $\mathcal{N}$: set of indices of the nonbasic variables ($|\mathcal{N}| = n - m$).

We then have $B = A_{\mathcal{B}}$ (basis matrix, invertible) and $N = A_{\mathcal{N}}$.

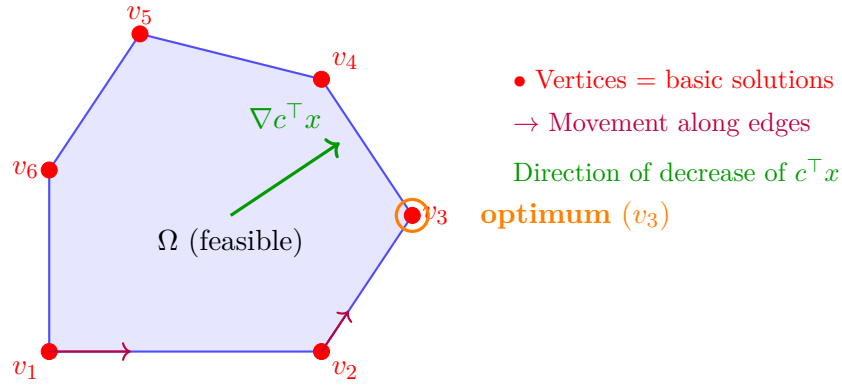


Figure 8.3: Geometry of linear programming. The feasible set Ω is a polyhedron. The vertices correspond to basic feasible solutions. The simplex method moves from vertex to vertex along the edges.

Figure 8.3 illustrates the geometry of the linear programming problem. Basic feasible solutions correspond to the vertices of the polyhedron. The simplex method starts at a vertex and moves to an adjacent vertex that improves the objective function, until reaching the optimum (here v_3).

8.5.3 Algorithm 16: One Step of the Simplex Method

Algorithm 16: One step of the Simplex Method

- 1 Given $\mathcal{B}$, $\mathcal{N}$, $x_B = B^{-1}b \geq 0$, $x_N = 0$;
 - 2 Solve $B^T \lambda = c_B$ for λ ;
 - 3 Compute $s_N = c_N - N^T \lambda$;
 - ▷ Pricing
 - 4 **if** $s_N \geq 0$ **then**
 - 5 **stop** ;
 - ▷ optimal point found
 - 6 Select $q \in \mathcal{N}$ with $s_q < 0$ as the entering index;
 - 7 Solve $Bd = A_q$ for d ;
 - 8 **if** $d \leq 0$ **then**
 - 9 **stop** ;
 - ▷ problem is unbounded
 - 10 Calculate $x_q^+ = \min_{i|d_i > 0} \left(\frac{(x_B)_i}{d_i} \right)$, use p to denote the minimizing i ;
 - 11 Update $x_B^+ = x_B - dx_q^+$, $x_N^+ = (0, \dots, 0, x_q^+, 0, \dots, 0)^T$;
 - 12 Change $\mathcal{B}$ by adding q and removing the basic variable corresponding to column p of B ;
-

8.6 Practice Exercises (Nocedal & Wright)

Exercise 8.1 (Constrained Optimization - KKT Rigor). Consider the problem:

$$\min_{x \in \mathbb{R}^2} f(x) = x_1 + x_2$$

subject to the constraint $c_1(x) = x_1^2 + x_2^2 - 2 = 0$.

1. Write the Lagrangian and the first-order KKT conditions.
2. Find the candidate points (x^*, λ^*) .
3. Verify the constraint qualification (LICQ) at these points.
4. Verify the second-order sufficient conditions by explicitating the critical (tangent) cone $\mathcal{C}(x^*, \lambda^*)$.

Suggested Correction. **1. KKT (First Order):** $L(x, \lambda) = x_1 + x_2 - \lambda(x_1^2 + x_2^2 - 2)$. Conditions (FONC):

$$\nabla_x L = \begin{pmatrix} 1 - 2\lambda x_1 \\ 1 - 2\lambda x_2 \end{pmatrix} = 0$$

$$c_1(x) = x_1^2 + x_2^2 - 2 = 0$$

2. Candidate Points: $1 = 2\lambda x_1 \implies x_1 = 1/(2\lambda)$ (if $\lambda \neq 0$). $1 = 2\lambda x_2 \implies x_2 = 1/(2\lambda)$. Thus $x_1 = x_2$. Constraint: $2x_1^2 = 2 \implies x_1^2 = 1 \implies x_1 = \pm 1$. Two candidates: - $P_1 = (1, 1)$. Then $2\lambda = 1 \implies \lambda = 1/2$. - $P_2 = (-1, -1)$. Then $-2\lambda = 1 \implies \lambda = -1/2$.

3. Qualification (LICQ): Gradient of the constraint: $\nabla c_1(x) = (2x_1, 2x_2)^\top$. - At $P_1(1, 1)$: $\nabla c_1 = (2, 2)^\top \neq 0$. Independent vectors (only 1 non-zero vector). LICQ OK. - At $P_2(-1, -1)$: $\nabla c_1 = (-2, -2)^\top \neq 0$. LICQ OK.

4. Second Order (SOSC): Hessian of the Lagrangian: $\nabla_{xx}^2 L(x, \lambda) = \nabla^2 f - \lambda \nabla^2 c_1$. $\nabla^2 f = 0$ (linear), $\nabla^2 c_1 = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} = 2I$. Thus $W = \nabla_{xx}^2 L = -2\lambda I$.

At $P_1(1, 1)$ with $\lambda = 1/2$: $W = -I$. Critical cone $C(P_1) = \ker(\nabla c_1^\top) = \{w \in \mathbb{R}^2 \mid (2, 2) \cdot w = 0\} = \{w \mid w_1 + w_2 = 0\}$. Vector in the cone: $w = (1, -1)^\top$. Test: $w^\top W w = (1, -1)(-I) \begin{pmatrix} 1 \\ -1 \end{pmatrix} = -(1^2 + (-1)^2) = -2 < 0$. Condition not satisfied (negative curvature). Thus P_1 is a local **maximum**.

At $P_2(-1, -1)$ with $\lambda = -1/2$: $W = -2(-1/2)I = I$. Critical cone $C(P_2) = \{w \mid -2w_1 - 2w_2 = 0\} = \{w \mid w_1 + w_2 = 0\}$. Test: $w^\top W w = w^\top I w = \|w\|^2 > 0$ for all $w \neq 0$. Sufficient condition satisfied. P_2 is a strict local **minimum**.

Exercise 8.2 (Inequalities and Critical Cone). Consider $\min(x_1 - 1)^2 + x_2^2$ s.t. $x_1 \geq 0, x_2 \geq 0$. Is the point $x^* = (1, 0)$ a solution?

1. Verify KKT.
2. Explicitate the critical cone $\mathcal{C}(x^*, \lambda^*)$. (Pay attention to zero/active multipliers).
3. Verify SOSC.

Suggested Correction. $f(x) = (x_1 - 1)^2 + x_2^2$. Constraints $c_1(x) = x_1 \geq 0, c_2(x) = x_2 \geq 0$. $\nabla f = \begin{pmatrix} 2(x_1 - 1) \\ 2x_2 \end{pmatrix}$. **1. KKT at $(1, 0)$:** $\nabla f(1, 0) = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$. Active constraints: $c_2(x) = x_2 \geq 0$ is active ($x_2 = 0$). c_1 inactive ($x_1 = 1 > 0$). Relation: $\nabla f(x^*) = \lambda_1 \nabla c_1 + \lambda_2 \nabla c_2$. $\begin{pmatrix} 0 \\ 0 \end{pmatrix} = 0 \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \lambda_2 \begin{pmatrix} 0 \\ 1 \end{pmatrix}$. $\implies \lambda_2 = 0$. $\lambda_1 = 0$ (since inactive). Multipliers are ≥ 0 . KKT satisfied. Remark: $\lambda_2 = 0$ even though the constraint is active (degeneracy / non-strict complementarity).

2. Critical Cone: $\mathcal{A}(x^*) = \{2\}$. $C(x^*, \lambda^*) = \{w \in \mathbb{R}^2 \mid \nabla c_i^\top w = 0 \text{ if } \lambda_i > 0, \text{ and } \nabla c_i^\top w \geq 0 \text{ if } \lambda_i = 0\}$. Here $\lambda_2 = 0$ for active constraint 2. $\nabla c_2 = (0, 1)^\top$. Thus conditions: $\nabla c_2^\top w \geq 0 \implies w_2 \geq 0$. The critical cone is the upper half-plane $\{w \mid w_2 \geq 0\}$. (This is much larger than the simple kernel $\ker \nabla c$).

3. Second Order: $\nabla^2 f = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$. $W = \nabla^2 f$ (since $\lambda = 0$). For any non-zero $w \in C$: $w^\top \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} w = 2(w_1^2 + w_2^2) > 0$. SOSC satisfied. It is a strict local minimum.

Checkpoint : Did you understand the essentials ?

- **Definitions:** What is an active constraint?
- **Qualifications:** What does LICQ mean? (Linear Independence of gradients of active constraints).
- **Geometry:** What is the difference between the tangent cone and the linearized cone?

Chapter 9

Optimality Conditions

9.1 Introduction and Definitions

Definition 9.1 (Active Set). Let $x \in \Omega$ be a feasible point. The active set, denoted $\mathcal{A}(x)$, consists of the indices of the equality constraints and the indices of the inequality constraints that are active (equal to 0) at x .

$$\mathcal{A}(x) = \mathcal{E} \cup \{i \in \mathcal{I} \mid c_i(x) = 0\}.$$

- A constraint $i \in \mathcal{I}$ is said to be **active** if $c_i(x) = 0$ (so $i \in \mathcal{A}(x)$).
- It is said to be **inactive** if $c_i(x) > 0$ (so $i \notin \mathcal{A}(x)$).

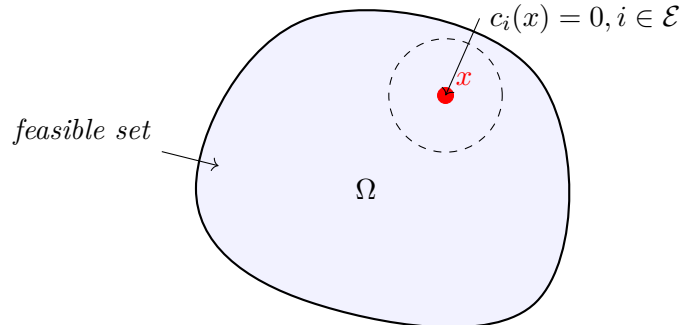


Figure 9.1: Illustration of the feasible set Ω and a point x on the boundary where some constraints are active.

Geometric interpretation:

- The set Ω is the feasible region (in blue).
- The point x is on the boundary. Constraints are active ($c_i(x) = 0$) if the boundary passes exactly through x .
- The other constraints (inactive) satisfy $c_i(x) > 0$ and do not *locally* limit infinitesimal movements around x .

9.2 Examples

9.2.1 One Equality Constraint

Consider the following minimization problem in $\mathbb{R}^2$:

$$\min_{(x_1, x_2) \in \mathbb{R}^2} x_1 + x_2 \quad \text{subject to} \quad x_1^2 + x_2^2 - 2 = 0. \quad (9.1)$$

We have:

$$\begin{aligned} f(x) &= x_1 + x_2, \\ c_1(x) &= x_1^2 + x_2^2 - 2, \\ \mathcal{E} &= \{1\}, \quad \mathcal{I} = \emptyset. \end{aligned}$$

The feasible set is the circle centered at $(0, 0)$ with radius $\sqrt{2}$. Let's compute the gradients:

$$\nabla f(x) = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \nabla c_1(x) = \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix}. \quad (9.2)$$

Note that $\nabla f(x)$ is constant (the same at every point), unlike $\nabla c_1(x)$ which depends on the position on the circle.

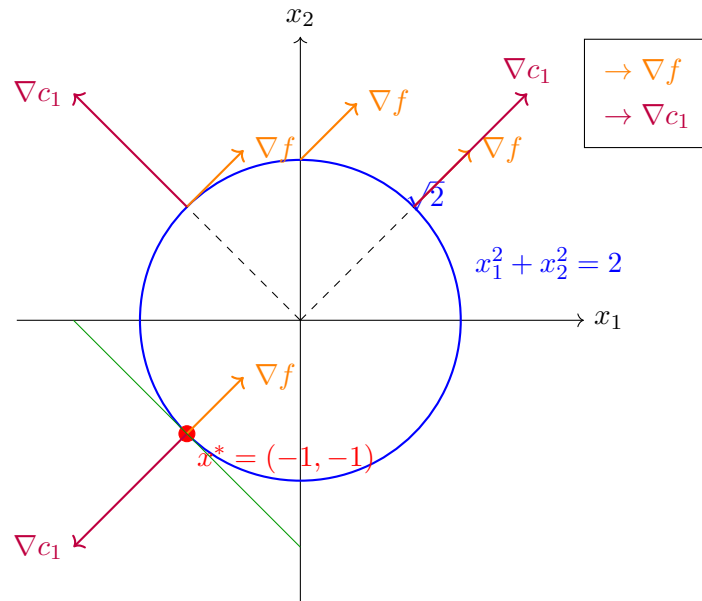


Figure 9.2: Minimization of $x_1 + x_2$ on the circle. At the optimal point $x^* = (-1, -1)$, the gradients ∇f and ∇c_1 are collinear and pointing in opposite directions. Note that $\|\nabla c_1(x)\| = 2\|\nabla f(x)\|$ on the circle.

Comments:

- **Variation of the objective function:** From any point on the circle (other than the optimum), it is possible to move along the circle ("reduce the function while staying on the circle") to decrease the value of f .
- **Behavior of the gradients:** The gradient of the objective function ∇f is the same at every point (constant vector $(1, 1)$), unlike the constraint vectors $\nabla c_1(x)$ which change direction depending on the point considered on the circle (radial vectors).
- **Optimality condition:** At the optimum $x^* = (-1, -1)$, it is no longer possible to decrease f while remaining on the circle. Geometrically, this corresponds to the point where the level curves of f are tangent to the circle. Algebraically, we observe that $\nabla f(x^*)$ and $\nabla c_1(x^*)$ are collinear.

Remark 9.2. At the point $x^* = (-1, -1)$, the gradient of the constraint $\nabla c_1(x^*) = \begin{pmatrix} -2 \\ -2 \end{pmatrix}$ is parallel to the gradient of the objective function $\nabla f(x^*) = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$.

There exists a scalar $\lambda_1^* \in \mathbb{R}$ such that:

$$\nabla f(x^*) = \lambda_1^* \nabla c_1(x^*). \quad (9.3)$$

In this example, we find $\lambda_1^* = -1/2$.

Derivation (Heuristic justification) Let x be a feasible point. A small step s starting from x remains feasible if, to first order:

$$0 = c_1(x + s) \approx \underbrace{c_1(x)}_{=0} + \nabla c_1(x)^\top s. \quad (9.4)$$

Therefore, to stay on the constraint surface (to first order), the step s must be orthogonal to the gradient of the constraint: $\nabla c_1(x)^\top s = 0$.

Moreover, the step s produces a decrease in the objective function f if:

$$0 > f(x + s) - f(x) \approx \nabla f(x)^\top s \quad (\text{to first order}). \quad (9.5)$$

That is, $\nabla f(x)^\top s < 0$ (for a sufficiently small step s).

If x^* is a local minimum, there must be no direction s that is both feasible (tangent to the constraint) and a descent direction. This implies that the gradient $\nabla f(x^*)$ must be aligned with $\nabla c_1(x^*)$. If $\nabla f(x^*)$ had a non-zero component in the tangent space, we could find an s reducing f .

This suggests that we are looking for a direction d (for example with $\|d\| = 1$) such that:

$$\begin{cases} \nabla c_1(x)^\top d = 0 & (\text{tangent to the surface}) \\ \nabla f(x)^\top d < 0 & (\text{descent}) \end{cases} \quad (9.6)$$

If there is no such d , then it is likely that x is a local minimizer.

This can only be the case if $\nabla f(x)$ and $\nabla c_1(x)$ are parallel, i.e.:

$$\nabla f(x) = \lambda_1 \nabla c_1(x) \quad \text{for some } \lambda_1 \in \mathbb{R}. \quad (9.7)$$

Otherwise, we can explicitly construct a *projected* descent direction using the following formula:

$$\bar{d} = - \left(I - \frac{\nabla c_1(x) \nabla c_1(x)^\top}{\|\nabla c_1(x)\|^2} \right) \nabla f(x). \quad (9.8)$$

By normalizing $d = \frac{\bar{d}}{\|\bar{d}\|}$ (if $\bar{d} \neq 0$), this direction satisfies $\nabla c_1(x)^\top d = 0$ and $\nabla f(x)^\top d < 0$.

Remark 9.3 (Orthogonal projector). The matrix $P = I - \frac{\nabla c_1(x) \nabla c_1(x)^\top}{\|\nabla c_1(x)\|^2}$ is the **orthogonal projector** onto the null space of $\nabla c_1(x)^\top$, i.e., onto the tangent space to the constraint surface at x . Thus, $\bar{d}$ is the projection of $-\nabla f(x)$ (steepest descent direction) onto this tangent space.



Figure 9.3: Comparison of cases: On the left, ∇f is not collinear to ∇c_1 , the projection $\bar{d}$ provides a tangent descent direction. On the right, the gradients are collinear, no tangent descent is possible (optimality condition).

Interpretation: The projected direction $\bar{d}$ only exists if ∇f and ∇c_1 are not collinear. When they are parallel, any tangent component of $-\nabla f$ is zero: we cannot descend while staying on the constraint.

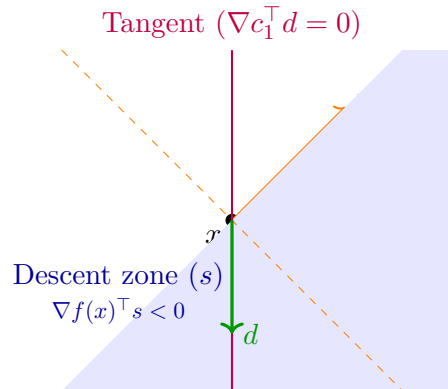


Figure 9.4: Illustration of the direction search. The blue zone represents the set of descent directions (forming an obtuse angle with ∇f). The vertical line represents the directions tangent to the constraint. Here, a direction d exists (intersection of the line and the blue zone), so x is not an optimum.

Interpretation: We look for a direction d such that $\nabla f^\top d < 0$ (descent) and $\nabla c_1^\top d = 0$ (tangent). If the tangent line passes through the blue zone, such a direction exists and x is not optimal.

Lagrangian function Let us define the Lagrangian function:

$$\mathcal{L}(x, \lambda_1) = f(x) - \lambda_1 c_1(x). \quad (9.9)$$

Its gradient with respect to x is:

$$\nabla_x \mathcal{L}(x, \lambda_1) = \nabla f(x) - \lambda_1 \nabla c_1(x). \quad (9.10)$$

The condition for the gradients to be parallel can then be rewritten as follows:

$$\text{There exists } \lambda_1^* \in \mathbb{R} \text{ such that } \nabla_x \mathcal{L}(x^*, \lambda_1^*) = 0.$$

This suggests that we must look for the stationary points of $\mathcal{L}$ to find the constrained minimizers of f .

Remark 9.4. The point $(1, 1)$ is also a stationary point of $\mathcal{L}$ (with a different Lagrange multiplier). It corresponds here to the global maximum of f on the circle.

9.2.2 One Inequality Constraint

Now consider the problem with one inequality constraint:

$$\min_{(x_1, x_2) \in \mathbb{R}^2} x_1 + x_2 \quad \text{subject to} \quad 2 - x_1^2 - x_2^2 \geq 0. \quad (9.11)$$

The constraint is $c_1(x) = 2 - x_1^2 - x_2^2 \geq 0$. The feasible set is the closed disk of radius $\sqrt{2}$. The solution is still the same point $x^* = (-1, -1)$.

Let's compute the gradients at x^* :

$$\begin{aligned} \nabla f(x^*) &= \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \\ \nabla c_1(x) &= \begin{pmatrix} -2x_1 \\ -2x_2 \end{pmatrix} \implies \nabla c_1(x^*) = \begin{pmatrix} 2 \\ 2 \end{pmatrix}. \end{aligned}$$

The parallelism condition:

$$\nabla f(x^*) = \lambda_1^* \nabla c_1(x^*) \quad (9.12)$$

is satisfied for $\lambda_1^* = \frac{1}{2}$. Note that this time, $\lambda_1^* > 0$.

Remark 9.5 (Interpretation of the positive multiplier). The fact that $\lambda_1^* > 0$ indicates that the constraint is **binding**: relaxing this constraint would allow f to be decreased further. Indeed, if one could "go outside" the disk (i.e. move in the direction $-\nabla f$), one could continue to decrease the objective function. The multiplier λ^* quantifies this "pressure" of the constraint at the optimum.

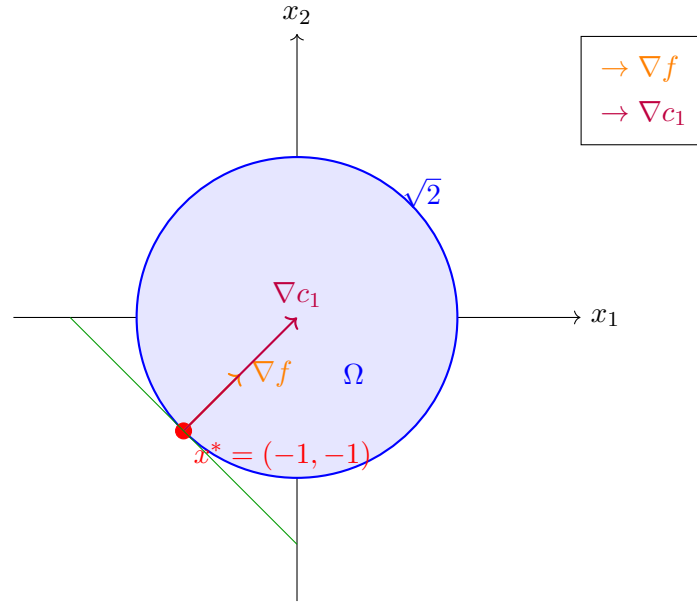


Figure 9.5: Minimization with an inequality constraint. At the optimal point, ∇f and ∇c_1 point in the same direction ($\lambda^* > 0$).

Interpretation: Unlike the equality case, the gradients point in the **same direction**. The multiplier $\lambda^* > 0$ reflects the fact that the constraint "pushes" the optimum: without it, we could descend further.

Derivation (Heuristic justification) Let x be a feasible point. A small step s starting from x produces a decrease in the objective function if, to first order:

$$\nabla f(x)^\top s < 0. \quad (9.13)$$

This step preserves feasibility with respect to the constraint c_1 if:

$$0 \leq c_1(x + s) \approx c_1(x) + \nabla c_1(x)^\top s. \quad (9.14)$$

Thus, to first order, we must have:

$$c_1(x) + \nabla c_1(x)^\top s \geq 0. \quad (9.15)$$

We then distinguish two cases:

1. **x is strictly inside the domain** (i.e. $c_1(x) > 0$).

Any sufficiently small step s will ensure that $c_1(x) + \nabla c_1(x)^\top s \geq 0$. For instance, one can choose the direction of steepest descent $s = -\alpha \nabla f(x)$ with $\alpha > 0$ small (if $\nabla f(x) \neq 0$). In this case, the constraint is not *active* and does not locally limit the minimization.

2. **x is on the boundary of the feasible set** (i.e. $c_1(x) = 0$).

In this case, the conditions for function decrease and feasibility become:

$$\nabla f(x)^\top s < 0 \quad \text{and} \quad \nabla c_1(x)^\top s \geq 0. \quad (9.16)$$

At the optimum, there must be no such s . This implies a specific relationship between the gradients (see KKT condition later).

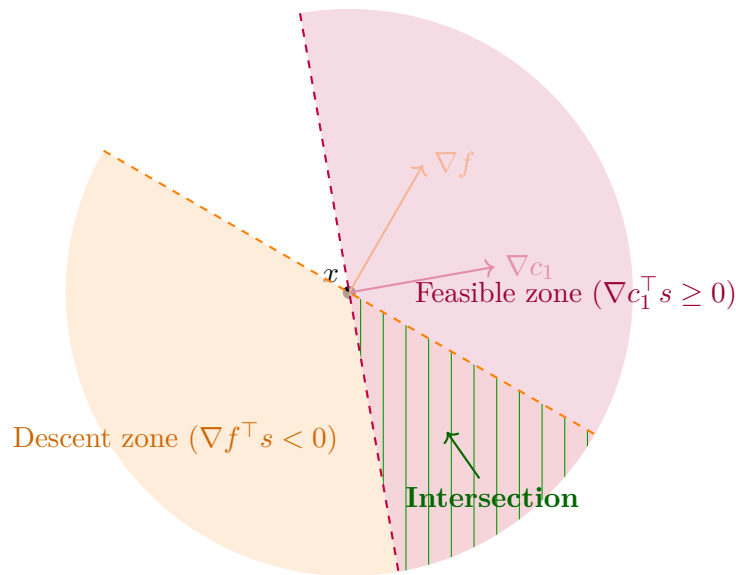
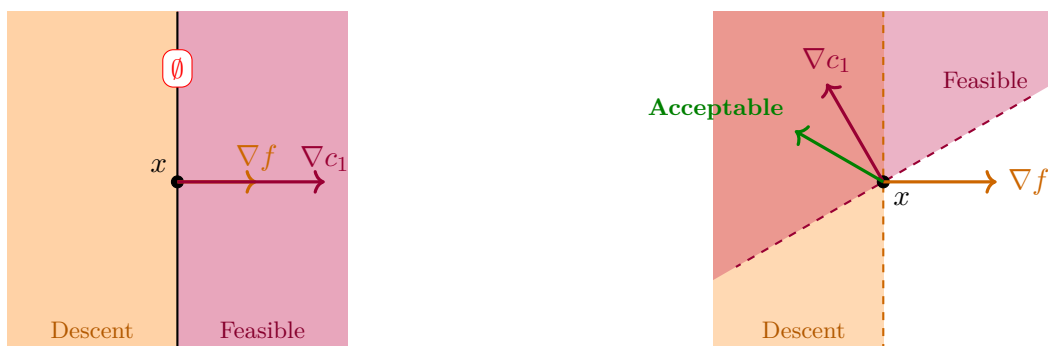


Figure 9.6: Visualization of acceptable directions. The two colored zones (orange for descent, purple for feasibility) overlap. The intersection (hatched in green) represents the set of acceptable directions.

Geometric interpretation:

- For a direction s to be a **descent direction**, it must form an obtuse angle with the gradient of the objective function ∇f (orange zone).
- For it to be **feasible** (to first order), it must point towards the interior of the domain with respect to the active constraint, hence forming an acute angle with the gradient of the constraint ∇c_1 (purple zone).
- The **intersection** (green hatching) contains the acceptable directions. As long as this intersection is not empty, the solution can be improved.

The fundamental question then becomes: **When is there no acceptable direction?**



(a) Aligned gradients: disjoint zones, no acceptable direction.

(b) Regular case: The intersection of the zones (color mixing) gives the acceptable directions.

Figure 9.7: Comparison of "Half-plane" configurations. On the left, parallel gradients in the same direction create a total obstruction (perfect opposition of zones).

Geometric interpretation:

- **Case (a):** The descent and feasibility zones are disjoint (no overlap). This is the optimality condition.
- **Case (b):** The two zones overlap through color transparency. The intersection (darker) gives the acceptable directions.

We deduce the following conclusion:

There is no acceptable direction when the gradients are parallel and point in the same direction:

$$\nabla f(x) = \lambda \nabla c_1(x) \quad \text{with } \lambda > 0. \quad (9.17)$$

Summary of the two cases (Optimality conditions) When there is no feasible descent direction, we have the following system:

$$\begin{cases} \nabla_x \mathcal{L}(x, \lambda_1^*) = 0 & \text{for some } \lambda_1^* \geq 0, \\ \lambda_1^* c_1(x) = 0. \end{cases} \quad (9.18)$$

The second equation is called the **complementarity condition**. It means that the multiplier λ_1^* can only be (strictly) positive if the constraint c_1 is active ($c_1(x) = 0$).

Recall the definition of the Lagrangian:

$$\mathcal{L}(x, \lambda_1) = f(x) - \lambda_1 c_1(x). \quad (9.19)$$

If $\lambda_1 = 0$, then $\mathcal{L}(x, \lambda_1) = f(x)$ and $\nabla_x \mathcal{L} = \nabla f(x)$. This corresponds to the case where the optimum is inside the domain (inactive constraint), and we recover the classic condition $\nabla f(x) = 0$.

9.2.3 Two Inequality Constraints

Consider the following problem:

$$\min_{(x_1, x_2) \in \mathbb{R}^2} x_1 + x_2 \quad \text{subject to} \quad \begin{cases} 2 - x_1^2 - x_2^2 \geq 0, \\ x_2 \geq 0. \end{cases} \quad (9.20)$$

The feasible set is the upper half of the disk of radius $\sqrt{2}$. The optimal point is $x^* = (-\sqrt{2}, 0)$.

Let's compute the gradients at this point:

$$\begin{aligned} \nabla f(x) &= \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \\ \nabla c_1(x) &= \begin{pmatrix} -2x_1 \\ -2x_2 \end{pmatrix} \implies \nabla c_1(x^*) = \begin{pmatrix} 2\sqrt{2} \\ 0 \end{pmatrix}, \\ \nabla c_2(x) &= \begin{pmatrix} 0 \\ 1 \end{pmatrix} \implies \nabla c_2(x^*) = \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \end{aligned}$$

Note that both constraints are active at x^* ($c_1(x^*) = 0$ and $c_2(x^*) = 0$).

We would expect a feasible descent direction d to satisfy:

$$\begin{cases} \nabla c_i(x^*)^\top d \geq 0 & \text{for } i = 1, 2 \quad (\text{feasible}), \\ \nabla f(x^*)^\top d < 0 & (\text{descent}). \end{cases} \quad (9.21)$$

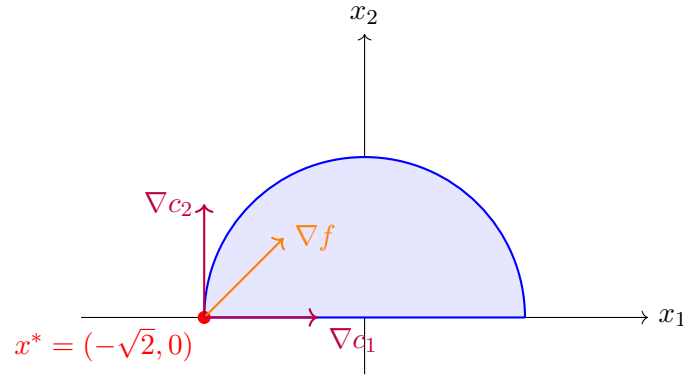


Figure 9.8: Example with two active constraints. Both constraints (disk and non-negativity) are active at x^* .

Situation with multiple active constraints: Here, the point x^* is at the intersection of two boundaries ($c_1 = 0$ for the circle, $c_2 = 0$ for the horizontal line).

- The gradient ∇f must be a linear combination with positive coefficients of the gradients of the active constraints (∇c_1 and ∇c_2).
- Geometrically, this means that $-\nabla f$ (desired descent direction) points outside the cone formed by the tangents to the constraints. We are "blocked" by both walls at once.

Modification of the Lagrangian For this problem with two constraints, we define the Lagrangian:

$$\mathcal{L}(x, \lambda) = f(x) - \lambda_1 c_1(x) - \lambda_2 c_2(x). \quad (9.22)$$

The conditions become:

- $\nabla_x \mathcal{L}(x^*, \lambda^*) = 0$ with $\lambda^* \geq 0$ (i.e., $\lambda_1^* \geq 0$ and $\lambda_2^* \geq 0$).
- Complementarity conditions: $\lambda_1^* c_1(x^*) = 0$ and $\lambda_2^* c_2(x^*) = 0$.

Let's check these conditions for our point $x^* = (-\sqrt{2}, 0)$. We look for λ_1^*, λ_2^* such that:

$$\begin{aligned} \nabla f(x^*) - \lambda_1^* \nabla c_1(x^*) - \lambda_2^* \nabla c_2(x^*) &= 0 \\ \begin{pmatrix} 1 \\ 1 \end{pmatrix} - \lambda_1^* \begin{pmatrix} 2\sqrt{2} \\ 0 \end{pmatrix} - \lambda_2^* \begin{pmatrix} 0 \\ 1 \end{pmatrix} &= \begin{pmatrix} 0 \\ 0 \end{pmatrix}. \end{aligned}$$

This system gives:

$$\begin{cases} 1 - 2\sqrt{2}\lambda_1^* = 0 \implies \lambda_1^* = \frac{1}{2\sqrt{2}} > 0, \\ 1 - \lambda_2^* = 0 \implies \lambda_2^* = 1 > 0. \end{cases} \quad (9.23)$$

The multipliers are positive, which is consistent with the fact that both constraints are active and restrict descent.

Analysis of another candidate point Consider the point $x = (\sqrt{2}, 0)$. This point is feasible (on the boundary). Let's check if the optimality conditions are satisfied.

$$\begin{aligned}\nabla f(x) &= \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \\ \nabla c_1(x) &= \begin{pmatrix} -2\sqrt{2} \\ 0 \end{pmatrix}, \\ \nabla c_2(x) &= \begin{pmatrix} 0 \\ 1 \end{pmatrix}.\end{aligned}$$

We look for λ_1^*, λ_2^* such that:

$$\begin{pmatrix} 1 \\ 1 \end{pmatrix} - \lambda_1^* \begin{pmatrix} -2\sqrt{2} \\ 0 \end{pmatrix} - \lambda_2^* \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}. \quad (9.24)$$

This gives the system:

$$\begin{cases} 1 + 2\sqrt{2}\lambda_1^* = 0 \implies \lambda_1^* = -\frac{1}{2\sqrt{2}} < 0, \\ 1 - \lambda_2^* = 0 \implies \lambda_2^* = 1 \geq 0. \end{cases} \quad (9.25)$$

Here, we find $\lambda_1^* < 0$. The KKT condition $\lambda^* \geq 0$ is not met. **Conclusion:** The point $x = (\sqrt{2}, 0)$ is not a local minimum (indeed, we could increase x_1 to decrease f while staying in the domain).

9.3 Tangent Cone and Constraint Qualifications

In this section, we will study the conditions under which analytical and geometrical points of view coincide.

9.3.1 Definitions

Definition 9.6 (Feasible Sequence). A sequence $(z_k)_k$ is said to be a **feasible sequence approaching the feasible point** x if:

- $z_k \rightarrow x$ as $k \rightarrow \infty$,
- $z_k \in \Omega$ for large enough k .

Definition 9.7 (Tangent Vector). A vector $d \in \mathbb{R}^n$ is said to be **tangent** (or tangent vector) to Ω at the feasible point x if there exists a feasible sequence $(z_k)_k$ approaching x and a sequence of positive scalars $(t_k)_k$ with $t_k \rightarrow 0$ such that:

$$\lim_{k \rightarrow \infty} \frac{z_k - x}{t_k} = d. \quad (9.26)$$

The set of tangent vectors to Ω at x is denoted $T_\Omega(x)$ and is called the **tangent cone** to Ω at x .

Remark 9.8. $T_\Omega(x)$ is indeed a cone:

- $0 \in T_\Omega(x)$ (take the constant sequence $z_k = x$).
- If $d \in T_\Omega(x)$, then for all $\alpha > 0$, $\alpha d \in T_\Omega(x)$.
Indeed, one just replaces the sequence $(t_k)_k$ by $(t'_k)_k$ defined by $t'_k = \alpha^{-1}t_k$. As $t_k \rightarrow 0$, we also have $t'_k \rightarrow 0$, and

$$\lim_{k \rightarrow \infty} \frac{z_k - x}{t'_k} = \lim_{k \rightarrow \infty} \alpha \frac{z_k - x}{t_k} = \alpha d.$$

Definition 9.9 (Linearized Feasible Directions). Given a feasible point x and the active set $\mathcal{A}(x)$, the set of **linearized feasible directions** $\mathcal{F}(x)$ is defined by:

$$\mathcal{F}(x) = \left\{ d \in \mathbb{R}^n \mid \begin{array}{l} d^\top \nabla c_i(x) = 0 \quad \forall i \in \mathcal{E}, \\ d^\top \nabla c_i(x) \geq 0 \quad \forall i \in \mathcal{I} \cap \mathcal{A}(x) \end{array} \right\}. \quad (9.27)$$

Remark 9.10. $\mathcal{F}(x)$ is also a cone.

Example 9.11 (Tangent cone for an equality constraint). Consider the problem:

$$\min_{x \in \mathbb{R}^2} x_1 + x_2 \quad \text{subject to} \quad x_1^2 + x_2^2 - 2 = 0. \quad (9.28)$$

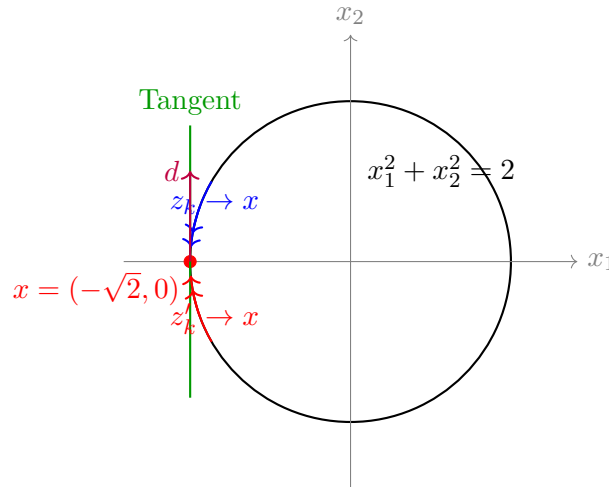
Let $x = (-\sqrt{2}, 0)$. The **feasible sequences** $(z_k)_k$ approaching x must lie on the circle. Consider the feasible sequences:

$$z_k = \left(-\sqrt{2 - \frac{1}{k^2}}, \frac{1}{k} \right) \quad \text{or} \quad z'_k = \left(-\sqrt{2 - \frac{1}{k^2}}, -\frac{1}{k} \right). \quad (9.29)$$

Let's check they are on the circle: $\left(-\sqrt{2 - 1/k^2} \right)^2 + (\pm 1/k)^2 = 2 - 1/k^2 + 1/k^2 = 2$.

With $t_k = \|z_k - x\|$, we find that the vectors $(0, 1)$ and $(0, -1)$ are tangent. More generally, the set of tangent vectors is the vertical line:

$$T_\Omega(x) = \{(0, d_2) \mid d_2 \in \mathbb{R}\}. \quad (9.30)$$



Now let's compute $\mathcal{F}(x)$ at $x = (-\sqrt{2}, 0)$. We have $\mathcal{E} = \{1\}$ and $\mathcal{I} = \emptyset$, so $\mathcal{A}(x) = \{1\}$. The constraint $c_1(x) = x_1^2 + x_2^2 - 2$ has the gradient:

$$\nabla c_1(x) = \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix} \implies \nabla c_1(x) = \begin{pmatrix} -2\sqrt{2} \\ 0 \end{pmatrix}. \quad (9.31)$$

The set of linearized feasible directions is:

$$\mathcal{F}(x) = \left\{ d = \begin{pmatrix} d_1 \\ d_2 \end{pmatrix} \in \mathbb{R}^2 \mid d^\top \nabla c_1(x) = 0 \right\}. \quad (9.32)$$

The condition $d^\top \nabla c_1(x) = 0$ gives:

$$\begin{pmatrix} d_1 \\ d_2 \end{pmatrix}^\top \begin{pmatrix} -2\sqrt{2} \\ 0 \end{pmatrix} = -2\sqrt{2}d_1 = 0 \implies d_1 = 0. \quad (9.33)$$

So:

$$\mathcal{F}(x) = \left\{ (0, d_2)^\top \mid d_2 \in \mathbb{R} \right\} = T_\Omega(x). \quad (9.34)$$

Remark 9.12 (Cases where $\mathcal{F}(x) \neq T_\Omega(x)$). • If we change the constraint to $(x_1^2 + x_2^2 - 2)^2 = 0$, it can be shown that $\mathcal{F}(x) = \mathbb{R}^2 \neq T_\Omega(x)$.

Indeed, the gradient of this new constraint at x vanishes:

$$\nabla \left[(x_1^2 + x_2^2 - 2)^2 \right] = 2(x_1^2 + x_2^2 - 2) \cdot \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix} = 0 \text{ on the circle.}$$

Thus the condition $d^\top \nabla c(x) = 0$ is always satisfied, hence $\mathcal{F}(x) = \mathbb{R}^2$.

- If we change the constraint to $2 - x_1^2 - x_2^2 \geq 0$ (inequality), then:

$$T_\Omega(x) = \left\{ (d_1, d_2)^\top \in \mathbb{R}^2 \mid d_1 \geq 0 \right\} = \mathcal{F}(x). \quad (9.35)$$

9.3.2 Linear Independence Constraint Qualification (LICQ)

Definition 9.13 (LICQ – Linear Independence Constraint Qualification). Given a feasible point $x \in \Omega$ and the active set $\mathcal{A}(x)$, we say that the **LICQ** condition is satisfied at x if the set of gradients of the active constraints

$$\{\nabla c_i(x) \mid i \in \mathcal{A}(x)\} \quad (9.36)$$

is linearly independent.

Proposition 9.14. Under the LICQ condition, the tangent cone $T_\Omega(x)$ and the set of linearized feasible directions $\mathcal{F}(x)$ coincide:

$$T_\Omega(x) = \mathcal{F}(x). \quad (9.37)$$

Lemma 9.15 (Case of linear constraints). Suppose that at a point $x^* \in \Omega$, all active constraints c_i (for $i \in \mathcal{A}(x^*)$) are **linear** (or affine) functions. Then:

$$T_\Omega(x^*) = \mathcal{F}(x^*). \quad (9.38)$$

Remark 9.16. This lemma is a special case of the previous proposition. Linear constraints automatically satisfy LICQ if their gradients are linearly independent. Moreover, for affine constraints, the linear approximation is exact, which explains the equality $T_\Omega = \mathcal{F}$.

9.4 Optimality Conditions

In this section, we formalize the optimality conditions for constrained optimization problems. For detailed proofs, we refer to Nocedal and Wright [9].

9.4.1 The Lagrangian Function

Definition 9.17 (Lagrangian Function). For the optimization problem

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad c_i(x) \begin{cases} = 0 & \text{if } i \in \mathcal{E}, \\ \geq 0 & \text{if } i \in \mathcal{I}, \end{cases} \quad (9.39)$$

the **Lagrangian function** is defined by:

$$\mathcal{L}(x, \lambda) = f(x) - \sum_{i \in \mathcal{E} \cup \mathcal{I}} \lambda_i c_i(x), \quad (9.40)$$

where $\lambda = (\lambda_i)_{i \in \mathcal{E} \cup \mathcal{I}}$ is the vector of **Lagrange multipliers**.

Remark 9.18. The sign convention can vary among authors. Our choice (minus sign before the λ_i) implies that for inequality constraints, the optimal multipliers will be positive $\lambda_i^* \geq 0$, which corresponds to the standard economic interpretation.

9.4.2 KKT Conditions

Theorem 9.19 (Karush-Kuhn-Tucker (KKT) Conditions). *Let x^* be a local minimum of the optimization problem. If LICQ is satisfied at x^* , then there exists a multiplier vector $\lambda^* = (\lambda_i^*)_{i \in \mathcal{E} \cup \mathcal{I}}$ such that:*

1. **Stationarity:**

$$\nabla_x \mathcal{L}(x^*, \lambda^*) = \nabla f(x^*) - \sum_{i \in \mathcal{E} \cup \mathcal{I}} \lambda_i^* \nabla c_i(x^*) = 0. \quad (9.41)$$

2. **Primal Feasibility:**

$$c_i(x^*) = 0 \quad \forall i \in \mathcal{E}, \quad c_i(x^*) \geq 0 \quad \forall i \in \mathcal{I}. \quad (9.42)$$

3. **Dual Feasibility:**

$$\lambda_i^* \geq 0 \quad \forall i \in \mathcal{I}. \quad (9.43)$$

(No sign constraint for λ_i^* with $i \in \mathcal{E}$.)

4. **Complementarity:**

$$\lambda_i^* c_i(x^*) = 0 \quad \forall i \in \mathcal{I}. \quad (9.44)$$

Remark 9.20 (Interpretation of Complementarity). The complementarity condition $\lambda_i^* c_i(x^*) = 0$ means that:

- If an inequality constraint $i \in \mathcal{I}$ is not active ($c_i(x^*) > 0$), then the corresponding multiplier must be zero ($\lambda_i^* = 0$).
- A strictly positive multiplier ($\lambda_i^* > 0$) can only appear if the constraint is active ($c_i(x^*) = 0$).

Intuitively, only "tight" constraints exert a "force" on the optimum.

9.4.3 Second-Order Conditions

Assumption. In this section III.2, we assume that f and c_i are **twice continuously differentiable**.

Definition 9.21 (Critical Cone). Given $\mathcal{A}(x^*)$ and Lagrange multipliers λ^* satisfying the KKT conditions, the **critical cone** $\mathcal{C}(x^*, \lambda^*)$ is defined by:

$$\mathcal{C}(x^*, \lambda^*) = \left\{ w \in \mathcal{F}(x^*) \mid \nabla c_i(x^*)^\top w = 0 \text{ for all } i \in \mathcal{I} \cap \mathcal{A}(x^*) \text{ with } \lambda_i^* > 0 \right\}. \quad (9.45)$$

Remark 9.22. The critical cone is a subset of the set of linearized feasible directions $\mathcal{F}(x^*)$. It contains the "critical" directions that are orthogonal to the gradients of the inequality constraints that are both active **and** have a strictly positive multiplier.

Intuitively, these are the directions along which the constraint "really counts" because it exerts a non-zero force at the optimum.

Remark 9.23 (Equivalence for the critical cone). Equivalently, one can characterize the critical cone by:

$$w \in \mathcal{C}(x^*, \lambda^*) \iff \begin{cases} \nabla c_i(x^*)^\top w = 0 & \text{for all } i \in \mathcal{E}, \\ \nabla c_i(x^*)^\top w \geq 0 & \text{for all } i \in \mathcal{I} \cap \mathcal{A}(x^*) \text{ with } \lambda_i^* = 0, \\ \nabla c_i(x^*)^\top w = 0 & \text{for all } i \in \mathcal{I} \cap \mathcal{A}(x^*) \text{ with } \lambda_i^* > 0. \end{cases} \quad (9.46)$$

In other words, the directions of the critical cone "adhere" to the equality constraints and to the active inequality constraints when the objective function is slightly perturbed.

Fundamental Property. We have the following important property:

$$w \in \mathcal{C}(x^*, \lambda^*) \implies \lambda_i^* \nabla c_i(x^*)^\top w = 0 \quad \forall i \in \mathcal{E} \cup \mathcal{I}. \quad (9.47)$$

Indeed:

- For $i \in \mathcal{E}$, we have $\nabla c_i(x^*)^\top w = 0$ by definition of $\mathcal{F}(x^*)$.
- For $i \in \mathcal{I} \cap \mathcal{A}(x^*)$ with $\lambda_i^* > 0$, we have $\nabla c_i(x^*)^\top w = 0$ by definition of $\mathcal{C}(x^*, \lambda^*)$.
- For $i \in \mathcal{I} \cap \mathcal{A}(x^*)$ with $\lambda_i^* = 0$, the product $\lambda_i^* \nabla c_i(x^*)^\top w = 0$.
- For $i \in \mathcal{I} \setminus \mathcal{A}(x^*)$ (inactive constraints), we have $\lambda_i^* = 0$ by complementarity.

Consequence. Since $\nabla_x \mathcal{L}(x^*, \lambda^*) = 0$ by the KKT condition of stationarity, we have:

$$w \in \mathcal{C}(x^*, \lambda^*) \implies w^\top \nabla f(x^*) = w^\top \sum_{i \in \mathcal{E} \cup \mathcal{I}} \lambda_i^* \nabla c_i(x^*) = \sum_{i \in \mathcal{E} \cup \mathcal{I}} \lambda_i^* \nabla c_i(x^*)^\top w = 0. \quad (9.48)$$

Thus, the critical cone gathers the directions along which, based on first-order information only, we cannot determine whether f will increase or decrease.

Theorem 9.24 (Second-Order Necessary Conditions). *Let x^* be a local minimum. Suppose that LICQ is satisfied at x^* . Let λ^* be the vector of Lagrange multipliers satisfying the KKT conditions. Then, for all $w \in \mathcal{C}(x^*, \lambda^*)$:*

$$w^\top \nabla_{xx}^2 \mathcal{L}(x^*, \lambda^*) w \geq 0. \quad (9.49)$$

*In other words, the Hessian of the Lagrangian is **positive semi-definite** on the critical cone.*

Theorem 9.25 (Second-Order Sufficient Conditions). *Let x^* be a feasible point satisfying the KKT conditions with multipliers λ^* . Suppose that for all $w \in \mathcal{C}(x^*, \lambda^*) \setminus \{0\}$:*

$$w^\top \nabla_{xx}^2 \mathcal{L}(x^*, \lambda^*) w > 0. \quad (9.50)$$

Then x^ is a **strict local minimum** of the optimization problem.*

Remark 9.26. The difference between the necessary and sufficient conditions lies in the inequality:

- **Necessary:** ≥ 0 (positive semi-definite)
- **Sufficient:** > 0 strictly for $w \neq 0$ (positive definite on $\mathcal{C}$)

This margin guarantees that x^* is a strict local minimum, and not just a saddle point or an inflection point.

Example 9.27 (Application of KKT Conditions). Consider the following minimization problem:

$$\min_{x \in \mathbb{R}^2} f(x) = x_1 + x_2 \quad \text{subject to} \quad c_1(x) = 2 - x_1^2 - x_2^2 \geq 0. \quad (9.51)$$

We have $\mathcal{I} = \{1\}$ and $\mathcal{E} = \emptyset$.

The Lagrangian. The Lagrangian function is:

$$\mathcal{L}(x, \lambda) = x_1 + x_2 - \lambda(2 - x_1^2 - x_2^2). \quad (9.52)$$

KKT Conditions. The KKT conditions are written as:

1. **Stationarity:**

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial x_1} = 1 + 2\lambda x_1 = 0, \\ \frac{\partial \mathcal{L}}{\partial x_2} = 1 + 2\lambda x_2 = 0. \end{cases} \quad (9.53)$$

2. **Primal Feasibility:**

$$2 - x_1^2 - x_2^2 \geq 0. \quad (9.54)$$

3. **Dual Feasibility:**

$$\lambda \geq 0. \quad (9.55)$$

4. **Complementarity:**

$$\lambda(2 - x_1^2 - x_2^2) = 0. \quad (9.56)$$

Resolution. From (1) and (2), we get $x_1 = x_2 = -\frac{1}{2\lambda}$ (if $\lambda \neq 0$).

Case 2: $\lambda \neq 0$. By the complementarity condition (4), we have $c_1(x) = 0$, so the constraint is active:

$$2 - x_1^2 - x_2^2 = 0 \iff x_1^2 + x_2^2 = 2. \quad (9.57)$$

From (1) and (2), we have $x_1 = x_2 = -\frac{1}{2\lambda}$. Substituting into the circle equation:

$$2x_1^2 = 2 \implies x_1^2 = 1 \implies x_1 = 1 \quad \text{or} \quad x_1 = -1. \quad (9.58)$$

Let's analyze the two possibilities:

- **If $x_1 = 1$:** Then $x_2 = 1$. Equation (1) gives $1 + 2\lambda(1) = 0 \implies \lambda = -1/2$. However, the dual feasibility condition (3) requires $\lambda \geq 0$. Therefore, this case is **impossible** (contradiction).

- **If $x_1 = -1$:** Then $x_2 = -1$. Equation (1) gives $1 + 2\lambda(-1) = 0 \implies \lambda = 1/2$. We indeed have $\lambda > 0$, which is valid.

Thus, only the pair $(x^*, \lambda^*) = ((-1, -1), 1/2)$ satisfies the KKT conditions.

Verification.

- $x^* = (-1, -1)$ is feasible: $2 - 1 - 1 = 0 \geq 0$. ✓
- $\lambda^* = 1/2 > 0$. ✓
- Complementarity: $\lambda^* \cdot c_1(x^*) = \frac{1}{2} \cdot 0 = 0$. ✓

The point $x^* = (-1, -1)$ with $\lambda^* = 1/2$ satisfies the KKT conditions.

Verification of Second-Order Conditions. The Hessian of the Lagrangian is:

$$\nabla_{xx}^2 \mathcal{L}(x, \lambda) = 2\lambda \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = 2\lambda I_2. \quad (9.59)$$

At x^* with $\lambda^* = 1/2$:

$$\nabla_{xx}^2 \mathcal{L}(x^*, \lambda^*) = I_2, \quad (9.60)$$

which is positive definite. Thus, for all $w \neq 0$ in the critical cone:

$$w^\top \nabla_{xx}^2 \mathcal{L}(x^*, \lambda^*) w = \|w\|^2 > 0. \quad (9.61)$$

By the sufficient conditions theorem, $x^* = (-1, -1)$ is a **strict local minimum**.

9.5 Duality

Duality theory allows one to construct a new problem starting from the functions and data of the original problem. This dual problem may prove to be easier to solve numerically.

In this section, we consider only inequality constraints and we assume that f and the functions $-c_i$ are convex.

9.5.1 Primal Problem

The primal problem is defined by:

$$(P) \quad \min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad c(x) \geq 0, \quad (9.62)$$

where $c(x) = (c_1(x), \dots, c_m(x))^\top$.

9.5.2 Dual Function and Dual Problem

Definition 9.28 (Dual Objective Function). We define the Lagrangian by $\mathcal{L}(x, \lambda) = f(x) - \lambda^\top c(x)$. The **dual objective function** $q : \mathbb{R}^m \rightarrow \mathbb{R}$ is defined by:

$$q(\lambda) = \inf_x \mathcal{L}(x, \lambda). \quad (9.63)$$

The domain of q , denoted $\mathcal{D}$, is the set of values of λ for which this infimum is finite (i.e. $> -\infty$):

$$\mathcal{D} = \{\lambda \in \mathbb{R}^m \mid q(\lambda) > -\infty\}. \quad (9.64)$$

If f is convex and the constraints c_i are concave (i.e. $-c_i$ convex), then the computation of $q(\lambda)$ is a convex optimization problem (often feasible).

Definition 9.29 (Dual Problem). The **dual problem** consists in maximizing the dual function over non-negative multipliers:

$$(D) \quad \max_{\lambda \in \mathbb{R}^m} q(\lambda) \quad \text{subject to} \quad \lambda \geq 0. \quad (9.65)$$

9.5.3 Example

Let the following primal problem be given:

$$\min_{x \in \mathbb{R}^2} \frac{1}{2}(x_1^2 + x_2^2) \quad \text{subject to} \quad x_1 - 1 \geq 0. \quad (9.66)$$

Here $f(x) = \frac{1}{2}(x_1^2 + x_2^2)$ and $c_1(x) = x_1 - 1$.

Computation of the dual function. The Lagrangian is:

$$\mathcal{L}(x, \lambda) = \frac{1}{2}(x_1^2 + x_2^2) - \lambda(x_1 - 1). \quad (9.67)$$

When λ is fixed, $\mathcal{L}(\cdot, \lambda)$ is a convex function with respect to x (sum of convex functions). The infimum is reached when the gradient with respect to x vanishes:

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial x_1} = x_1 - \lambda = 0 \implies x_1 = \lambda, \\ \frac{\partial \mathcal{L}}{\partial x_2} = x_2 = 0. \end{cases} \quad (9.68)$$

Substituting these values back into the expression of the Lagrangian, we obtain $q(\lambda)$:

$$q(\lambda) = \mathcal{L}((\lambda, 0), \lambda) \quad (9.69)$$

$$= \frac{1}{2}(\lambda^2 + 0^2) - \lambda(\lambda - 1) \quad (9.70)$$

$$= \frac{1}{2}\lambda^2 - \lambda^2 + \lambda \quad (9.71)$$

$$= -\frac{1}{2}\lambda^2 + \lambda. \quad (9.72)$$

Resolution of the dual problem. The dual problem is thus:

$$\max_{\lambda \geq 0} \left(-\frac{1}{2}\lambda^2 + \lambda \right). \quad (9.73)$$

The function $h(\lambda) = -\frac{1}{2}\lambda^2 + \lambda$ is a concave parabola (opening downwards). Its unconstrained maximum is reached at $h'(\lambda) = -\lambda + 1 = 0 \implies \lambda = 1$. Since $\lambda = 1$ satisfies the constraint $\lambda \geq 0$, the optimal dual solution is $\lambda^* = 1$.

This allows us to naturally recover the primal solution: $x_1^* = \lambda^* = 1$ and $x_2^* = 0$. The point $x^* = (1, 0)$ is indeed the projection of the origin onto the half-plane $x_1 \geq 1$.

9.5.4 Duality Properties and Theorems

Theorem 9.30 (Properties of the dual function). *The dual objective function q is **concave** and its domain $\mathcal{D}$ is a **convex** set. This is true regardless of the convexity of f or the constraints c_i .*

Theorem 9.31 (Weak Duality). *Let $\bar{x}$ be a feasible point for the primal problem and $\bar{\lambda} \geq 0$ a feasible point for the dual problem. Then:*

$$q(\bar{\lambda}) \leq f(\bar{x}). \quad (9.74)$$

*Thus, the value of the dual function for any $\lambda \geq 0$ provides a **lower bound** for the solution to the primal problem.*

Theorem 9.32 (Link between KKT and Duality). *Suppose that $\bar{x}$ is a solution to the primal problem, that f and the functions $-c_i$ are convex, and that they are continuously differentiable at $\bar{x}$.*

*Then, any vector $\bar{\lambda}$ for which the pair $(\bar{x}, \bar{\lambda})$ satisfies the KKT conditions is a **solution to the dual problem**.*

Remark 9.33. The KKT conditions for the primal problem are written as:

$$\begin{aligned} \nabla f(\bar{x}) - \sum_{i=1}^m \bar{\lambda}_i \nabla c_i(\bar{x}) &= 0, \\ c(\bar{x}) &\geq 0, \\ \bar{\lambda} &\geq 0, \\ \bar{\lambda}_i c_i(\bar{x}) &= 0 \quad \text{for } i = 1, \dots, m. \end{aligned}$$

Theorem 9.34 (Partial Converse). *Suppose that f and the functions $-c_i$ are convex and continuously differentiable on $\mathbb{R}^n$. Suppose that $\bar{x}$ is a solution to the primal problem satisfying LICQ. Suppose that $\hat{\lambda}$ is a solution to the dual problem and that the infimum $\inf_x \mathcal{L}(x, \hat{\lambda})$ is achieved at $\hat{x}$. Assume finally that $\mathcal{L}(\cdot, \hat{\lambda})$ is **strictly convex**.*

Then $\bar{x} = \hat{x}$ (i.e. $\hat{x}$ is the unique solution to the primal problem) and $f(\bar{x}) = \mathcal{L}(\hat{x}, \hat{\lambda})$.

Remark 9.35. In the previous example, for the dual solution $\hat{\lambda} = 1$, the infimum of $\mathcal{L}(x, 1)$ is reached at $x_1 = 1, x_2 = 0$, which is the solution to the primal problem.

9.5.5 Example: Duality in Linear Programming

Consider the following primal linear program:

$$(P) \quad \min_{x \in \mathbb{R}^n} c^\top x \quad \text{subject to} \quad Ax - b \geq 0. \quad (9.75)$$

Here, the objective function is linear (hence convex) and the constraints are affine. The dual objective function is:

$$q(\lambda) = \inf_{x \in \mathbb{R}^n} [c^\top x - \lambda^\top (Ax - b)] \quad (9.76)$$

$$= \inf_{x \in \mathbb{R}^n} [(c - A^\top \lambda)^\top x + \lambda^\top b]. \quad (9.77)$$

The infimum of this affine function with respect to x is $-\infty$, unless the coefficient of x is strictly zero.

$$\inf_x (c - A^\top \lambda)^\top x = \begin{cases} 0 & \text{if } c - A^\top \lambda = 0, \\ -\infty & \text{otherwise.} \end{cases} \quad (9.78)$$

So, to be in the domain $\mathcal{D}$ (where $q > -\infty$), we must impose $c = A^\top \lambda$. In this case, $q(\lambda) = \lambda^\top b$.

The dual problem is then written as:

$$(D) \quad \max_{\lambda} \lambda^{\top} b \quad \text{subject to} \quad c = A^{\top} \lambda, \quad \lambda \geq 0. \quad (9.79)$$

This illustrates how the dual of a linear minimization problem is a linear maximization problem.

Checkpoint : Did you understand the essentials ?

- **Gradients:** What is the geometric condition on ∇f and ∇c_i at the optimum? (Parallelism).
- **Lagrangian:** Define the Lagrangian function.
- **Sign of multipliers:** Why is $\lambda \geq 0$ for an inequality? (Binding constraint).
- **Complementarity:** What does $\lambda_i c_i(x) = 0$ mean?

Chapter 10

Linear Programming: The Simplex Method

10.1 Optimality and Duality

10.1.1 Optimality of the Primal Problem

Consider the linear programming (LP) problem in standard form:

$$\min_{x \in \mathbb{R}^n} c^\top x \quad \text{subject to} \quad Ax = b, \quad x \geq 0, \quad (10.1)$$

where A is an $m \times n$ matrix of rank m , $c \in \mathbb{R}^n$ is the cost vector and $b \in \mathbb{R}^m$.

A key step is to identify a **basic feasible solution**. We partition the matrix A into $A = [B \mid N]$ where B is an invertible $m \times m$ matrix (the basis) and N contains the remaining $n - m$ columns. Similarly, we partition $x = (x_B, x_N)$ and $c = (c_B, c_N)$. The equation $Ax = b$ becomes $Bx_B + Nx_N = b$, or $x_B = B^{-1}b - B^{-1}Nx_N$.

By setting $x_N = 0$, we obtain the basic solution $x_B = B^{-1}b$. If $x_B \geq 0$, the solution is said to be feasible.

The reduced cost is defined by the vector s_N :

$$s_N = c_N - (B^{-1}N)^\top c_B = c_N - N^\top \lambda, \quad (10.2)$$

with $\lambda = B^{-T}c_B$ the vector of multipliers (or dual variables).

Theorem 10.1 (Optimality Condition). *A basic feasible solution $(x_B, 0)$ is optimal if and only if the reduced cost is non-negative, that is $s_N \geq 0$ (for all its components).*

10.1.2 The Dual Problem

Every primal problem (P) corresponds to a dual problem (D). For the standard form above, the dual is:

$$\max_{\lambda \in \mathbb{R}^m} b^\top \lambda \quad \text{subject to} \quad A^\top \lambda \leq c. \quad (10.3)$$

The fundamental relationships between primal and dual are:

- **Weak Duality:** For any feasible x for (P) and any feasible λ for (D), we have $b^\top \lambda \leq c^\top x$.
- **Strong Duality:** If (P) has an optimal solution x^* , then (D) has an optimal solution λ^* and their values are equal: $c^\top x^* = b^\top \lambda^*$.

10.2 Geometry of the Feasible Set

The set of feasible solutions $\mathcal{F} = \{x \in \mathbb{R}^n \mid Ax = b, x \geq 0\}$ is a convex polyhedron (intersection of an affine subspace and the positive orthant).

Proposition 10.2. *The vertices (or extreme points) of this polyhedron correspond exactly to the basic feasible solutions of the linear problem.*

Geometrically, the simplex method consists in moving from a vertex to an adjacent vertex of the polyhedron, along an edge that improves (decreases) the objective function. Since the number of vertices is finite (at most $\binom{n}{m}$), the algorithm terminates in a finite number of steps.

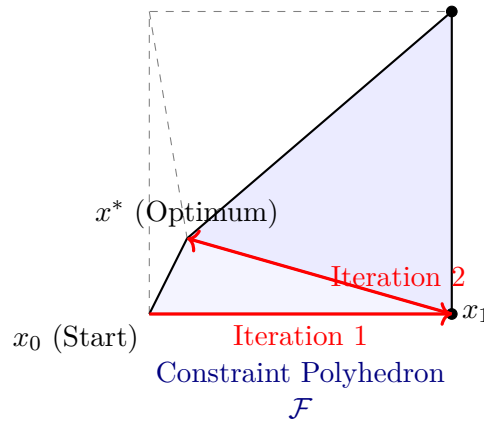


Figure 10.1: Geometric visualization of the Simplex Method. The algorithm moves from vertex to vertex along the edges of the polyhedron until it finds the optimum.

Walking on the edges: The set of feasible points forms a polyhedron. The optimum is always located at a vertex (or a face). The algorithm starts from a vertex x_0 , looks at the adjacent edges, and chooses the one that decreases the cost function the fastest (the most negative reduced cost). It "slides" along this edge to the next vertex (x_1), and repeats the operation.

10.3 Overview of the Simplex Method

Algorithm 17: One step of the Simplex Method

- 1 Given: $\mathcal{B}, \mathcal{N}, x_B = B^{-1}b \geq 0, x_N = 0$;
 - 2 Solve $B^\top \lambda = c_B$ for λ ;
 - 3 Compute $s_N = c_N - N^\top \lambda$;
 ▷ Pricing
 - 4 **if** $s_N \geq 0$ **then**
 - 5 | **stop** ▷ optimal point found
 - 6 Select $q \in \mathcal{N}$ with $s_q < 0$ as entering index;
 - 7 Solve $Bd = A_q$ for d ;
 - 8 **if** $d \leq 0$ **then**
 - 9 | **stop** ▷ problem is unbounded
 - 10 Compute $x_q^+ = \min_{i|d_i > 0} \left(\frac{(x_B)_i}{d_i} \right)$, use p to denote the minimizing index i ;
 - 11 Update $x_B^+ = x_B - dx_q^+, x_N^+ = (0, \dots, 0, x_q^+, 0, \dots, 0)^\top$;
 - 12 Change $\mathcal{B}$ by adding q and removing the basic variable corresponding to column p of B ;
-

10.4 Implementation and Practical Aspects

The simplex algorithm presented above is conceptual. Its efficient and robust implementation requires addressing several critical points (see Nocedal & Wright, Chapter 13).

10.4.1 Linear Algebra and Revised Simplex

In practice, explicitly computing the inverse of the basis B^{-1} is computationally expensive and unstable. The **revised simplex** method instead maintains a factorization of the basis (often LU) which is updated at each iteration.

- **Multipliers computation (Pricing):** We solve the linear system $B^T \lambda = c_B$ to obtain λ , then we compute $s_N = c_N - N^T \lambda$.
- **Displacement direction:** We solve the system $Bd = A_q$ to obtain the direction d .

These resolutions are fast if an up-to-date factorization of B is available.

10.4.2 Degeneracy and Cycling

A basis is said to be **degenerate** if one or more basic variables are zero ($x_i = 0$ for $i \in \mathcal{B}$). In this case, the maximum possible step may be zero ($\alpha = 0$). The algorithm performs a basis change without changing the geometric point or improving the objective function. If a sequence of such steps leads back to an already visited basis, the algorithm loops indefinitely: this is the phenomenon of **cycling**.

- **Bland's Rule:** To avoid cycling, a simple rule is to always choose, among the candidates for entering (negative reduced cost) or leaving (minimum ratio), the one with the **smallest index**. This rule guarantees the termination of the algorithm.

10.4.3 Initialization (Phase I and Phase II)

The simplex algorithm requires an initial basic feasible solution. This is not always trivial to find. We often proceed in two phases:

- **Phase I:** We introduce artificial variables z and solve an auxiliary problem whose objective is to minimize the sum of these variables (for example, $\min \sum z_i$ subject to $Ax + Ez = b, x, z \geq 0$). If the minimum is 0, a feasible solution for the initial problem has been found.
- **Phase II:** We use the solution found in Phase I as a starting point to solve the original problem.

Checkpoint : Did you understand the essentials ?

- **Geometry:** What does a basic feasible solution correspond to? (A vertex of the polyhedron).
- **Optimality:** Which vector must be positive to guarantee optimality? (Reduced costs).
- **Duality:** State the strong duality theorem.

Chapter 11

Quadratic Programming

11.1 Introduction

A general quadratic program (QP) is written in the form:

$$\min_{x \in \mathbb{R}^n} q(x) = \frac{1}{2}x^\top Gx + c^\top x, \quad (11.1)$$

subject to the constraints:

$$a_i^\top x = b_i, \quad i \in \mathcal{E}, \quad (11.2)$$

$$a_i^\top x \geq b_i, \quad i \in \mathcal{I}, \quad (11.3)$$

where G is an $n \times n$ symmetric matrix, $c \in \mathbb{R}^n$, $a_i \in \mathbb{R}^n$ and $b_i \in \mathbb{R}$.

Remark 11.1. For QPs, a finite amount of computation is required to solve them or to declare them infeasible.

- We speak of a **convex QP** when G is positive semi-definite.
- We speak of a **strictly convex QP** when G is positive definite.

11.2 QP with Equality Constraints

Consider problems of the form:

$$\min_{x \in \mathbb{R}^n} \frac{1}{2}x^\top Gx + c^\top x, \quad \text{subject to} \quad Ax = b, \quad (11.4)$$

where $A \in \mathbb{R}^{m \times n}$ (with $m < n$) is the Jacobian of the constraints. Its rows are the vectors a_i . The vector $b \in \mathbb{R}^m$ gathers the scalars b_i . We assume that the matrix A has **full rank**.

11.2.1 Properties

The first-order optimality conditions (KKT) are written as:

$$\begin{cases} Gx^* + c - A^\top \lambda^* = 0 \\ Ax^* = b \end{cases} \quad (11.5)$$

This is equivalent to the following matrix system:

$$\begin{pmatrix} G & -A^\top \\ A & 0 \end{pmatrix} \begin{pmatrix} x^* \\ \lambda^* \end{pmatrix} = \begin{pmatrix} -c \\ b \end{pmatrix} \quad (11.6)$$

Remark 11.2. If we rewrite x^* as $x + p$, where x is a candidate for the solution and p is a step, the system becomes:

$$\begin{pmatrix} G & A^\top \\ A & 0 \end{pmatrix} \begin{pmatrix} -p \\ \lambda^* \end{pmatrix} = \begin{pmatrix} g \\ h \end{pmatrix} \quad (11.7)$$

where $g = c + Gx$ is the gradient of the objective function at point x , $h = Ax - b$ is the residual of the constraints, and $p = x^* - x$.

The matrix $\begin{pmatrix} G & A^\top \\ A & 0 \end{pmatrix}$ is called the **KKT matrix**.

Let Z be an $n \times (n - m)$ matrix whose columns form a basis for the null-space of A . That is, $AZ = 0$ and A has full rank.

Lemma 11.3. *Assume that A has full row rank. Assume that the reduced Hessian matrix $Z^\top GZ$ is positive definite. Then the KKT matrix $\begin{pmatrix} G & A^\top \\ A & 0 \end{pmatrix}$ is non-singular.*

Proof. Let the vectors w and v be such that:

$$\begin{pmatrix} G & A^\top \\ A & 0 \end{pmatrix} \begin{pmatrix} w \\ v \end{pmatrix} = 0 \quad (11.8)$$

The second block of rows gives $Aw = 0$. Thus, w belongs to the null space of A , so there exists a vector u such that $w = Zu$.

Multiply the first block of rows by $w^\top$ from the left:

$$\begin{aligned} 0 &= \begin{pmatrix} w \\ v \end{pmatrix}^\top \begin{pmatrix} G & A^\top \\ A & 0 \end{pmatrix} \begin{pmatrix} w \\ v \end{pmatrix} \\ &= w^\top Gw + w^\top A^\top v + v^\top Aw \end{aligned}$$

Since $Aw = 0$, we have $v^\top Aw = 0$. Similarly, $w^\top A^\top v = (Aw)^\top v = 0$. Thus we obtain $0 = w^\top Gw$. Replacing w by Zu , we get:

$$0 = (Zu)^\top G(Zu) = u^\top (Z^\top GZ)u \quad (11.9)$$

Since $Z^\top GZ$ is positive definite, this implies that $u = 0$, and consequently $w = Zu = 0$.

The first block of rows then becomes $G \cdot 0 + A^\top v = 0$, i.e., $A^\top v = 0$. Since A has full row rank, $A^\top$ has full column rank, which implies that $v = 0$.

Therefore, the only solution vector is the zero vector $\begin{pmatrix} w \\ v \end{pmatrix} = 0$, which proves that the matrix is non-singular. $\square$

We will prove that under the same conditions, x^* is a strict local minimizer, and even a global solution.

Theorem 11.4. *Assume that A has full row rank. Assume that the reduced Hessian matrix $Z^\top GZ$ is positive definite. Then the vector x^* satisfying*

$$\begin{pmatrix} G & -A^\top \\ A & 0 \end{pmatrix} \begin{pmatrix} x^* \\ \lambda^* \end{pmatrix} = \begin{pmatrix} -c \\ b \end{pmatrix} \quad (11.10)$$

is the unique global solution.

Proof. Let x be another feasible point: $Ax = b$. Let $p = x^* - x$. Since $Ax = b$ and $Ax^* = b$, we have

$$Ap = A(x^* - x) = 0. \quad (11.11)$$

Substitute into $q(x)$:

$$\begin{aligned} q(x) &= q(x^* - p) = \frac{1}{2}(x^* - p)^\top G(x^* - p) + c^\top(x^* - p) \\ &= \frac{1}{2}p^\top Gp - p^\top Gx^* + \frac{1}{2}(x^*)^\top Gx^* - c^\top p + c^\top x^* \\ &= \frac{1}{2}p^\top Gp - p^\top Gx^* - c^\top p + q(x^*) \quad (*) \end{aligned}$$

Now use the first block of rows of the KKT system:

$$Gx^* + c - A^\top \lambda^* = 0 \implies Gx^* + c = A^\top \lambda^* \quad (11.12)$$

Multiply from the left by $p^\top$:

$$p^\top(Gx^* + c) = p^\top A^\top \lambda^* = (Ap)^\top \lambda^* \quad (11.13)$$

But we have seen that $Ap = 0$, so the right-hand side is zero. Thus:

$$p^\top Gx^* + p^\top c = 0 \implies -p^\top Gx^* - c^\top p = 0 \quad (11.14)$$

Substituting this into equation (*), we obtain:

$$q(x) = q(x^*) + \frac{1}{2}p^\top Gp \quad (11.15)$$

Since p is in the null space of A , there exists a vector $u \in \mathbb{R}^{n-m}$ such that $p = Zu$. Then:

$$p^\top Gp = (Zu)^\top G(Zu) = u^\top (Z^\top GZ)u \quad (11.16)$$

Since the reduced Hessian matrix $Z^\top GZ$ is assumed positive definite, we have $u^\top Z^\top GZ u > 0$ for all $u \neq 0$. But $u \neq 0 \iff p \neq 0 \iff x \neq x^*$.

Consequently, for every feasible $x \neq x^*$,

$$q(x) = q(x^*) + \frac{1}{2}u^\top Z^\top GZ u > q(x^*) \quad (11.17)$$

Which proves that x^* is the unique global minimizer. $\square$

11.2.2 Direct Solution of the KKT System

Remark 11.5. It can be proven (Forsgren et al.) that the KKT matrix is always indefinite when $m \geq 1$. In other words, there exist vectors x and y such that $x^\top Kx > 0$ and $y^\top Ky < 0$ (where K denotes the KKT matrix).

Factorization of the Full KKT System

We cannot use Cholesky factorization due to the indefinite nature of the matrix. An LU decomposition is feasible, but it ignores the symmetry of the system.

Thus, we use the **symmetric indefinite factorization** for K (symmetric matrix):

$$PKP^\top = LBL^\top, \quad (11.18)$$

where:

- P is a permutation matrix;
- L is a unit lower triangular matrix;
- B is a block-diagonal matrix, with blocks of size 1×1 or 2×2 .

To solve the KKT system $K \begin{pmatrix} -p \\ \lambda^* \end{pmatrix} = \begin{pmatrix} g \\ h \end{pmatrix}$:

1. Solve $Lz = P^\top \begin{pmatrix} g \\ h \end{pmatrix}$ for z (forward substitution).
2. Solve $B\hat{z} = z$ for $\hat{z}$ (block-diagonal solve).
3. Solve $L^\top \bar{z} = \hat{z}$ for $\bar{z}$ (backward substitution).
4. Set $\begin{pmatrix} -p \\ \lambda^* \end{pmatrix} = P\bar{z}$.

Schur Complement Method

This method is applicable if G is **positive definite** (and thus invertible). Starting from the KKT system:

$$\begin{cases} -Gp + A^\top \lambda^* = g \\ Ap = h \end{cases} \quad (11.19)$$

Multiply the first equation by AG^{-1} :

$$\begin{aligned} -AG^{-1}Gp + AG^{-1}A^\top \lambda^* &= AG^{-1}g \\ -Ap + (AG^{-1}A^\top) \lambda^* &= AG^{-1}g \end{aligned}$$

Using the second equation ($Ap = h$), we obtain:

$$(AG^{-1}A^\top) \lambda^* = AG^{-1}g - h \quad (11.20)$$

The system $(AG^{-1}A^\top) \lambda^* = AG^{-1}g - h$ is positive definite. We can solve it to find λ^* . Next, we use the first block of rows of the KKT system to find p :

$$Gp = A^\top \lambda^* - g \implies p = G^{-1}(A^\top \lambda^* - g). \quad (11.21)$$

Numerical considerations: This method requires operations with G^{-1} and the factorization of the matrix $AG^{-1}A^\top$. It is therefore suitable when:

- G is well-conditioned and easy to invert (example: diagonal).
- G^{-1} is explicitly known.
- m (the number of constraints) is small.

Null-Space Method

The non-singularity of G is not required here; we only need the assumptions of the lemma (A full rank, $Z^\top GZ$ positive definite). Suppose we partition the vector p as follows:

$$p = Yp_Y + Zp_Z \quad (11.22)$$

where Z is the $n \times (n - m)$ matrix of the null space of A , and Y is an $n \times m$ matrix such that the matrix $[Y \mid Z]$ is non-singular. For simplification, we often choose Y such that $AY = I$ (if possible) or $A[Y \mid Z]$ is triangular.

Substitute p into the second equation of the KKT system ($Ap = h$):

$$A(Yp_Y + Zp_Z) = h \implies AYp_Y + AZp_Z = h \quad (11.23)$$

Since Z is in the null space, $AZ = 0$. Thus:

$$(AY)p_Y = h \quad (11.24)$$

Since A has rank m and $[Y \mid Z]$ is non-singular, the matrix AY is non-singular of size $m \times m$. We can therefore solve this system to find p_Y . (If we choose Y such that $AY = I$, then $p_Y = h$).

Next, substitute p into the first equation of the KKT system ($-Gp + A^\top \lambda^* = g$):

$$-G(Yp_Y + Zp_Z) + A^\top \lambda^* = g \quad (11.25)$$

Multiply from the left by $Z^\top$:

$$\begin{aligned} -Z^\top GYp_Y - Z^\top GZp_Z + Z^\top A^\top \lambda^* &= Z^\top g \\ -Z^\top GYp_Y - Z^\top GZp_Z + (AZ)^\top \lambda^* &= Z^\top g \end{aligned}$$

Since $AZ = 0$, the term involving λ^* disappears:

$$-(Z^\top GZ)p_Z = Z^\top g + Z^\top GYp_Y \quad (11.26)$$

The system $(Z^\top GZ)p_Z = -(Z^\top g + Z^\top GYp_Y)$ allows us to find p_Z , because the reduced matrix $Z^\top GZ$ is assumed positive definite (hence invertible).

Finally, to find λ^* , we multiply the first equation by $Y^\top$:

$$(AY)^\top \lambda^* = Y^\top (g + Gp) \quad (11.27)$$

This linear system allows us to determine λ^* .

Remarks:

- This method is efficient when $n - m$ is small.
- Computing Z is the main difficulty; it is not unique. Poor choices can result in poor conditioning of $Z^\top GZ$.

Example: Let $G = \begin{pmatrix} 6 & 2 & 1 \\ 2 & 5 & 2 \\ 1 & 2 & 4 \end{pmatrix}$, $c = \begin{pmatrix} -8 \\ -3 \\ -3 \end{pmatrix}$. Constraints: $\begin{cases} x_1 + x_3 = 3 \\ x_2 + x_3 = 0 \end{cases}$. Hence $A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}$ and $b = \begin{pmatrix} 3 \\ 0 \end{pmatrix}$. We can choose $Z = \begin{pmatrix} -1 \\ -1 \\ 1 \end{pmatrix}$ (verify $AZ = 0$) and $Y = \begin{pmatrix} 2/3 & -1/3 \\ -1/3 & 2/3 \\ 1/3 & 1/3 \end{pmatrix}$ such that $AY = I_2$.

Example (continued): Let $x = \begin{pmatrix} -1 \\ -2 \\ 2 \end{pmatrix}$. We compute $h = Ax - b = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$ and $g = c + Gx = \begin{pmatrix} -7/2 \\ -2 \\ 2 \end{pmatrix}$ (note: values from the course example).

1. **Find** p_Y : Solve $(AY)p_Y = -h$. Here $p_Y = -h = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$.
2. **Find** p_Z : Solve $(Z^\top GZ)p_Z = -(Z^\top g + Z^\top GYp_Y)$. Let's compute the terms: $Z^\top GZ = \begin{pmatrix} -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} 6 & 2 & 1 \\ 2 & 5 & 2 \\ 1 & 2 & 4 \end{pmatrix} \begin{pmatrix} -1 \\ -1 \\ 1 \end{pmatrix} = 13$ (scalar). $Z^\top g = -13$. The term $Z^\top GYp_Y$ is zero since $p_Y = 0$. The equation becomes: $13p_Z = -(-13) \implies p_Z = 1$.
3. **Reconstruct** p : $p = Yp_Y + Zp_Z = Y \cdot 0 + Z \cdot 1 = \begin{pmatrix} -1 \\ -1 \\ 1 \end{pmatrix}$.
4. **Solution**: $x^* = x + p = \begin{pmatrix} -1 \\ -2 \\ 2 \end{pmatrix} + \begin{pmatrix} -1 \\ -1 \\ 1 \end{pmatrix} = \begin{pmatrix} -2 \\ -3 \\ 3 \end{pmatrix}$.

11.2.3 Iterative Solution of the KKT System

For large systems, direct methods become too expensive. Iterative methods, such as Conjugate Gradient (CG), are then preferred. However, using CG directly on the full KKT system is unstable because the KKT matrix is not positive definite (it is indefinite).

The idea is to apply CG on the **reduced system**, which is positive definite (under the assumption that $Z^\top GZ$ is).

Substitute $x^* = Yp_Y + Zp_Z$ into the problem. p_Z is the solution to the unconstrained minimization problem:

$$\min_{p_Z \in \mathbb{R}^{n-m}} \frac{1}{2} p_Z^\top (Z^\top GZ) p_Z + p_Z^\top c_Z \quad (11.28)$$

where $c_Z = Z^\top GYp_Y + Z^\top c$ (noting that the linear term also depends on p_Y , which is determined by the constraints).

Thus, p_Z satisfies the normal equation:

$$(Z^\top GZ)p_Z = -c_Z \quad (11.29)$$

Since $Z^\top GZ$ is positive definite, we can use the Conjugate Gradient (CG) to find p_Z .

Preconditioning: In practice, a preconditioner W_{ZZ} is used to accelerate convergence. A good preconditioner should cluster the eigenvalues of $W_{ZZ}^{-1/2} (Z^\top GZ) W_{ZZ}^{-1/2}$. Ideally, one would choose $W_{ZZ} = Z^\top GZ$, but this is too expensive. Good choices are of the form $W_{ZZ} = Z^\top HZ$, where H is a symmetric approximation of G such that $Z^\top HZ$ is positive definite.

The following algorithm (Algorithm 17 in some references) describes this procedure:

Algorithm 18: Preconditioned Conjugate Gradient for reduced systems

```

1 Choose an initial point  $x_Z$  (corresponding to our  $p_Z$ );
2 Compute  $r_Z = Z^\top GZx_Z + c_Z$ ,  $g_Z = W_{ZZ}^{-1}r_Z$ , and  $d_Z = -g_Z$ ;
3 repeat
4    $\alpha \leftarrow \frac{r_Z^\top g_Z}{d_Z^\top Z^\top GZd_Z}$ ;
5    $x_Z \leftarrow x_Z + \alpha d_Z$ ;
6    $r_Z^+ \leftarrow r_Z + \alpha Z^\top GZd_Z$ ;
7    $g_Z^+ \leftarrow W_{ZZ}^{-1}r_Z^+$ ;
8    $\beta \leftarrow \frac{(r_Z^+)^\top g_Z^+}{r_Z^\top g_Z}$ ;
9    $d_Z \leftarrow -g_Z^+ + \beta d_Z$ ;
10   $g_Z \leftarrow g_Z^+$ ;
11   $r_Z \leftarrow r_Z^+$ ;
12 until a termination test is satisfied;

```

11.3 Problems with Inequality Constraints

11.3.1 Properties

Consider the following optimization problem:

$$\min_{x \in \mathbb{R}^n} \frac{1}{2}x^\top Gx + c^\top x \quad \text{subject to} \quad \begin{cases} a_i^\top x - b_i \geq 0, & \forall i \in \mathcal{I} \\ a_i^\top x - b_i = 0, & \forall i \in \mathcal{E} \end{cases} \quad (11.30)$$

The Lagrangian is defined by:

$$\mathcal{L}(x, \lambda) = \frac{1}{2}x^\top Gx + c^\top x - \sum_{i \in \mathcal{E} \cup \mathcal{I}} \lambda_i (a_i^\top x - b_i) \quad (11.31)$$

Recall the active set:

$$\mathcal{A}(x) = \{i \in \mathcal{E} \cup \mathcal{I} \mid a_i^\top x - b_i = 0\} \quad (11.32)$$

KKT Conditions: Any solution x^* satisfies, for some multipliers λ_i^* :

$$\begin{cases} Gx^* + c - \sum_{i \in \mathcal{E} \cup \mathcal{I}} \lambda_i^* a_i = 0 \\ a_i^\top x^* - b_i \geq 0, & \forall i \in \mathcal{I} \setminus \mathcal{A}(x^*) \\ a_i^\top x^* - b_i = 0, & \forall i \in \mathcal{E} \\ \lambda_i^* \geq 0, & \forall i \in \mathcal{I} \cap \mathcal{A}(x^*) \end{cases} \quad (11.33)$$

Note: In the system, feasibility conditions are implicitly noted by the index sets, and the complementarity condition is summarized by $\lambda_i^* \geq 0$ for active inequality constraints (and $\lambda_i^* = 0$ for inactive ones).

Remark 11.6. We do not need to assume linear independence constraint qualification (LICQ) because the constraints are linear. We merely assume that the matrix A of equality constraints has full row rank.

Theorem 11.7. *If x^* satisfies the KKT conditions for some λ^* , and if G is positive semi-definite, then x^* is a global solution to the problem.*

Proof. Let x be another feasible point. We have:

$$\begin{aligned} a_i^\top x &= b_i, \quad \forall i \in \mathcal{E} \\ a_i^\top x &\geq b_i, \quad \forall i \in \mathcal{I} \end{aligned} \quad (11.34)$$

Thus, considering the difference with x^* :

$$\begin{aligned} a_i^\top (x - x^*) &= 0, \quad \forall i \in \mathcal{E} \\ a_i^\top (x - x^*) &\geq 0, \quad \forall i \in \mathcal{I} \cap \mathcal{A}(x^*) \end{aligned} \quad (11.35)$$

The second inequality comes from the fact that for $i \in \mathcal{A}(x^*)$, $a_i^\top x^* = b_i$, so $a_i^\top x \geq b_i \implies a_i^\top (x - x^*) \geq 0$.

From the first KKT condition:

$$(Gx^* + c) = \sum_{i \in \mathcal{E} \cup \mathcal{I}} \lambda_i^* a_i \quad (11.36)$$

Multiply by $(x - x^*)^\top$:

$$(x - x^*)^\top (Gx^* + c) = \sum_{i \in \mathcal{E} \cup \mathcal{I}} \lambda_i^* a_i^\top (x - x^*) \quad (11.37)$$

Let's decompose the sum:

$$\sum_{i \in \mathcal{E}} \lambda_i^* \underbrace{a_i^\top (x - x^*)}_{=0} + \sum_{i \in \mathcal{I} \cap \mathcal{A}(x^*)} \underbrace{\lambda_i^*}_{\geq 0} \underbrace{a_i^\top (x - x^*)}_{\geq 0} + \sum_{i \in \mathcal{I} \setminus \mathcal{A}(x^*)} \underbrace{\lambda_i^*}_{=0} a_i^\top (x - x^*) \quad (11.38)$$

All terms are zero or positive. Therefore:

$$(x - x^*)^\top (Gx^* + c) \geq 0 \quad (11.39)$$

Now consider the Taylor series expansion (exact for a quadratic) of $q(x)$ around x^* :

$$q(x) = q(x^*) + (x - x^*)^\top \nabla q(x^*) + \frac{1}{2} (x - x^*)^\top \nabla^2 q(x^*) (x - x^*) \quad (11.40)$$

Now $\nabla q(x^*) = Gx^* + c$ and $\nabla^2 q(x^*) = G$.

$$q(x) = q(x^*) + \underbrace{(x - x^*)^\top (Gx^* + c)}_{\geq 0} + \frac{1}{2} \underbrace{(x - x^*)^\top G (x - x^*)}_{\geq 0 \text{ since } G \succeq 0} \quad (11.41)$$

Thus, $q(x) \geq q(x^*)$, which proves that x^* is a global solution. $\square$

Remark 11.8. If G is positive definite, the proof can be adapted to show that x^* is unique. (Indeed, if $x \neq x^*$, the quadratic term is strictly positive).

Remark 11.9. There are methods for general quadratic problems, for example:

- "Active-set" methods (Gould);
- Interior point methods (Wright or Vanderbei).

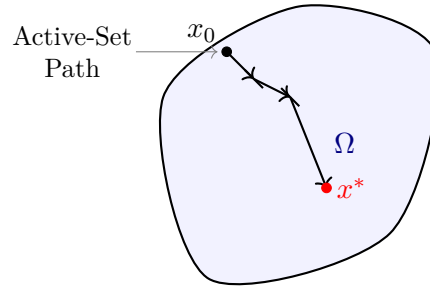


Figure 11.1: Illustration of an Active-Set type method for a general QP problem. The algorithm moves on the surface of the feasible set Ω , activating or deactivating constraints until finding the optimum.

General Active-Set Methods: Unlike the simple case of bounds, the region Ω can have a complex polyhedral shape defined by general linear inequalities. The algorithm maintains a set of "active" (tight) constraints and moves on the corresponding boundary. If the gradient indicates that the objective function can be reduced by "taking off" from a constraint, it is relaxed (removed from the active set).

We restrict our attention to problems with **bound constraints**:

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} x^\top G x + c^\top x \quad \text{subject to} \quad l \leq x \leq u \quad (11.42)$$

where G is symmetric, and $l, u \in \mathbb{R}^n$. The inequalities are understood component-wise ($l_i \leq x_i \leq u_i$). Some components of l and u can be $-\infty$ or $+\infty$.

The feasible region looks like a "box".

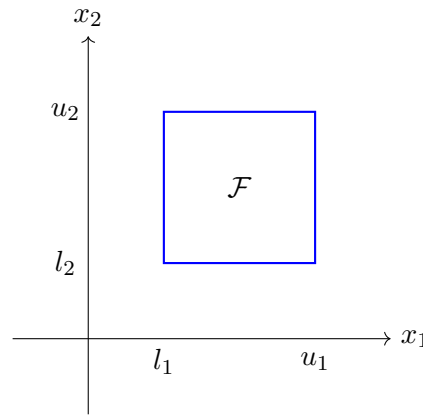


Figure 11.2: Feasible Region (Box)

Geometry of bound constraints: The feasible set is a hyper-rectangle (box). For each dimension i , the variable x_i is constrained to stay in the interval $[l_i, u_i]$. The "projection" onto this set is very simple: if a coordinate exceeds a bound, it is simply brought back to the value of the bound (clipping).

The iterations consist of two steps:

Step 1: Computation of the Cauchy Point

Projection onto the feasible region: The projection of a point x onto the box $[l, u]$ is defined component by component:

$$P(x, l, u)_i = \begin{cases} l_i & \text{if } x_i < l_i \\ x_i & \text{if } l_i \leq x_i \leq u_i \\ u_i & \text{if } x_i > u_i \end{cases} \quad (11.43)$$

Piecewise linear path: The path $x(t)$, starting from the current iterate x , is defined by:

$$x(t) = P(x - tg, l, u) \quad (11.44)$$

where $g = Gx + c$ is the gradient.

This path follows the direction of steepest descent $-g$ and "bends" when it reaches a bound, thus remaining feasible. The first local minimum along this path is the **Cauchy Point** x^c .

Computation of the Cauchy Point: x^c is the first local minimizer of $q(x(t))$. To find it, let's examine the **breakpoints**.

1) Identify the t at which the components of $x(t)$ reach a bound along $-g$:

The value $\bar{t}_i$ for each component is:

$$\bar{t}_i = \begin{cases} \frac{x_i - u_i}{g_i} & \text{if } g_i < 0 \text{ and } u_i < +\infty \\ \frac{x_i - l_i}{g_i} & \text{if } g_i > 0 \text{ and } l_i > -\infty \\ +\infty & \text{otherwise} \end{cases} \quad (11.45)$$

The components of $x(t)$ then evolve as follows:

$$x_i(t) = \begin{cases} x_i - tg_i & \text{if } t < \bar{t}_i \\ x_i - \bar{t}_i g_i & \text{if } t \geq \bar{t}_i \end{cases} \quad (11.46)$$

2) Eliminate zeros and duplicates, then sort the list of $\bar{T}$:

We obtain: $t_1 < t_2 < \dots < t_\ell$.

3) Successively examine the segments $[0, t_1], [t_1, t_2], \dots$:

Suppose we have not found a local minimum up to t_{j-1} . On the segment $[t_{j-1}, t_j]$:

$$x(t) = x(t_{j-1}) + (\Delta t) p^{j-1} \quad (11.47)$$

where $\Delta t = t - t_{j-1}$ and:

$$p_i^{j-1} = \begin{cases} -g_i & \text{if } t_{j-1} < \bar{t}_i \\ 0 & \text{otherwise} \end{cases} \quad (11.48)$$

We can write:

$$q(x(t)) = q(x(t_{j-1})) + (\Delta t) \delta_{j-1} + \frac{1}{2} (\Delta t)^2 (p^{j-1})^\top G p^{j-1} \quad (11.49)$$

where $\delta_{j-1} = (p^{j-1})^\top (Gx(t_{j-1}) + c)$ is the directional derivative.

Three possible cases:

- **Case 1:** $\delta_{j-1} \geq 0 \Rightarrow$ there is a local minimizer of $q(x(t))$ at $x^c = x(t_{j-1})$.
- **Case 2:** $(\Delta t)^* \in [0, t_j - t_{j-1}] \Rightarrow$ there is a local minimizer at $x^c = x(t_{j-1} + (\Delta t)^*)$.
- **Case 3:** $(\Delta t)^* > t_j - t_{j-1} \Rightarrow$ explore the next segment.

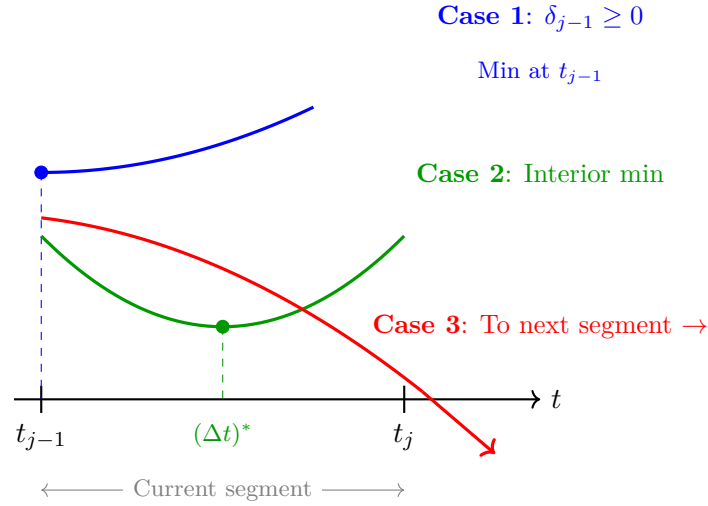


Figure 11.3: Search for the Cauchy Point: the three possible cases during the minimization of $q(x(t))$ over a segment $[t_{j-1}, t_j]$.

On each linear segment of the projected trajectory, the objective function $q(x(t))$ is a parabola (quadratic function of t).

- **Case 1 (Blue):** The slope is upward from the start. The minimum on the segment is the starting point t_{j-1} . This is the Cauchy Point.
- **Case 2 (Green):** The parabola reaches its minimum inside the segment (the derivative vanishes at $(\Delta t)^* < t_j - t_{j-1}$). This is the Cauchy Point.
- **Case 3 (Red):** The function decreases throughout the segment. The minimum is not reached here, we must continue the search on the next segment (beyond t_j).

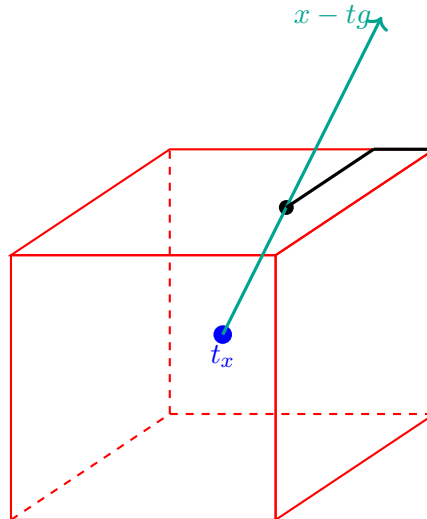


Figure 11.4: Trajectory projected onto the feasible box (3D). The trajectory $x - tg$ crosses the cube and exits through the top face at X , then follows the edges (projection).

The Cauchy Path: This 3D figure illustrates the path $x(t) = P(x - tg)$.

- Initially (center), we follow the steepest descent direction $-g$ (straight line).
- As soon as we hit a face of the box (here the top face), we can no longer "go up". Projection forces us to stay on the face.
- Then we continue to slide on the face, or along an edge, until we find a local minimum along this broken path. This minimum is the **Cauchy Point** x^c .

Step 2: Minimization in the Subspace

Once x^c is computed, we work on the active set:

$$\mathcal{A}(x^c) = \{i \mid x_i^c = l_i \text{ or } x_i^c = u_i\} \quad (11.50)$$

We approximately solve the QP problem by **fixing** x_i to x_i^c for $i \in \mathcal{A}(x^c)$. The other components are determined by:

$$\min_{x \in \mathbb{R}^n} q(x) = \frac{1}{2}x^\top Gx + c^\top x \quad (11.51)$$

subject to:

$$\begin{cases} x_i = x_i^c & \text{if } i \in \mathcal{A}(x^c) \\ l_i \leq x_i \leq u_i & \text{if } i \notin \mathcal{A}(x^c) \end{cases} \quad (11.52)$$

Remark 11.10. It is not necessary (nor desirable) to solve this subproblem exactly. We only need to find a point that is better than x^c . For example, we can use a Conjugate Gradient algorithm (as in Section 2) and stop as soon as the constraints $l_i \leq x_i \leq u_i$ for $i \notin \mathcal{A}(x^c)$ are satisfied.

Remark 11.11. In principle, the algorithm can be used for general inequality constraints $a_i^\top x \geq b_i$, but significantly more complex calculations are needed. For instance, to find the projection of a point x onto the feasible set, one must solve:

$$\min_{\bar{x} \in \mathbb{R}^n} \|x - \bar{x}\|^2, \quad \text{subject to } a_i^\top \bar{x} \geq b_i, \forall i \in \mathcal{I} \quad (11.53)$$

which can be as difficult as the original problem.

Solutions: resort to active-set methods or interior-point methods.

Summary

Algorithm 19: Projected gradient method for Quadratic Programming

```

1 Compute a feasible starting point  $x_0$ ;
2 for  $k = 0, 1, \dots$  do
3   if  $x_k$  satisfies the KKT conditions for the bound-constrained problem then
4     stop with solution  $x^* = x_k$ ;
5   Set  $x = x_k$  and find the Cauchy point  $x^c$ ;
6   Find an approximate solution  $x^+$  to
      
$$\begin{aligned} \min_x q(x) &= \frac{1}{2}x^\top Gx + x^\top c \\ \text{subject to } x_i &= x_i^c, \quad i \in \mathcal{A}(x^c) \\ l_i &\leq x_i \leq u_i, \quad i \notin \mathcal{A}(x^c) \end{aligned}$$

      such that  $q(x^+) \leq q(x^c)$  and  $x^+$  is feasible;
7    $x_{k+1} \leftarrow x^+$ ;
```

11.4 Practice Exercises (Nocedal & Wright)

These exercises correspond to chapter 16 (Quadratic Programming) of the reference book.

Exercise 11.1 (KKT Conditions for QP). Write the KKT conditions for the convex quadratic program with mixed constraints:

$$\begin{aligned} \min_x \quad & \frac{1}{2}x^\top Gx + x^\top c \\ \text{s.t.} \quad & Ax \geq b \\ & \bar{A}x = \bar{b} \end{aligned}$$

Use these conditions to derive an analogue of the generic primal-dual system.

Suggested Correction. Lagrangian:

$$L(x, \lambda, \mu) = \frac{1}{2}x^\top Gx + c^\top x - \lambda^\top (Ax - b) - \nu^\top (\bar{A}x - \bar{b}).$$

KKT Conditions: 1. Stationarity: $Gx + c - A^\top \lambda - \bar{A}^\top \nu = 0$. 2. Primal Feasibility: $Ax \geq b$, $\bar{A}x = \bar{b}$. 3. Dual Feasibility: $\lambda \geq 0$. 4. Complementarity: $\lambda_i(A_i x - b_i) = 0, \forall i$.

Exercise 11.2 (Resolution by Active-Set). Program (or manually unroll) the Active-Set algorithm to solve:

$$\begin{aligned} \min_{x_1, x_2} \quad & x_1^2 + 2x_2^2 - 2x_1 - 6x_2 - 2x_1x_2 \\ \text{s.t.} \quad & \frac{1}{2}x_1 + \frac{1}{2}x_2 \leq 1 \\ & -x_1 + 2x_2 \leq 2 \\ & x_1 \geq 0, x_2 \geq 0 \end{aligned}$$

Test with several starting points (interior, vertex, non-vertex boundary).

Suggested Correction. This is an implementation exercise for Algorithm 16.3. You have to start from a feasible point x_0 , identify the active set $\mathcal{W}_0$. Solve the equality subproblem. Compute multipliers. If $\lambda_i < 0$ for an active constraint, relax it. (Specific solution depends on the chosen starting point).

Exercise 11.3 (Graphical Example and Resolution). Consider the QP problem:

$$\begin{aligned} \max \quad & 6x_1 + 4x_2 - 13 - x_1^2 - x_2^2 \\ \text{s.t.} \quad & x_1 + x_2 \leq 3 \\ & x_1 \geq 0, x_2 \geq 0 \end{aligned}$$

1. Solve this problem graphically.
2. Apply the Active-Set method to find the solution.

Suggested Correction. Minimize $f(x) = (x_1 - 3)^2 + (x_2 - 2)^2$ (by completing the squares and changing the sign of max). The unconstrained minimum is $(3, 2)$. Constraint $x_1 + x_2 \leq 3$. The point $(3, 2)$ has $3 + 2 = 5 > 3$, so it is not feasible. The point closest to $(3, 2)$ on the line $x_1 + x_2 = 3$ (in the positive quadrant) is the solution. Projection of $(3, 2)$ onto $x_1 + x_2 - 3 = 0$: Normal vector $(1, 1)$. Line passing through $(3, 2)$: $(3 + t, 2 + t)$. Intersection: $(3 + t) + (2 + t) = 3 \implies 2t + 5 = 3 \implies 2t = -2 \implies t = -1$. Projected point: $(2, 1)$. This point satisfies $x \geq 0$. So $x^* = (2, 1)$.

Exercise 11.4 (Null-Space Method (p_Y, p_Z) - Exam Type). Consider the following equality QP problem (analogous to one iteration of an EQP):

$$\begin{aligned} \min_p \quad & \frac{1}{2} p^\top G p + g^\top p \\ \text{s.t.} \quad & A p = -h \end{aligned}$$

with $G = \begin{pmatrix} 4 & 0 \\ 0 & 2 \end{pmatrix}$, $A = \begin{pmatrix} 1 & 1 \end{pmatrix}$, $h = -2$, $g = \begin{pmatrix} -1 \\ -1 \end{pmatrix}$.

1. Construct a matrix Z (basis of the null space of A) and a matrix Y (such that AY is invertible, or pseudo-inverse).
2. Write the decomposition $p = Y p_Y + Z p_Z$.
3. Calculate p_Y by solving $AY p_Y = -h$.
4. Write the reduced system for p_Z : $(Z^\top G Z) p_Z = -(Z^\top g + Z^\top G Y p_Y)$ and solve it.
5. Deduce the optimal step p .

Suggested Correction. 1. Construction of Y and Z : Linear constraint $p_1 + p_2 = 2$ (since $A p = -(-2) = 2$). $A = \begin{pmatrix} 1 & 1 \end{pmatrix}$. Basis of the null space ($\ker A$): we look for z such that $z_1 + z_2 = 0$. Let's set $Z = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$. For Y , we look for y such that $A y = I$ (here 1). Let's choose $Y = \begin{pmatrix} 1/2 \\ 1/2 \end{pmatrix}$ (Pseudo-inverse) or $Y = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$. Let's take the simple choice from the course (often $Y = A^\top (A A^\top)^{-1}$). $A A^\top = 2$, so $Y = \begin{pmatrix} 1/2 \\ 1/2 \end{pmatrix}$. Check: $A Z = 1(1) + 1(-1) = 0$. $A Y = 1(1/2) + 1(1/2) = 1$. OK.

2. Decomposition: $p = Y p_Y + Z p_Z$.

3. Calculation of p_Y : $AY p_Y = -h \implies 1 \cdot p_Y = 2 \implies p_Y = 2$. So the range space component contributes $Y p_Y = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$. The point $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ indeed satisfies the constraint $1 + 1 = 2$.

4. Calculation of p_Z : Reduced matrix $Z^\top G Z = \begin{pmatrix} 1 & -1 \end{pmatrix} \begin{pmatrix} 4 & 0 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 1 & -1 \end{pmatrix} \begin{pmatrix} 4 \\ -2 \end{pmatrix} = 4 + 2 = 6$. Reduced vector $Z^\top g = \begin{pmatrix} 1 & -1 \end{pmatrix} \begin{pmatrix} -1 \\ -1 \end{pmatrix} = -1 + 1 = 0$. Cross term $Z^\top G Y p_Y = \begin{pmatrix} 1 & -1 \end{pmatrix} \begin{pmatrix} 4 & 0 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & -1 \end{pmatrix} \begin{pmatrix} 4 \\ 2 \end{pmatrix} = 4 - 2 = 2$. Reduced equation: $6 p_Z = -(0 + 2) \implies p_Z = -2/6 = -1/3$.

5. Solution p : $p = \begin{pmatrix} 1 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ -1 \end{pmatrix} (-1/3) = \begin{pmatrix} 1 - 1/3 \\ 1 + 1/3 \end{pmatrix} = \begin{pmatrix} 2/3 \\ 4/3 \end{pmatrix}$. Check: $p_1 + p_2 = 6/3 = 2$. OK.

Checkpoint : Did you understand the essentials ?

- **QP KKT:** Write the KKT system for an equality-constrained QP problem.
- **Null Space:** How to decompose the step p into p_Y and p_Z ? Which one minimizes the reduced quadratic?
- **Active Set:** How does this method handle inequalities? (By adding/removing constraints from the working set).

Chapter 12

Penalty and Augmented Lagrangian Methods

12.1 The Quadratic Penalty Method

12.1.1 Motivation

Idea: Replace constraints by **non-negative penalties** that are added to the objective function.

Consider the general problem:

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad \begin{cases} c_i(x) = 0, & i \in \mathcal{E} \\ c_i(x) \geq 0, & i \in \mathcal{I} \end{cases} \quad (12.1)$$

Define the **quadratic penalty function** as:

$$Q(x; \mu) = f(x) + \frac{\mu}{2} \sum_{i \in \mathcal{E}} c_i(x)^2 + \frac{\mu}{2} \sum_{i \in \mathcal{I}} [c_i(x)^-]^2 \quad (12.2)$$

where $x^- = \max(-x, 0)$ and $\mu > 0$ is the **penalty parameter**.

The idea is to replace the constrained problem with an **unconstrained** problem:

$$\min_{x \in \mathbb{R}^n} Q(x; \mu) \quad (12.3)$$

The objective function of this subproblem contains:

- the original objective function $f(x)$;
- an additional term for each constraint, which is positive if the constraint is violated, and zero otherwise.

Remark 12.1. Because of the inequality constraints, the new problem is **less smooth** due to the max function.

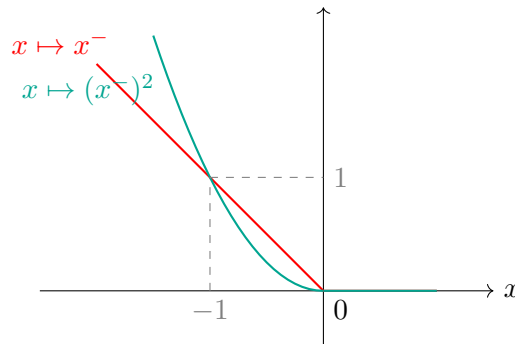


Figure 12.1: Comparison of penalty functions for an inequality $x \geq 0$. In red, the exact penalty x^- (non-differentiable at 0). In green, the quadratic penalty $(x^-)^2$ (differentiable, C^1).

The linear penalty x^- has a kink at the origin (non-differentiable), while $(x^-)^2$ is smooth and tangent to the axis. This differentiability allows the application of gradient methods.

Example: Let $\min_{x \in \mathbb{R}^2} x_1 + x_2$ subject to $x_1^2 + x_2^2 - 2 = 0$. The quadratic penalty function is:

$$Q(x; \mu) = x_1 + x_2 + \frac{\mu}{2}(x_1^2 + x_2^2 - 2)^2 \quad (12.4)$$

The gradient vanishes when (with $x_1 = x_2$ by symmetry):

$$1 + 4\mu x_1(x_1^2 - 1) = 0 \quad (12.5)$$

There exists a local minimum that tends to $(-1, -1)$ as $\mu \rightarrow \infty$.

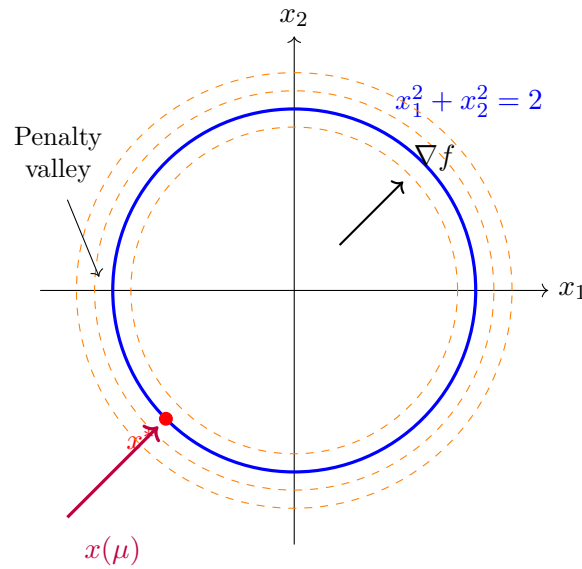


Figure 12.2: Penalty method: As μ increases, the function $Q(x; \mu)$ forms a steep "valley" along the constraint $c(x) = 0$. The sequence of minimizers $x(\mu)$ (purple arrow) descends to the bottom of this valley while approaching the constraint, finally ending at x^* .

The penalty "valley": Imagine the constraint (the blue circle) is the bottom of a canyon.

- Far from the circle, the penalty term $\frac{\mu}{2}c(x)^2$ is huge: the walls of the canyon are very high.
- To minimize Q , one must descend to the bottom of the canyon ($c(x) \approx 0$).
- Once at the bottom, we try to minimize the original function $f(x)$ by moving *along* the constraint.
- Numerical problem: The bottom is somewhat "flat" in the constraint direction, but the walls are very steep. This creates an ill-conditioned ("long and narrow") Hessian matrix.

Algorithm 20: Quadratic Penalty Method

-
- 1 Given: $\mu_0 > 0$, a nonnegative sequence $\{\tau_k\}$ with $\tau_k \rightarrow 0$, and a starting point x_0^s ;
 - 2 **for** $k = 0, 1, \dots$ **do**
 - 3 Find an approximate minimizer x_k of $Q(\cdot; \mu_k)$, starting at x_k^s , and terminating when
 $\|\nabla_x Q(x; \mu_k)\| \leq \tau_k$;
 - 4 **if** *final convergence test satisfied* **then**
 - 5 **stop** with approximate solution x_k ;
 - 6 Choose new penalty parameter $\mu_{k+1} > \mu_k$;
 - 7 Choose new starting point x_{k+1}^s ;
-

12.1.2 Convergence of the Method (with Equality Constraints)

Theorem 12.2 (Global Convergence). *Suppose that each x_k is the exact minimizer of $Q(\cdot; \mu_k)$ and that $\mu_k \uparrow \infty$. Then, any limit point x^* of the sequence $\{x_k\}$ is a global solution to the problem.*

Proof. Let $\bar{x}$ be a global solution to the constrained problem. Then $\bar{x}$ is feasible, i.e., $c_i(\bar{x}) = 0$ for all $i \in \mathcal{E}$. By definition of x_k as the minimizer of $Q(\cdot; \mu_k)$, we have:

$$Q(x_k; \mu_k) \leq Q(\bar{x}; \mu_k).$$

Expanding the expression of the penalty function:

$$f(x_k) + \frac{\mu_k}{2} \sum_{i \in \mathcal{E}} c_i(x_k)^2 \leq f(\bar{x}) + \frac{\mu_k}{2} \sum_{i \in \mathcal{E}} c_i(\bar{x})^2.$$

Since $\bar{x}$ is feasible, the penalty term for $\bar{x}$ is zero:

$$f(x_k) + \frac{\mu_k}{2} \sum_{i \in \mathcal{E}} c_i(x_k)^2 \leq f(\bar{x}).$$

Rearranging this inequality to isolate the constraint violation term:

$$\sum_{i \in \mathcal{E}} c_i(x_k)^2 \leq \frac{2}{\mu_k} [f(\bar{x}) - f(x_k)].$$

Suppose x^* is a limit point of the sequence $\{x_k\}$. There exists a subsequence $\mathcal{K}$ such that:

$$\lim_{k \in \mathcal{K}} x_k = x^*.$$

Take the limit in the previous inequality for $k \in \mathcal{K}$ tending to infinity. Since $\mu_k \rightarrow \infty$, the right-hand side tends to 0 (assuming $f(x_k)$ is bounded or converges to $f(x^*)$):

$$\sum_{i \in \mathcal{E}} c_i(x^*)^2 = \lim_{k \in \mathcal{K}} \sum_{i \in \mathcal{E}} c_i(x_k)^2 \leq 0.$$

Since a sum of squares is always non-negative, we deduce that $\sum c_i(x^*)^2 = 0$, and thus:

$$c_i(x^*) = 0, \quad \forall i \in \mathcal{E}.$$

Therefore, x^* is a feasible point.

Moreover, returning to the inequality $f(x_k) + \text{penalty} \leq f(\bar{x})$. Since the penalty term is non-negative, we have simply:

$$f(x_k) \leq f(\bar{x}).$$

Passing to the limit over the subsequence $\mathcal{K}$:

$$f(x^*) \leq f(\bar{x}).$$

Since x^* is feasible and its objective value is less than or equal to that of the global solution $\bar{x}$, x^* is itself a global solution. $\square$

Remark 12.3. In practice, finding the exact minimizer is not realistic at every step.

Theorem 12.4 (KKT Convergence). *Suppose that the tolerances and penalties satisfy $\tau_k \downarrow 0$ and $\mu_k \uparrow \infty$. Then, if a limit point x^* of the sequence $\{x_k\}$ is **infeasible**, then x^* is a stationary point of the function $x \mapsto \|c(x)\|^2 = \sum_{i \in \mathcal{E}} [c_i(x)]^2$. On the other hand, if x^* is **feasible** and if the constraint gradients $\nabla c_i(x^*)$ are linearly independent (LICQ), then x^* is a KKT point for the problem.*

Remark 12.5 (Lagrange Multipliers). For such points, for any index subsequence $\mathcal{K}$ such that $x^* = \lim_{k \in \mathcal{K}} x_k$, we have:

$$\lim_{k \in \mathcal{K}} (-\mu_k c_i(x_k)) = \lambda_i^*, \quad \text{for all } i \in \mathcal{E},$$

where λ^* are the Lagrange multipliers associated with x^* for the original problem.

Proof. See Nocedal and Wright. $\square$

Remark 12.6 (Ill-conditioning of the Hessian). The Hessian of the penalty function is given by:

$$\nabla_{xx}^2 Q(x; \mu_k) = \nabla^2 f(x) + \sum_{i \in \mathcal{E}} \mu_k c_i(x) \nabla^2 c_i(x) + \mu_k A(x)^\top A(x),$$

where $A(x)^\top = [\nabla c_i(x)]_{i \in \mathcal{E}}$.

When x is close to the minimizer of $Q(\cdot; \mu_k)$, the conditions of the theorem are satisfied and we have the approximation:

$$\nabla_{xx}^2 Q(x; \mu_k) \approx \nabla_{xx}^2 \mathcal{L}(x, \lambda^*) + \mu_k A(x)^\top A(x).$$

The matrix $\nabla_{xx}^2 \mathcal{L}(x, \lambda^*)$ is independent of μ_k . The term $\mu_k A(x)^\top A(x)$ has rank $|\mathcal{E}|$ (since $A(x)$ has $|\mathcal{E}|$ rows which are the assumed independent gradients).

Thus, the Hessian has $|\mathcal{E}|$ eigenvalues of order μ_k (from $A^\top A$) and $n - |\mathcal{E}|$ eigenvalues of order 1 (from the Lagrangian). This matrix is singular when $|\mathcal{E}| < n$ for the dominant term.

Since $\mu_k \rightarrow \infty$, the condition number of the matrix explodes: $\nabla_{xx}^2 Q(\cdot; \mu_k)$ becomes ill-conditioned. This makes minimization by Newton or quasi-Newton methods difficult.

Consequence on the computation of the Newton step: We seek to solve $\nabla_{xx}^2 Q(x; \mu_k) p_N = -\nabla_x Q(x; \mu_k)$. To avoid numerical errors due to ill-conditioning, an auxiliary variable $\zeta = \mu_k A(x) p_N$ is introduced. The system can then be rewritten in the form:

$$\begin{pmatrix} \nabla^2 f(x) + \sum_{i \in \mathcal{E}} \mu_k c_i(x) \nabla^2 c_i(x) & A(x)^\top \\ A(x) & -\frac{1}{\mu_k} I \end{pmatrix} \begin{pmatrix} p_N \\ \zeta \end{pmatrix} = \begin{pmatrix} -\nabla_x Q(x; \mu_k) \\ 0 \end{pmatrix}$$

This formulation avoids explicitly forming the ill-conditioned matrix $\mu_k A^\top A$.

12.2 Nonsmooth Penalty Functions (l^1)

12.2.1 Definition and Properties

A popular alternative is the l^1 penalty function, defined by:

$$\phi_1(x; \mu) = f(x) + \mu \sum_{i \in \mathcal{E}} |c_i(x)| + \mu \sum_{i \in \mathcal{I}} [c_i(x)]^-,$$

where $[y]^- = \max(-y, 0)$.

Unlike the quadratic penalty, this function is not differentiable at points where $c_i(x) = 0$. However, it has a very important property: it is an **exact penalty function**.

Theorem 12.7 (Exact Penalty). *Suppose that x^* is a strict local solution of the constrained problem:*

$$\min f(x) \quad \text{s.t.} \quad c_i(x) = 0, \quad i \in \mathcal{E}, \quad c_i(x) \geq 0, \quad i \in \mathcal{I},$$

and that the first-order KKT conditions are satisfied for some multipliers λ^ .*

Then, x^ is a local minimum of $\phi_1(\cdot; \mu)$ for all $\mu > \mu^*$, where*

$$\mu^* = \|\lambda^*\|_\infty = \max_i |\lambda_i^*|.$$

If the second-order sufficient conditions are also satisfied, then x^ is a **strict** local minimum.*

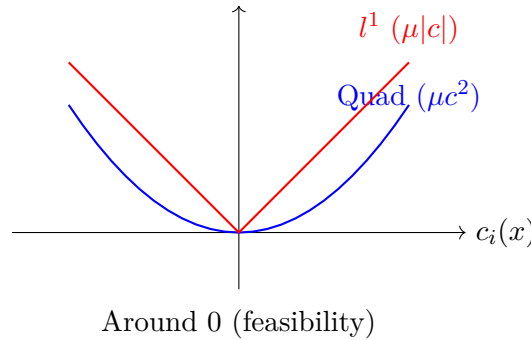


Figure 12.3: Comparison: The quadratic penalty (blue) is smooth but has zero slope at the origin, requiring $\mu \rightarrow \infty$ to force $c(x) \rightarrow 0$. The l^1 penalty (red) has a non-zero slope ("kink") at the origin, allowing an exact solution for finite μ .

12.2.2 Characterization of Behavior

We wish to study the stationary points of ϕ_1 , which is not differentiable. For this, we use directional derivatives.

Definition 12.8 (Directional derivative). The directional derivative of $f : \mathbb{R}^n \rightarrow \mathbb{R}$ in the direction $p \in \mathbb{R}^n$ is given by:

$$D(f(x), p) = \lim_{\epsilon \rightarrow 0} \frac{f(x + \epsilon p) - f(x)}{\epsilon}.$$

Definition 12.9 (Stationarity). A point $\hat{x} \in \mathbb{R}^n$ is a stationary point for the penalty function $\phi_1(\cdot; \mu)$ if:

$$D(\phi_1(\hat{x}; \mu), p) \geq 0, \quad \text{for all } p \in \mathbb{R}^n.$$

We also define an infeasibility measure:

$$h(x) = \sum_{i \in \mathcal{E}} |c_i(x)| + \sum_{i \in \mathcal{I}} [c_i(x)]^-.$$

A point $\hat{x}$ is a "stationary point for the infeasibility measure" if $D(h(\hat{x}); p) \geq 0$ for all $p \in \mathbb{R}^n$. If in addition x is infeasible for the original problem, we call it an **infeasible stationary point**.

Theorem 12.10. *Suppose that $\hat{x}$ is a stationary point for the penalty function $\phi_1(\cdot; \mu)$ for all $\mu > \mu^*$. Then:*

- *If $\hat{x}$ is feasible for the problem, then it satisfies the KKT conditions.*
- *Otherwise, it is an infeasible stationary point.*

Example 12.11. Consider the problem:

$$\min_{x \in \mathbb{R}^2} x_1 + x_2 \quad \text{s.t.} \quad x_1^2 + x_2^2 - 2 = 0.$$

The l^1 penalty function is:

$$\phi_1(x; \mu) = x_1 + x_2 + \mu |x_1^2 + x_2^2 - 2|.$$

It can be shown that the minimizer of $\phi_1(\cdot; \mu)$ is the solution $(-1, -1)$ for all $\mu > \mu^* = 0.5$.

12.3 Practical Approach: Modified l^1 Model

Since $\phi_1(\cdot; \mu)$ is nonsmooth, one cannot directly apply Newton's method. A practical approach (Framework 20) consists of forming a modified model by:

- Linearizing the constraints c_i .
- Replacing f by a quadratic approximation.

We then seek to minimize with respect to p the function:

$$q(p; \mu) = f(x) + \nabla f(x)^\top p + \frac{1}{2} p^\top W p + \mu \sum_{i \in \mathcal{E}} |c_i(x) + \nabla c_i(x)^\top p| + \mu \sum_{i \in \mathcal{I}} [c_i(x) + \nabla c_i(x)^\top p]^-,$$

where W is an approximation of the Hessian of the Lagrangian. This model $q(p; \mu)$ is nonsmooth. However, if W is positive definite, this subproblem can be reformulated as a smooth quadratic program (QP) by introducing artificial variables r, s, t :

$$\min_{p, r, s, t} \nabla f(x)^\top p + \frac{1}{2} p^\top W p + \mu \sum_{i \in \mathcal{E}} (r_i + s_i) + \mu \sum_{i \in \mathcal{I}} t_i$$

subject to the constraints:

$$\begin{cases} \nabla c_i(x)^\top p + c_i(x) = r_i - s_i, & i \in \mathcal{E} \\ \nabla c_i(x)^\top p + c_i(x) \geq -t_i, & i \in \mathcal{I} \\ r, s, t \geq 0 \end{cases}$$

This problem can be efficiently solved by standard QP solvers.

12.4 Augmented Lagrangian Methods

12.4.1 Definition and Motivation

We have seen that for the quadratic penalty, the sequence $\{x_k\}$ converges to a solution, but it requires that $\mu_k \rightarrow \infty$. This leads to ill-conditioning of the Hessian. Moreover, we have the asymptotic relationship:

$$\lim_{k \rightarrow \infty} -\mu_k c_i(x_k) = \lambda_i^*.$$

The idea of Augmented Lagrangian methods is to use an explicit estimate of the Lagrange multipliers λ to avoid having to send μ to infinity too quickly.

We define the **Augmented Lagrangian** $\mathcal{L}_A$ by:

$$\mathcal{L}_A(x, \lambda; \mu) = f(x) - \sum_{i \in \mathcal{E}} \lambda_i c_i(x) + \frac{\mu}{2} \sum_{i \in \mathcal{E}} c_i(x)^2.$$

This function combines the Lagrangian and the quadratic penalty.

12.4.2 Algorithm (Method of Multipliers)

The proposed algorithm is as follows:

1. At iteration k , we fix an estimate of the multipliers λ^k and a penalty $\mu_k > 0$.
2. We seek a minimizer x_k of $\mathcal{L}_A(\cdot, \lambda^k; \mu_k)$. The optimality condition is:

$$\nabla_x \mathcal{L}_A(x_k, \lambda^k; \mu_k) = \nabla f(x_k) - \sum_{i \in \mathcal{E}} (\lambda_i^k - \mu_k c_i(x_k)) \nabla c_i(x_k) \approx 0.$$

3. By comparing with the KKT condition $\nabla f(x^*) - \sum \lambda_i^* \nabla c_i(x^*) = 0$, this suggests the following update rule for the multipliers:

$$\lambda_i^{k+1} = \lambda_i^k - \mu_k c_i(x_k).$$

Comparison of behavior:

- **Before (Quadratic Penalty only):** We had $-\mu_k c_i(x_k) \approx \lambda_i^*$, which implied $c_i(x_k) \approx -\frac{\lambda_i^*}{\mu_k}$. To have c_i small, an extremely large μ_k was needed.
- **With Augmented Lagrangian:** The condition becomes $\lambda_i^k - \mu_k c_i(x_k) \approx \lambda_i^*$. Thus:

$$c_i(x_k) \approx \frac{\lambda_i^k - \lambda_i^*}{\mu_k}.$$

If λ^k converges to λ^* , the numerator tends to 0. We can therefore obtain a small constraint violation $c_i(x_k)$ even without μ_k going to infinity.

12.5 Augmented Lagrangian Method

12.5.1 Properties of the equality-constrained case

Algorithm 21: Augmented Lagrangian method for equality constraints

- 1 Given $\mu_0 > 0$, tolerance $\tau_0 > 0$, starting points x_0^s and λ^0 ;
 - 2 **for** $k = 0, 1, \dots$ **do**
 - 3 Find an approximate minimizer x_k of $\mathcal{L}_A(\cdot, \lambda^k; \mu_k)$, starting at x_k^s , and terminating when $\|\nabla_x \mathcal{L}_A(x_k, \lambda^k; \mu_k)\| \leq \tau_k$;
 - 4 **if** a convergence test for the equality-constrained problem is satisfied **then**
 - 5 **stop** with approximate solution x_k ;
 - 6 Update Lagrange multipliers using

$$\lambda_i^{k+1} = \lambda_i^k - \mu_k c_i(x_k), \text{ for all } i \in \mathcal{E};$$

Choose new penalty parameter $\mu_{k+1} \geq \mu_k$;
 - 7 Set starting point for the next iteration to $x_{k+1}^s = x_k$;
 - 8 Select tolerance τ_{k+1} ;
-

12.5.2 Practical Augmented Lagrangian Methods

Formulation with bound constraints

Algorithm 22: Bound-constrained Lagrangian method

- 1 Choose an initial point x_0 and initial multipliers λ^0 ;
 - 2 Choose convergence tolerances η_* and ω_* ;
 - 3 Set $\mu_0 = 10$, $\omega_0 = \frac{1}{\mu_0}$, and $\eta_0 = \frac{1}{\mu_0^{0.1}}$;
 - 4 **for** $k = 0, 1, \dots$ **do**
 - 5 Find an approximate solution x_k of the problem

$$\min_x \mathcal{L}_A(x, \lambda^k; \mu_k) \text{ subject to } l \leq x \leq u$$

such that

$$\|x_k - P(x_k - \nabla_x \mathcal{L}_A(x_k, \lambda^k; \mu_k), l, u)\| \leq \omega_k;$$
 - 6 **if** $\|c(x_k)\| \leq \eta_k$ **then**
 - ▷ **test for convergence**
 - 7 **if** $\|c(x_k)\| \leq \eta_*$ **and** $\|x_k - P(x_k - \nabla_x \mathcal{L}_A(x_k, \lambda^k; \mu_k), l, u)\| \leq \omega_*$ **then**
 - ▷ **stop** with approximate solution x_k ;
 - ▷ **update multipliers, tighten tolerances**
 - 8 $\lambda^{k+1} \leftarrow \lambda^k - \mu_k c(x_k)$;
 - 9 $\mu_{k+1} \leftarrow \mu_k$;
 - 10 $\eta_{k+1} \leftarrow \frac{\eta_k}{\mu_{k+1}^{0.9}}$;
 - 11 $\omega_{k+1} \leftarrow \frac{\omega_k}{\mu_{k+1}}$;
 - 12 **else**
 - ▷ **increase penalty parameters, tighten tolerances**
 - 13 $\lambda^{k+1} \leftarrow \lambda^k$;
 - 14 $\mu_{k+1} \leftarrow 100\mu_k$;
 - 15 $\eta_{k+1} \leftarrow \frac{1}{\mu_{k+1}^{0.1}}$;
 - 16 $\omega_{k+1} \leftarrow \frac{1}{\mu_{k+1}}$;
-

12.6 Practice Exercises (Nocedal & Wright)

Exercises related to Chapter 17 of the book (Penalty and Augmented Lagrangian Methods).

Exercise 12.1 (Convergence of the quadratic penalty method). Verify that if x^* is a solution to the constrained problem, then it is a stationary point of the quadratic penalty function $Q(x; \mu)$ as $\mu \rightarrow \infty$. (See Theorem 17.1 and Exercise 17.1 of the book).

Suggested Correction. $\nabla Q(x; \mu) = \nabla f(x) + \mu \sum c_i(x) \nabla c_i(x)$. For $\nabla Q \rightarrow 0$, ∇f must be balanced by the penalty term. In the limit, $x_k \rightarrow x^*$. If x^* is feasible, $c_i(x^*) = 0$. The limit relationship gives $\nabla f(x^*) + \sum \lambda_i^* \nabla c_i(x^*) = 0$ (KKT), where $\lambda_i^* \approx -\mu c_i(x_k)$.

Exercise 12.2 (Comparison of Augmented Lagrangian vs Penalty). Study Example 17.4 (page 536 of the Nocedal/Wright book) which compares the conditioning of the two methods for the problem:

$$\min x_1 + x_2 \quad \text{s.t.} \quad x_1^2 + x_2^2 - 2 = 0$$

Show that the Augmented Lagrangian converges better and with better conditioning than the simple penalty.

Suggested Correction. The circle $x_1^2 + x_2^2 = 2$ is the constraint. Solution $x^* = (-1, -1)$. Simple Penalty: The contours become very elongated ("valley") around the constraint, making minimization difficult (ill-conditioning of the Hessian). Augmented Lagrangian: The linear term $-\lambda^\top c(x)$ "shifts" the bottom of the valley so that it coincides with the constraint without needing an immense μ . This allows finding the precise solution with a moderate μ , thus with better conditioning.

Exercise 12.3 (Explicit Computation - Penalty Method). Consider the scalar problem:

$$\min_{x \in \mathbb{R}} \frac{1}{2} x^2 \quad \text{s.t.} \quad x = 1$$

1. Write the quadratic penalty function $Q(x; \mu)$.
2. Find the minimizer x_μ of $Q(x; \mu)$ for a given $\mu > 0$.
3. Verify that $x_\mu \rightarrow x^*$ (the exact solution) as $\mu \rightarrow \infty$.
4. Estimate the Lagrange multiplier λ^* from x_μ and verify its convergence.

Suggested Correction. **1. Penalty Function:** $Q(x; \mu) = \frac{1}{2} x^2 + \frac{\mu}{2} (x - 1)^2$.

2. Minimizer x_μ : $Q'(x) = x + \mu(x - 1) = 0$. $x(1 + \mu) = \mu \implies x_\mu = \frac{\mu}{1 + \mu}$.

3. Primal Convergence: $\lim_{\mu \rightarrow \infty} x_\mu = \lim_{\mu \rightarrow \infty} \frac{\mu}{1 + \mu} = 1$. The exact solution to the problem is obviously $x^* = 1$. Convergence verified.

4. Dual Multiplier: Exact KKT: $L = \frac{1}{2} x^2 - \lambda(x - 1)$. $\nabla L = x - \lambda = 0 \implies \lambda^* = x^* = 1$. Penalty estimate (Formula $\lambda \approx -\mu c(x)$ for $L = f - \lambda c$): $\lambda_\mu = -\mu(x_\mu - 1) = -\mu(\frac{\mu}{1 + \mu} - 1) = -\mu(\frac{\mu - (1 + \mu)}{1 + \mu}) = -\mu(\frac{-1}{1 + \mu}) = \frac{\mu}{1 + \mu}$. As $\mu \rightarrow \infty$, $\lambda_\mu \rightarrow 1$. Convergence verified towards $\lambda^* = 1$.

Checkpoint : Did you understand the essentials ?

- **Quadratic Penalty:** Why is $\mu \rightarrow \infty$ problematic? (Conditioning of the Hessian).
- **l^1 Penalty:** What is its main advantage over the quadratic one? (Exact penalty for finite μ).
- **Disadvantage of l^1 :** What is the price to pay for exactness? (Non-differentiable function at the solution).
- **Augmented Lagrangian:** What is the main advantage over the simple penalty? (Convergence of multipliers without infinite μ).
- **Update:** How is λ updated in the Augmented Lagrangian method? ($\lambda \leftarrow \lambda - \mu c(x)$).

Bibliography

- [1] Reeves Fletcher and Colin M Reeves. Function minimization by conjugate gradients. *The computer journal*, 7(2):149–154, 1964.
- [2] Philip E Gill, Walter Murray, and Margaret H Wright. *Practical optimization*. SIAM, 2019.
- [3] Magnus R Hestenes, Eduard Stiefel, et al. Methods of conjugate gradients for solving linear systems. *Journal of research of the National Bureau of Standards*, 49(6):409–436, 1952.
- [4] Carl Tim Kelley. Detection and remediation of stagnation in the Nelder–Mead algorithm using a sufficient decrease condition. *SIAM journal on optimization*, 10(1):43–55, 1999.
- [5] JC Lagarias, JA Reeds, MH Wright, and PE Wright. Convergence properties of the Nelder–Mead algorithm in low dimensions. *AT&T Bell Laboratories, Murray Hill, NJ, Technical Report*, 1995.
- [6] David G Luenberger. *Introduction to Linear and Nonlinear Programming*. Addison-Wesley, 1984.
- [7] Jorge J Moré and Danny C Sorensen. Newton’s method. Technical report, Argonne National Lab.(ANL), Argonne, IL (United States), 1982.
- [8] Jorge Nocedal. Updating quasi-Newton matrices with limited storage. *Mathematics of computation*, 35(151):773–782, 1980.
- [9] Jorge Nocedal and Stephen J Wright. *Numerical optimization*. Springer, 2006.